CareerPilot AI

Complete Technical Project Documentation,
Architecture, AI/ML Methodology
& Jury Preparation Guide

Agentic AI-Based Placement Preparation and Career Intelligence Platform

Every statement in this document was checked against the project repository.
Each feature carries a status: IMPLEMENTED, PARTIALLY IMPLEMENTED, PLANNED or NOT VERIFIED.
No metric is reported unless it exists in the project's own evaluation files.

Stack: React + TypeScript | Node.js + Express | MongoDB | Python FastAPI | Docker judge
Repository state documented: 8 October 2026

Table of contents

1. Executive summary **6**

1.1 What CareerPilot AI is . . . 6

1.2 How to read the status labels . . . 6

1.3 Status of the headline features . . . 6

2. Problem statement and proposed system **8**

2.1 The problem . . . 8

2.2 Existing approach and its limits . . . 8

2.3 Proposed system . . . 8

2.4 Objectives . . . 9

2.5 Users and roles . . . 9

2.6 Functional requirements (as built) . . . 10

2.7 Non-functional requirements . . . 10

3. System architecture **11**

3.1 High-level architecture . . . 11

3.2 Why three services instead of one . . . 11

3.3 Detailed request path . . . 12

3.4 Layer responsibilities . . . 12

3.5 Real-time and background behaviour . . . 13

4. Technology stack **14**

5. API architecture **16**

5.1 Principal endpoints . . . 16

5.2 Endpoint count by group . . . 17

6. Database architecture **18**

6.1 Collections by purpose . . . 18

6.2 Relationships . . . 19

6.3 Schemas worth knowing . . . 19

6.4 Why MongoDB . . . 19

7. Security architecture **20**

7.1 AI-specific security . . . 21

8. Deployment architecture **22**

8.1 Environment variables (names only) . . . 22

9. Functional modules **24**

9.1 Authentication and role-based access . . . 24

9.2 Resume intelligence and ATS scoring . . . 24

9.3 Job matching . . . 26

9.4 Assessment engine . . . 27

9.5 DSA practice and the online judge . . . 27
9.6 AI mock interview . . . 30
9.7 APIE — Placement Readiness Score . . . 30
9.8 Career Plan . . . 32
9.9 Learning-resource recommendation and YouTube integration . . . 33
9.10 RAG knowledge base . . . 34
9.11 Alumni mentorship . . . 34
9.12 Analytics, reports and administration . . . 35
10. AI architecture **36**
10.1 The design rule . . . 36
10.2 Agents and orchestration . . . 36
10.3 The LLM layer . . . 37
11. Algorithms used in CareerPilot **38**
11.1 Summary table . . . 38
11.2 Skill extraction — dictionary matching . . . 38
11.3 Feature hashing embedding . . . 39
11.4 Cosine similarity and vector search . . . 39
11.5 Harmonic recency weighting . . . 39
11.6 Random Forest . . . 39
11.7 Prerequisite graph and topological order . . . 39
11.8 Skill-gap priority score . . . 39
11.9 Item-based collaborative filtering (skill recommender) . . . 40
11.10 Judge comparator and verdict classification . . . 40
12. How the AI models are trained **41**
12.1 Beginner: what is an LLM? . . . 41
12.2 Intermediate: what is fine-tuning? . . . 41
12.3 Advanced: the terms you must be able to define . . . 41
12.4 Full fine-tuning, LoRA and QLoRA compared . . . 41
12.5 The two models . . . 42
12.6 ATS fine-tuning pipeline . . . 43
12.7 Career model fine-tuning pipeline . . . 45
12.8 Inference and fallback . . . 45
13. Evaluation framework **46**
13.1 Choosing the right metric . . . 46
13.2 Formulas . . . 46
13.3 What should be measured for each module . . . 46
14. Metrics and evaluation report **48**
14.1 Measured — ATS deterministic baselines . . . 48
14.2 Measured — ATS dataset . . . 48

14.3 Measured — readiness classifier . . . 48
14.4 Measured — skill recommender . . . 49
14.5 Measured — career model, run v1 . . . 49
14.6 Measured — career knowledge and resources . . . 50
14.7 Measured — RAG threshold calibration . . . 50
14.8 Measured — DSA judge correctness . . . 50
14.9 Measured — automated tests . . . 50
14.10 Not yet measured — and how to measure it . . . 51

15. Performance and speed 52

15.1 Recommended benchmark method . . . 52

16. Novelty, innovation and engineering contribution 54

16.1 What the project contributes . . . 54

17. Comparison with conventional platforms 56

18. Limitations and future work 57

18.1 Current limitations . . . 57
18.2 Future enhancements . . . 57

19. End-to-end demo script 58

19.1 Before you start . . . 58
19.2 Screen-by-screen . . . 58
19.3 If something fails during the demo . . . 59

20. Five-minute presentation 60

20.1 0:00 – 0:30 Problem . . . 60
20.2 0:30 – 1:00 Existing limitations . . . 60
20.3 1:00 – 1:30 Proposed solution . . . 60
20.4 1:30 – 2:30 Architecture . . . 60
20.5 2:30 – 3:30 AI and ML . . . 60
20.6 3:30 – 4:15 Key functionality . . . 60
20.7 4:15 – 4:45 Innovation . . . 60
20.8 4:45 – 5:00 Results and future scope . . . 61

21. Ten-minute technical presentation 62

21.1 Lines worth memorising . . . 62

22. Viva questions and answers 63

22.1 A. Basic project questions . . . 63
22.2 B. Architecture . . . 63
22.3 C. Frontend . . . 64
22.4 D. Backend . . . 65
22.5 E. Database . . . 65
22.6 F. APIs . . . 66

22.7 G. AI and ML . . . 67
22.8 H. LLM . . . 67
22.9 I. Fine-tuning . . . 68
22.10 J. LoRA . . . 69
22.11 K. QLoRA . . . 69
22.12 L. NLP . . . 70
22.13 M. ATS . . . 70
22.14 N. Career plan . . . 71
22.15 O. Recommendation systems . . . 72
22.16 P. YouTube API . . . 72
22.17 Q. DSA judge . . . 73
22.18 R. Docker . . . 73
22.19 S. Security . . . 74
22.20 T. Performance . . . 75
22.21 U. Algorithms . . . 75
22.22 V. Innovation . . . 76
22.23 W. Limitations . . . 76
22.24 X. Future scope . . . 76

23. Questions the jury may ask to challenge the project 78

24. Concepts explained like a professor 81

24.1 Web and system concepts . . . 81
24.2 AI and NLP concepts . . . 81
24.3 Recommendation, similarity and metrics . . . 82

25. Technical glossary 84

26. How to defend CareerPilot AI 87

26.1 What is strongest . . . 87
26.2 What is technically impressive . . . 87
26.3 What to emphasise . . . 87
26.4 What to avoid claiming . . . 87
26.5 Limitations to admit without being asked twice . . . 87
26.6 Handling question types . . . 87

27. Final cheat sheet 89

27.1 The essentials . . . 89
27.2 Formulas to know by heart . . . 89
27.3 Models . . . 89
27.4 DSA judge . . . 89
27.5 Training . . . 90
27.6 Innovation, in one sentence . . . 90
27.7 Future scope, in one sentence . . . 90

28. "What is CareerPilot AI?" at four lengths **91**

28.1 30-second answer . . . 91

28.2 1-minute answer . . . 91

28.3 3-minute answer . . . 91

28.4 5-minute answer . . . 91

29. Top 20 things to remember tomorrow **92**

30. Factual validation report **93**

30.1 Verified . . . 93

30.2 Partially verified . . . 94

30.3 Not verified from source . . . 94

30.4 Planned (not built) . . . 95

30.5 Corrections made while validating . . . 95

30.6 Things this document deliberately does not contain . . . 95

1. Executive summary

1.1 What CareerPilot AI is

CareerPilot AI is a web platform that prepares engineering students for campus placement. One student account covers resume analysis, job matching, aptitude and technical assessments, DSA coding practice, AI mock interviews, a placement readiness score, a personalised career plan and mentorship from verified alumni. Placement officers and administrators see the same data as cohort analytics.

One-sentence technical summary: it is a three-service system (React, Node.js, Python) in which *deterministic algorithms compute every score* and *language models are used only for language* — asking interview questions, judging free-text answers and phrasing advice — with every model output validated and every final score recomputed in code.

1.2 How to read the status labels

Every feature in this document carries one of these labels. Use the same words in front of the jury.

Label	Meaning
IMPLEMENTED	Code exists, runs, and is covered by tests or was exercised end to end
PARTIALLY IMPLEMENTED	Built, but a stated part is missing, switched off, or unverified
PLANNED	Designed or reserved in the code, not built
NOT VERIFIED FROM SOURCE	Could not be confirmed from the repository
NOT YET MEASURED	The measurement has not been run; no number may be quoted

1.3 Status of the headline features

Feature	Status	The one fact to remember
Authentication, roles, audit	**IMPLEMENTED**	JWT with refresh-token rotation; four active roles
Resume parsing and rule-based ATS score	**IMPLEMENTED**	Nine capped criteria summing to 100; no model needed

Feature	Status	The one fact to remember
Compare resume with a job description	**IMPLEMENTED**	Predefined JD or the student's own (pasted or uploaded)
Job matching	**IMPLEMENTED**	Weighted formula; eligibility is a hard gate
Assessment engine	**IMPLEMENTED**	Server-authoritative timer, autosave, server-side grading
AI mock interview (text and voice)	**IMPLEMENTED**	Difficulty adapts; overall score recomputed in code
APIE Placement Readiness Score	**IMPLEMENTED**	Seven weighted components; weights are not validated
Random Forest readiness classifier	**IMPLEMENTED**	Trained on synthetic data; accuracy 0.735
RAG knowledge base	**IMPLEMENTED**	Refuses to answer below a relevance threshold
Career Plan (rule-based planner)	**IMPLEMENTED**	Deterministic; 20 roles, 122 skills, 270 resources
DSA judge: code generation, protocol, verdicts	**IMPLEMENTED**	28 problems, four languages, 112 reference programs accepted
DSA judge: Docker container isolation	**PARTIALLY IMPLEMENTED**	Code complete; not yet run on a machine with Docker
Fine-tuned ATS model (Qwen3-4B, QLoRA)	**PARTIALLY IMPLEMENTED**	Pipeline and integration built; **training not run**
Fine-tuned career model (Qwen3-1.7B, QLoRA)	**PARTIALLY IMPLEMENTED**	v1 trained; **0% fully valid plans; switched off**
Alumni mentorship and WebRTC interview room	**IMPLEMENTED**	Human feedback feeds the readiness score
Institutional analytics and PDF reports	**IMPLEMENTED**	Ten report types
Recruiter role	**PLANNED**	The role constant exists; no recruiter screens
Admin screen to create DSA problems	**PLANNED**	Problems are loaded from a seed file
Job queue for the judge	**PLANNED**	An in-process concurrency limiter is used instead

> **The honest headline.** The platform works end to end on deterministic engines. Neither fine-tuned language model is serving users: the ATS model has not been trained, and the first career model was trained and measured as not usable. Say this plainly; the design that makes the product work without them is one of its strengths.

2. Problem statement and proposed system

2.1 The problem

A student preparing for placement uses separate tools that do not share information: a resume checker, a coding site, an aptitude site, interview videos and a learning platform. Three problems follow.

- **No single picture.** Nothing tells the student how ready they are overall, or what to fix first.
- **Unexplained scores.** Many resume checkers return a number without saying how it was computed.
- **No institutional view.** A placement officer cannot see which students are at risk until results arrive.

2.2 Existing approach and its limits

Conventional single-purpose tool	What it typically covers	What it leaves out
Resume checker	A resume score and keyword hints	Coding, interviews, a plan, institutional view
DSA practice site	Problems, judge, submissions	Resume, readiness, career guidance
Learning platform	Courses	Personal gap analysis against a target role
Job portal	Listings and applications	Preparation and measurement

This comparison is against a *typical single-purpose* tool. Large commercial platforms offer several of these features together; the claim is about integration and explainability for a campus setting, not about any feature being unique.

2.3 Proposed system

CareerPilot AI puts the preparation steps on one account and connects them through one scoring engine, APIE. Every scored activity (resume, assessment, coding submission, interview answer, alumni feedback) triggers a readiness recomputation, so the dashboard always reflects the latest evidence.

Figure 2. Student journey

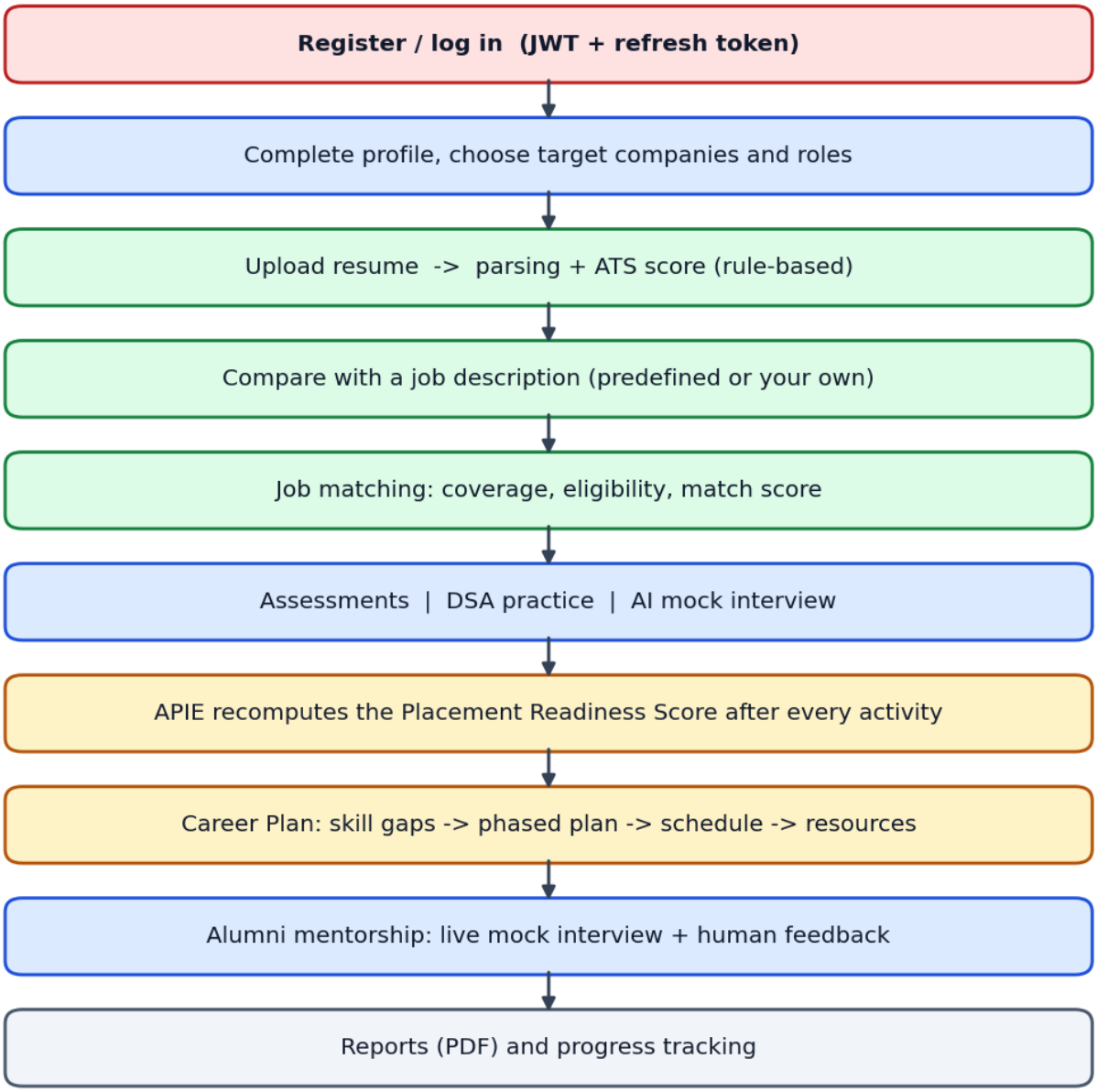


Figure 2. Student journey

2.4 Objectives

1. Give each student one explainable readiness score with its components, weights and reasons.
2. Analyse resumes with a transparent rule-based ATS score and job-description comparison.
3. Provide practice for each placement stage: aptitude, coding, technical and HR interviews.
4. Produce a personal career plan from the gap between a student's skills and a target role.
5. Connect students with verified alumni and feed human feedback back into the score.
6. Give placement staff cohort analytics and at-risk identification.
7. Keep AI use safe: validated outputs, deterministic scores, honest labels for prototype models.

2.5 Users and roles

Role	Exists in code	What they do
Student	**IMPLEMENTED**	All preparation features

Role	Exists in code	What they do
Alumni	**IMPLEMENTED**	Profile, mentorship requests, sessions, structured feedback
Placement Officer	**IMPLEMENTED**	Cohort dashboard, analytics, student list, reports
Administrator	**IMPLEMENTED**	Users, companies, jobs, alumni verification, AI usage, audit log, APIE weights
Recruiter	**PLANNED**	The role constant is reserved; no screens or routes
Faculty / Trainer	Not in the repository	Do not mention this role

2.6 Functional requirements (as built)

Area	Requirement	Status
Accounts	Register, log in, refresh, log out, role-based access	**IMPLEMENTED**
Resume	Upload PDF/DOCX/TXT, parse, score, compare with a JD, re-analyse, download	**IMPLEMENTED**
Jobs	Company and job catalogue, match a profile to a job	**IMPLEMENTED**
Assessments	Timed papers, autosave, recovery, grading, integrity signals	**IMPLEMENTED**
DSA	Problem list with filters, editor, Run, Submit, history, progress	**IMPLEMENTED**
Interview	Technical, HR and behavioural modes, text or voice	**IMPLEMENTED**
Readiness	Score, bands, skill gaps, recommendations, trend	**IMPLEMENTED**
Career plan	Role catalogue, skill gap, phased plan, schedule, resources, progress	**IMPLEMENTED**
Mentorship	Alumni directory, requests, scheduling, WebRTC room, feedback	**IMPLEMENTED**
Analytics	Cohort readiness, skill gaps, at-risk list, research export	**IMPLEMENTED**
Reports	PDF generation for ten report types	**IMPLEMENTED**
Privacy	Data export and deletion endpoints	**IMPLEMENTED**

2.7 Non-functional requirements

Quality	How it is addressed	Status
Security	JWT, RBAC, validation, rate limits, sandboxed code	**IMPLEMENTED**; sandbox **PARTIALLY IMPLEMENTED**
Explainability	Every score returns its components and reasons	**IMPLEMENTED**
Availability of scores	Scores are deterministic and work with the LLM down	**IMPLEMENTED**
Testability	Automated suites in all three services	**IMPLEMENTED**
Performance	Indexes, pagination, lazy loading, concurrency cap	**IMPLEMENTED**; no benchmark **NOT YET MEASURED**
Portability	Docker Compose for the three services and MongoDB	**IMPLEMENTED**
Honesty of data	Records labelled DEMO, SYNTHETIC or REAL	**IMPLEMENTED**

3. System architecture

3.1 High-level architecture

Three services with one trust boundary. The browser talks only to the Node.js gateway. The gateway is the only caller of the Python AI service, and it authenticates with a shared internal key. Model access and API keys never reach the browser.

```
React (Vite, TypeScript)  --HTTP / WebSocket-->  Node.js + Express gateway  --HTTP + internal key-->
    Python FastAPI AI service
   UI, Monaco, voice                                auth, RBAC, MongoDB,
      agents, NLP, ATS, APIE,
                                                    Socket.IO, PDF, judge
      RAG, ML, career planner
```

Figure 1. System architecture (layered)

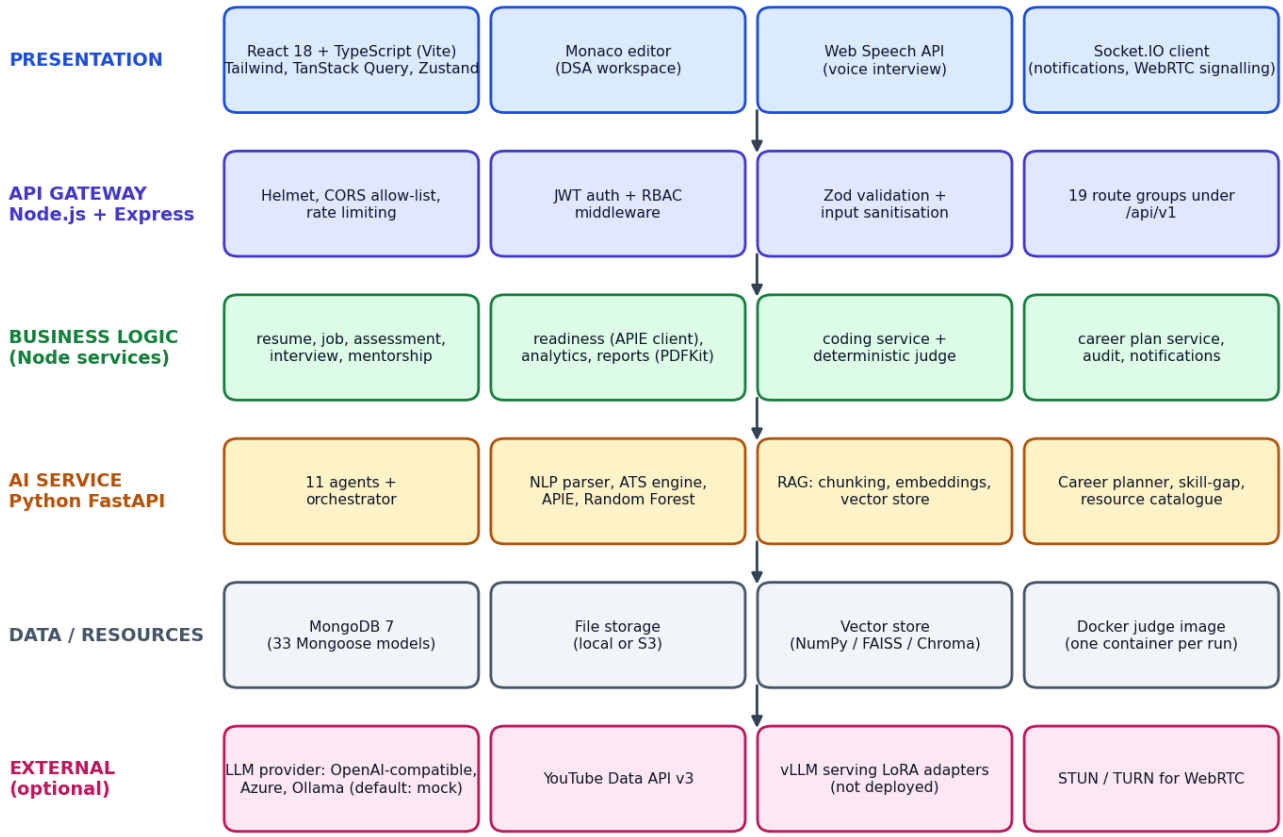


Figure 1. System architecture (layered)

3.2 Why three services instead of one

Decision	Reason	Alternative considered	Why not
Node.js for the gateway	I/O-heavy work: auth, sockets, database, file streaming	A single Python service	Weaker real-time and WebSocket ergonomics; one service would mix web and ML concerns
Python for AI	The NLP/ML libraries (spaCy, scikit-learn, NumPy, PyMuPDF) live there	Running ML in Node	No mature equivalents for spaCy or scikit-learn

Decision	Reason	Alternative considered	Why not
Shared TypeScript package	Frontend and backend import the same types and enums	Duplicated types	Types drift apart silently
Gateway is the only AI caller	One place to enforce auth, rate limits and audit	Browser calls the AI service	Would expose the internal key and bypass access control

This is a small **service-oriented** design with three services, not a microservices deployment: there is no service discovery, message bus or per-service database.

3.3 Detailed request path

Figure 3. Request path from browser to model

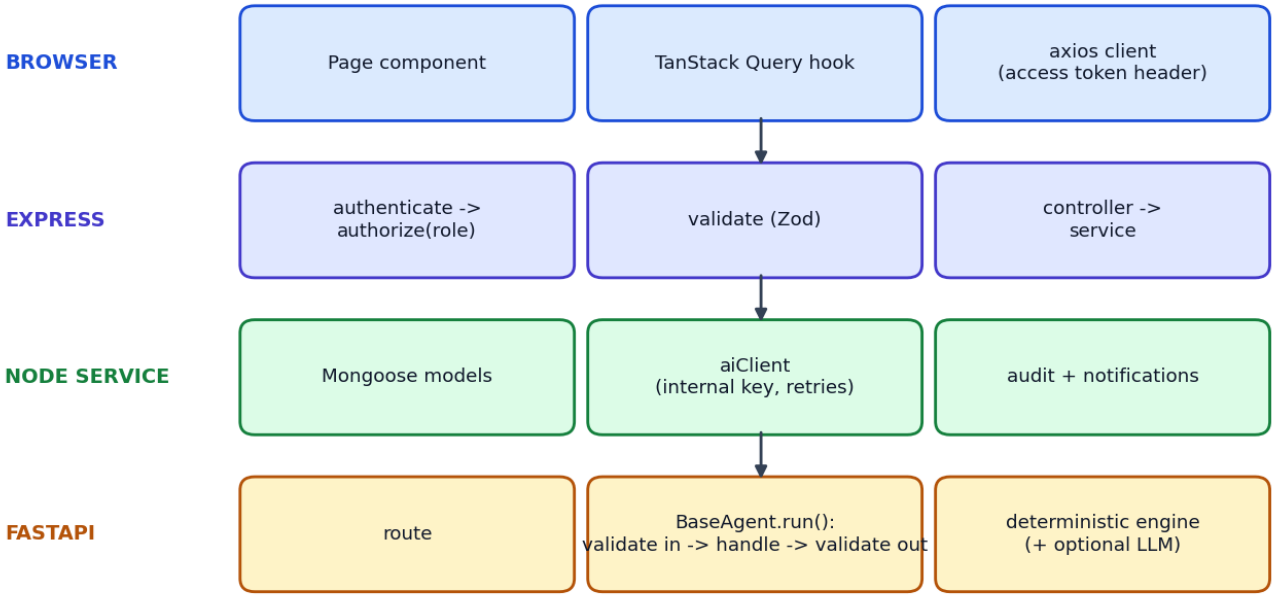


Figure 3. Request path from browser to model

1. A page component calls a TanStack Query hook, which calls the axios client with the access token.
2. Express runs global middleware: Helmet, CORS, compression, body parsing (2 MB limit), request id, timing, input sanitisation, then the global rate limiter.
3. The route applies `authenticate`, `authorize(role)`, `validate(zodSchema)` and then the controller.
4. The controller calls a service. Services own business rules and Mongoose access.
5. If AI work is needed, `aiClient` posts to FastAPI with the internal key, retries on connection errors and translates failures into typed API errors.
6. In FastAPI, `BaseAgent.run()` validates the input model, runs `handle()`, validates the output model and records latency and provider.

3.4 Layer responsibilities

Layer	Folder	Responsibility
Presentation	`frontend/src/pages`, `components`, `layouts`	38 page components, shared UI kit, route-level code splitting
Client state	`frontend/src/store`, TanStack Query	Auth store (Zustand), server-state cache
Routes	`backend/src/routes/v1`	15 route files, 135 endpoints, middleware chains
Controllers	`backend/src/controllers`	HTTP in and out; no business rules

Layer	Folder	Responsibility
Services	`backend/src/services`	Business logic, judge, AI client, storage, reports
Models	`backend/src/models`	33 Mongoose models
Agents	`ai-service/app/agents`	11 agents and the orchestrator
AI engines	`ai-service/app/services`, `nlp`, `rag`, `ml`, `career`	Deterministic engines and model wrappers
Shared	`shared/src`	Types, enums, constants used by both TypeScript apps
Seed data	`database/seeds`	Demo companies, questions, DSA problems and reference solutions

3.5 Real-time and background behaviour

- **Socket.IO** pushes notifications and `readiness:updated`, and carries WebRTC signalling for the alumni interview room. **IMPLEMENTED**
- **There is no job queue or worker process.** The DSA judge runs inside the request, bounded by an in-process semaphore (`CODE_RUNNER_MAX_CONCURRENT`, default 2; up to 25 requests wait, more are refused). A queue with separate judge workers is **PLANNED**.
- **There is no external cache** such as Redis. Caching is limited to an in-memory configuration cache, a 20-second cache of the Docker availability check, and the browser-side TanStack Query cache.

4. Technology stack

Layer	Technology	Role	Why selected	Alternative
Frontend	React 18, TypeScript, Vite	Single-page application	Component model, type safety, fast builds	Angular, Next.js
Frontend	Tailwind CSS, Radix UI primitives	Styling and accessible components	Consistent design with little custom CSS	Material UI
Frontend	TanStack Query, Zustand	Server-state cache; auth state	Caching and refetching without boilerplate	Redux
Frontend	Monaco editor	DSA code editor	The editor that powers VS Code	CodeMirror
Frontend	Recharts, react-hook-form, Zod	Charts, forms, validation	Small and typed	Chart.js, Formik
Backend	Node.js, Express 4	REST API gateway	Mature middleware ecosystem	NestJS, Fastify
Backend	Mongoose 8	Object modelling for MongoDB	Schemas, validation, indexes	Native driver, Prisma
Backend	Zod	Request validation	One schema gives types and runtime checks	Joi
Backend	jsonwebtoken, bcryptjs	Tokens and password hashing	Standard, well reviewed	Sessions, argon2
Backend	Socket.IO	Real-time events, WebRTC signalling	Reconnection and fallbacks built in	Raw WebSocket
Backend	PDFKit, multer, pino	PDF reports, uploads, structured logs	Simple and dependency-light	Puppeteer, winston
Database	MongoDB 7	Document store	Nested, evolving documents (resume analyses, plans, snapshots)	PostgreSQL
AI service	FastAPI, Pydantic 2, uvicorn	Typed HTTP API for AI	Validation is built into the framework	Flask
AI service	spaCy, PyMuPDF, python-docx	NER, PDF and DOCX text extraction	Reliable, offline	pdfminer, NLTK
AI service	scikit-learn, NumPy, pandas	Random Forest, vector maths	Standard for tabular ML	XGBoost
Fine-tuning	Qwen3 models, Unsloth, PEFT, bitsandbytes, TRL	QLoRA training in Colab	Fits a free T4 GPU; Apache-2.0 base models	Full fine-tuning
Judge	Docker, Debian image with g++, OpenJDK, Node.js, Python 3	Isolated code execution	Standard isolation primitive	Judge0, Firecracker
External	YouTube Data API v3	Video resources for the career plan	Official, returns availability	Scraping
External	OpenAI-compatible, Azure, Ollama, or mock	LLM provider behind one interface	Swappable; mock runs offline	A single hard-coded vendor
DevOps	Docker Compose, nginx, npm workspaces	Local and container deployment	One command to start the stack	Kubernetes
Testing	Vitest, supertest, mongodb-memory-server, pytest, Testing Library	Unit, API and component tests	Fast, isolated from real services	Jest

Check before the demo. The repository's local settings use `AI_PROVIDER=mock`, `EMBEDDING_PROVIDER=hash` and `VECTOR_DB_PROVIDER=numpy`. With the mock provider, interview questions and written feedback come from an offline rule-based generator, **not a language model**. Scores are unaffected because they are computed in code. If a judge asks "which LLM is answering right now?", the truthful answer depends on that setting.

5. API architecture

All endpoints are under `/api/v1`. There are **135 endpoints in 15 route files**. Responses share one envelope: `{ success, data, meta? }` or `{ success: false, error: { code, message, details? } }`.

5.1 Principal endpoints

Method	Endpoint	Purpose	Auth	Database	AI involvement
POST	`/auth/register`, `/auth/login`	Create account, sign in	Public	User, RefreshToken	None
POST	`/auth/refresh`	Rotate the refresh token	Refresh cookie	RefreshToken	None
GET	`/students/me/dashboard`	Student dashboard data	Student	Several	None
POST	`/resumes/upload`	Upload a resume	Student	Resume + file store	None
POST	`/resumes/:id/analyze`	Parse and score; optional JD comparison	Student	ResumeAnalysis	Resume agent (deterministic)
POST	`/resumes/jd/extract`	Read text from a JD file	Student	None (not stored)	Text extraction
POST	`/jobs/match`	Match profile to a job	Student	JobDescription	Job-matching agent
POST	`/assessments/:id/start`	Start a timed attempt	Student	AssessmentAttempt	None
POST	`/assessments/attempts/:id/submit`	Grade an attempt	Student	AssessmentAttempt	None
GET	`/coding/problems`	Problem list with filters	Student	CodingProblem	None
POST	`/coding/problems/:id/run`	Judge on public tests	Student	None stored	None
POST	`/coding/problems/:id/submit`	Judge on public + hidden tests	Student	CodingSubmission	Static review after the verdict
GET	`/coding/progress`	Solved counts by difficulty	Student	CodingSubmission	None
POST	`/interviews/start`	Start an interview	Student	InterviewSession	Interview agent (LLM or mock)
POST	`/interviews/sessions/:id/answer`	Evaluate an answer, get next question	Student	InterviewResponse, Evaluation	Evaluation + interview agents
GET	`/readiness`	Current readiness with gaps	Student, staff	PlacementReadiness	APIE + Random Forest
POST	`/career/plan`	Generate a career plan	Student	CareerPlan	Planner (model optional, off)
POST	`/career/progress`	Mark skills complete	Student	CareerPlan	None
POST	`/mentorship/request`	Request a mentor	Student	MentorshipRequest	Mentor-matching agent
GET	`/analytics/placement`	Cohort analytics	Officer, admin	Aggregations	Analytics agent
GET	`/reports/:type`	PDF report	Role-dependent	Several	None

Method	Endpoint	Purpose	Auth	Database	AI involvement
POST	`/admin/rag/query`	Test a knowledge-base question	Admin	RAGDocument	RAG pipeline
PUT	`/admin/config/:key`	Change APIE weights or bands	Admin	SystemConfiguration	None

5.2 Endpoint count by group

Group	Mount path	Endpoints
Authentication	`/auth`	12
Students	`/students`	6
Resumes	`/resumes`	10
Companies and jobs	`/companies`, `/jobs`	6
Assessments	`/assessments`	9
Coding (DSA)	`/coding`	8
Interviews	`/interviews`	6
Speech	`/speech`	2
Readiness and recommendations	`/readiness`, `/recommendations`	8
Career plan	`/career`	5
Alumni and mentorship	`/alumni`, `/mentorship`	19
Analytics	`/analytics`	4
Reports	`/reports`	4
Notifications and privacy	`/notifications`, `/privacy`	9
Admin	`/admin`	27

The AI service has its own internal API (`/ai/resume/*`, `/ai/interview/*`, `/ai/readiness/*`, `/ai/career/*`, `/ai/rag/*`, `/ai/coding/review`, `/ai/speech/*`, `/ai/agents`, `/health`). It is not reachable from the browser.

6. Database architecture

Technology: MongoDB 7 through Mongoose 8. There are **33 models** and about 186 index declarations. MongoDB has no foreign keys; relationships are `ObjectId` references that Mongoose can populate, and integrity is enforced in the service layer.

Figure 4. Main collections and how they relate

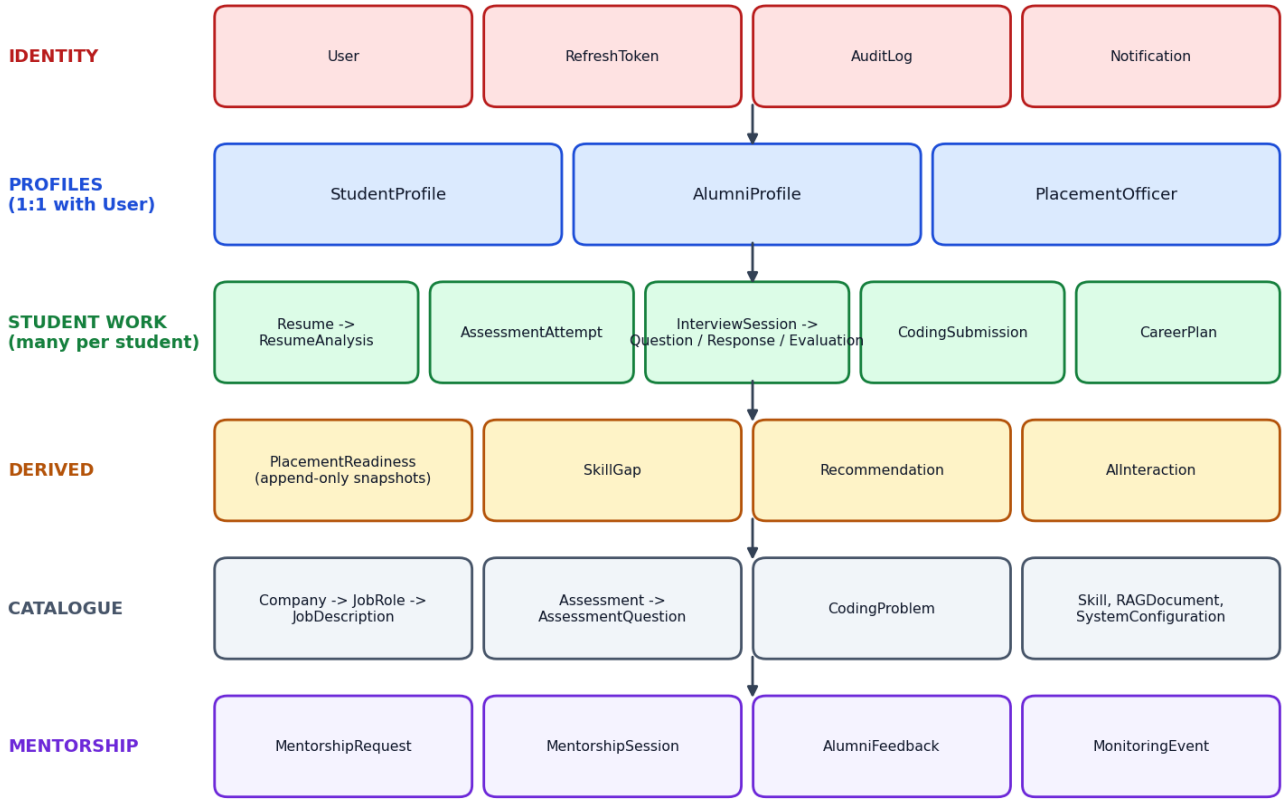


Figure 4. Main collections and how they relate

6.1 Collections by purpose

Group	Models
Identity and audit	User, RefreshToken, AuditLog, Notification
Profiles	StudentProfile, AlumniProfile, PlacementOfficer
Resume	Resume, ResumeAnalysis
Jobs	Company, JobRole, JobDescription, Skill
Assessment	Assessment, AssessmentQuestion, AssessmentAttempt, MonitoringEvent
Coding	CodingProblem, CodingSubmission
Interview	InterviewSession, InterviewQuestion, InterviewResponse, InterviewEvaluation
Readiness	PlacementReadiness, SkillGap, Recommendation
Career	CareerPlan
Mentorship	MentorshipRequest, MentorshipSession, AlumniFeedback
AI and configuration	AIInteraction, RAGDocument, SystemConfiguration

There is no separate "Admin" model: an administrator is a `User` with role `ADMIN`. There is no separate "Resource" collection: learning resources are a JSON catalogue inside the AI service.

6.2 Relationships

Relationship	Type	How it is stored
User — StudentProfile / AlumniProfile / PlacementOfficer	One-to-one	Profile holds `userId` (unique)
StudentProfile — Resume	One-to-many	Resume holds `studentId`; one is marked `isPrimary`
Resume — ResumeAnalysis	One-to-many	Each re-analysis adds a record
StudentProfile — PlacementReadiness	One-to-many	Append-only snapshots; the newest is current
StudentProfile — CodingSubmission — CodingProblem	Many-to-many through a join document	Submission holds `studentId` and `problemId`
Assessment — AssessmentQuestion	One-to-many	Question holds `assessmentId`
StudentProfile — AssessmentAttempt — Assessment	Many-to-many through a join document	Attempt holds both ids
InterviewSession — Question — Response — Evaluation	One-to-many chain	Each child holds its parent id
Company — JobRole — JobDescription	One-to-many	Child holds parent id
StudentProfile — target companies and roles	Many-to-many	Arrays of ids on the profile
MentorshipRequest — MentorshipSession — AlumniFeedback	One-to-one, then one-to-many	Child holds parent id

6.3 Schemas worth knowing

User. `email` (unique), `passwordHash`, `fullName`, `role`, `status`, `emailVerified`. The password is never stored in plain text.

ResumeAnalysis. `atsScore`, `parsed`, `breakdown` (the nine criteria), `matchedKeywords`, `missingKeywords`, `strengths`, `weaknesses`, recommendations, `targetJobDescriptionId`, `jobDescriptionSource` (CATALOGUE or CUSTOM), `jobMatchScore`, `engineVersion`, `provenance`.

PlacementReadiness. `score`, `band`, `components` (key, raw, normalised, weight, weighted, evidence count, stale flag), `modifiers`, `weightsSnapshot`, ML prediction, `trigger`, `computedAt`. Storing the weights with each snapshot keeps old scores interpretable after weights change.

CodingProblem. `title`, `slug` (unique), `difficulty`, `tags`, `description`, `examples`, `constraints`, `functionName`, `signature`, `compareMode`, `starterCode`, `supportedLanguages`, `testCases`, `hiddenTestCases`, `referenceSolutions`, `timeLimitMs` (default 2000), `memoryLimitMb` (default 128). `hiddenTestCases` and `referenceSolutions` are declared `select: false`, so no query returns them unless it asks for them by name.

CodingSubmission. `problemId`, `studentId`, `language`, `code`, `status` (the verdict), `passedCount`, `totalCount`, `runtimeMs`, `memoryKb`, `execution`, `score`, `aiReview`.

CareerPlan. Target role, inputs, the generated plan, resources, `resourcesBySkill`, schedule, readiness summary, progress, and which engine produced it.

6.4 Why MongoDB

The data is document-shaped and evolves: a resume analysis, a readiness snapshot and a career plan are nested structures whose fields changed during development. A relational schema would need many join tables and migrations. The cost is weaker referential integrity, which is handled in services, and no multi-document transactions are relied on.

7. Security architecture

Figure 17. Security controls by layer

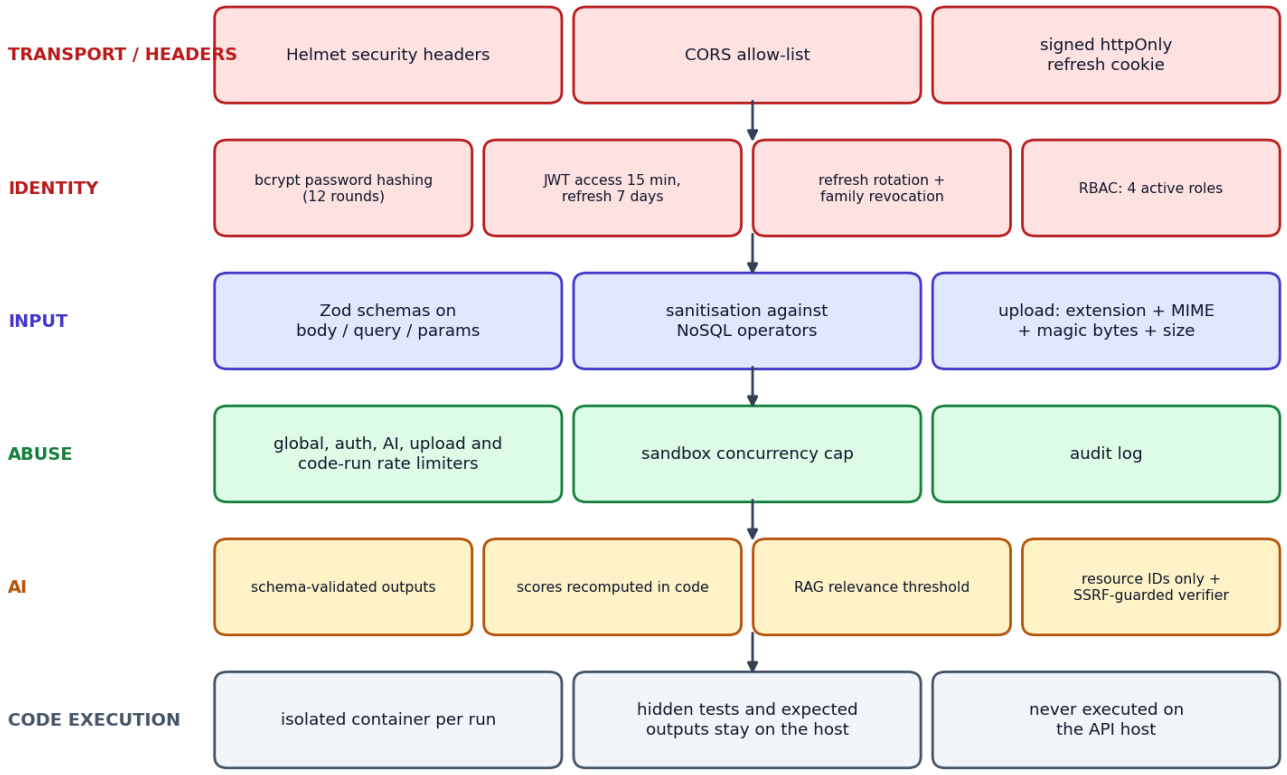


Figure 17. Security controls by layer

Concern	Control	Where	Status
Passwords	bcrypt hash with salt, 12 rounds	`User` model	**IMPLEMENTED**
Sessions	JWT access token (15 min) and refresh token (7 days)	auth service	**IMPLEMENTED**
Stolen refresh token	Rotation with family revocation: reuse of an old token revokes the whole family	auth service	**IMPLEMENTED**
Refresh token storage	Signed `httpOnly` cookie	auth controller	**IMPLEMENTED**
Authorisation	`authorize(role)` middleware; ownership checks in controllers	routes	**IMPLEMENTED**
Input validation	Zod schemas on body, query and params	`validators/`	**IMPLEMENTED**
NoSQL injection	`sanitizeInput` strips operator keys; validated types	middleware	**IMPLEMENTED**
XSS	React escapes output; Helmet headers; downloads sent as attachments with `nosniff`	frontend, backend	**IMPLEMENTED**
CSRF	Access token travels in a header, not a cookie; CORS allow-list	backend	**IMPLEMENTED**
Rate limiting	Global, auth, AI, upload and code-run limiters	middleware	**IMPLEMENTED**
File upload	Extension + MIME type + magic bytes + 5 MB limit + SHA-256 duplicate check	upload middleware, storage	**IMPLEMENTED**
Secrets	Environment variables; `.env` not committed; `.env.example` has no real values	config	**IMPLEMENTED**
Service boundary	AI service requires an internal key and is not exposed publicly	ai-service	**IMPLEMENTED**

Concern	Control	Where	Status
Audit	Sensitive actions written to `AuditLog`	audit service	**IMPLEMENTED**
Error handling	Central handler returns coded errors; stack traces are not sent to clients	middleware	**IMPLEMENTED**
Code execution	Isolated container, never on the host	judge	**PARTIALLY IMPLEMENTED** (not run with Docker yet)

7.1 AI-specific security

Risk	What the project does	What it does not do
Hallucinated scores	The model returns dimension scores; the overall is recomputed in code	—
Hallucinated facts	Strict Pydantic schemas (`extra="forbid"`); ATS grounding guard moves an unmatched "matched skill" to missing	—
Hallucinated links	The career model may output resource IDs only; unknown IDs are dropped	—
Unsupported answers	RAG returns a fixed "no verified information" answer below the relevance threshold, without calling the LLM	—
Prompt injection	Structural limits: the model cannot call tools, cannot set a final score, and its output must match a schema	There is **no dedicated prompt-injection classifier or filter**
Server-side request forgery	The resource verifier allows only HTTPS on the provider's domain, refuses private addresses, re-checks redirects	—
Bias from voice	Whether an answer was spoken or typed is not sent to the evaluator	—

> **If asked "how do you stop prompt injection?"** Do not claim a filter. Say: "A resume or answer could contain instructions, so the design limits what the model can do with them. It has no tools, it cannot decide a score because the score is recomputed from fixed weights, and anything outside the schema is rejected. A dedicated injection detector is future work."

8. Deployment architecture

Figure 16. Deployment (docker-compose.yml)

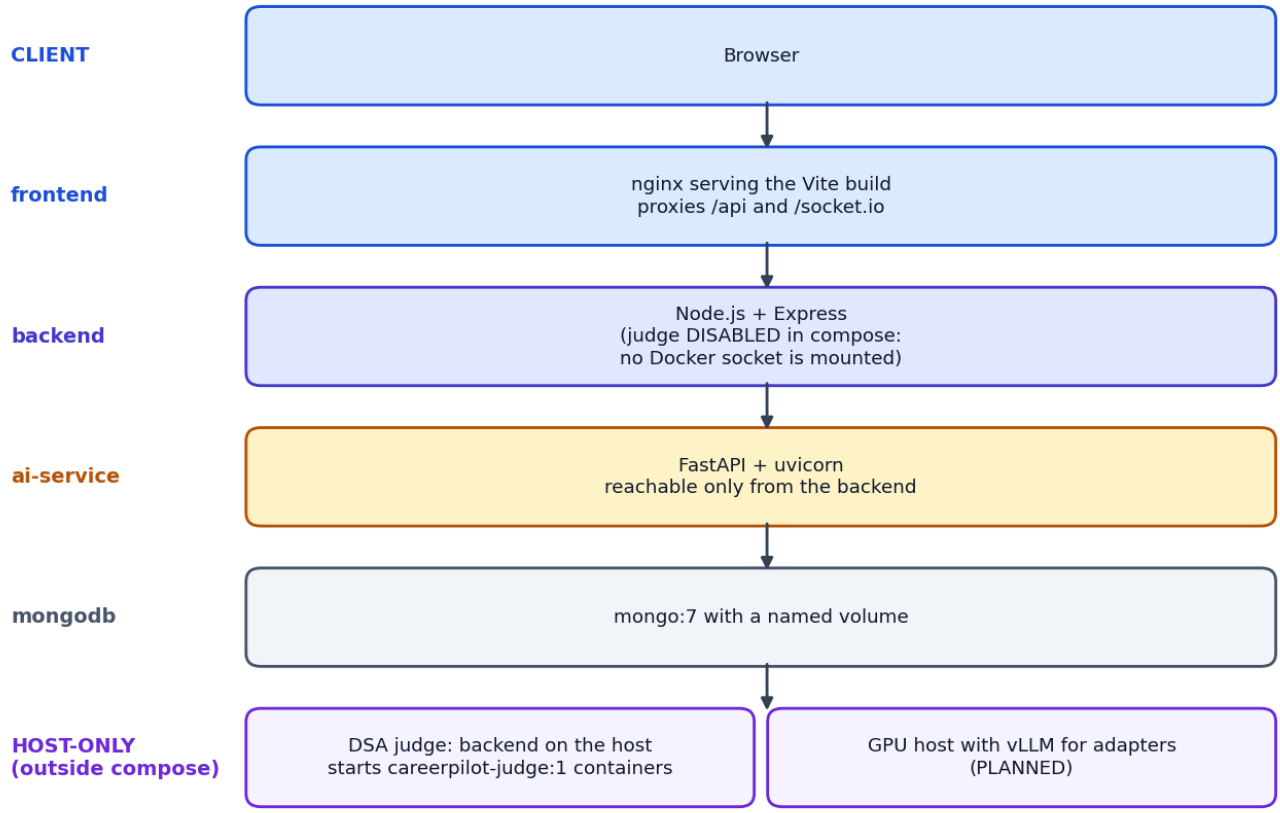


Figure 16. Deployment (docker-compose.yml)

- `docker-compose.yml` defines **mongodb** (mongo:7), **ai-service**, **backend** and **frontend** (nginx serving the built app and proxying `/api` and `/socket.io`). **IMPLEMENTED**
- In Compose the judge is set to `disabled`. A containerised backend would need the host's Docker socket to start judge containers, and mounting that socket is equivalent to root on the host. To use the judge, the backend runs directly on the host.
- For development, `npm run dev` starts all three services with hot reload.
- A GPU host running vLLM to serve the LoRA adapters is **PLANNED**; no such host exists in the repository's configuration.
- There is no CI pipeline, monitoring stack or cloud deployment in the repository. **NOT VERIFIED FROM SOURCE** for any production hosting.

8.1 Environment variables (names only)

Group	Variables
Core	`NODE_ENV`, `PORT`, `MONGODB_URI`, `USE_MEMORY_DB`, `CORS_ORIGINS`
Secrets	`JWT_SECRET`, `JWT_REFRESH_SECRET`, `COOKIE_SECRET`, `AI_SERVICE_INTERNAL_KEY` — values [REDACTED]
AI provider	`AI_PROVIDER`, `AI_API_KEY` [REDACTED], `AI_BASE_URL`, `AI_MODEL`, `AI_TEMPERATURE`
Retrieval	`VECTOR_DB_PROVIDER`, `EMBEDDING_PROVIDER`, `RAG_TOP_K`, `RAG_CHUNK_SIZE`
Fine-tuned models	`ATS_MODEL_PROVIDER`, `ATS_MODEL_ENDPOINT`, `CAREER_MODEL_PROVIDER`, `CAREER_MODEL_ENDPOINT`
Resources	`YOUTUBE_API_KEY` [REDACTED]

Group	Variables
Judge	`CODE_EXECUTION_PROVIDER`, `CODE_RUNNER_IMAGE`, `CODE_RUNNER_CPUS`, `CODE_RUNNER_PIDS_LIMIT`, `CODE_RUNNER_MAX_CONCURRENT`
Speech and storage	`SPEECH_PROVIDER`, `STORAGE_PROVIDER`, `MAX_UPLOAD_SIZE_MB`

9. Functional modules

Each module below follows the same template so you can answer "what goes in, what happens, what comes out" for any of them. Complexity uses *n* for the size of the input named in each block.

9.1 Authentication and role-based access

- **Problem:** only the right person may see or change a record.
- **Input:** email and password; later an access token on each request.
- **Processing:** the password is compared with its bcrypt hash. On success the server issues a 15-minute JWT access token and a 7-day refresh token stored as a signed `httpOnly` cookie. Each refresh issues a new refresh token; presenting an old one revokes the whole token family.
- **Algorithm:** bcrypt (12 rounds); HMAC-signed JWT; refresh-token rotation with family revocation.
- **Technology:** Express middleware `authenticate`, `authorize(role)`, `requireStudentProfile`.
- **Database:** `User`, `RefreshToken`, `AuditLog`.
- **AI:** none.
- **Output:** tokens and the user's profile; 401 or 403 otherwise.
- **Time complexity:** O(1) per request plus the fixed bcrypt cost at login. **Space:** O(1).
- **Advantages:** stateless access checks; a stolen refresh token is detected on reuse.
- **Limitations:** no multi-factor authentication.
- **Status:** **IMPLEMENTED**

9.2 Resume intelligence and ATS scoring

This is the module the jury will probe most. The full pipeline is deterministic.

Figure 5. Resume / ATS pipeline

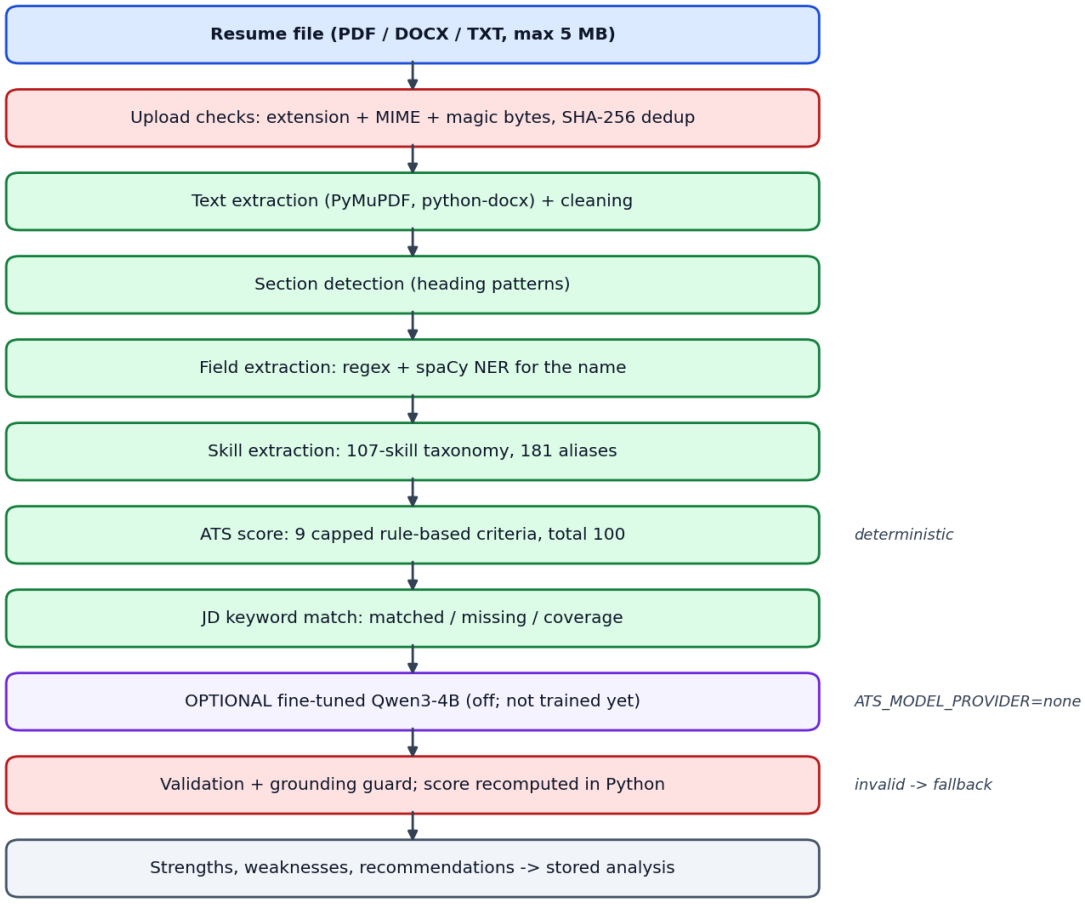


Figure 5. Resume / ATS pipeline

- **Problem:** a student does not know why a resume scores low or what to change.
- **Input:** a PDF, DOCX or TXT file up to 5 MB; optionally a job description (predefined, pasted, or uploaded as a file).
- **Processing:**
  1. **Ingestion.** Extension, MIME type and magic bytes are checked; a SHA-256 checksum short-circuits duplicate uploads.
  2. **Text extraction.** PyMuPDF for PDF, python-docx for DOCX. Fewer than 120 characters is rejected (usually a scanned image; there is no OCR).
  3. **Section detection.** A line of at most 45 characters matching a known heading starts a section.
  4. **Field extraction.** Regular expressions for email, phone, links, CGPA, dates; spaCy NER for the name.
  5. **Skill extraction.** A taxonomy of 107 canonical skills with 181 aliases, matched with custom word boundaries so `C++` and `C#` work and "Java" is not found inside "JavaScript".
  6. **Scoring.** Nine criteria, each a capped formula.
  7. **JD comparison.** JD keywords are canonicalised; `coverage = matched / (matched + missing) × 100`.
  8. **Recommendations.** Generated from the criteria that lost points.
- **Algorithm:** rule-based NLP (regex, dictionary matching, NER) and weighted multi-criteria scoring.
- **Technology:** FastAPI, spaCy, PyMuPDF, python-docx.
- **Database:** `Resume` (file metadata), `ResumeAnalysis` (result). The file itself is in local or S3 storage.
- **AI:** deterministic by default. The optional fine-tuned model is described in the AI chapter and is off.
- **Output:** ATS score out of 100, per-criterion breakdown, extracted profile, matched and missing keywords, strengths, weaknesses, recommendations.
- **Time complexity:** O(n · t) for text length *n* and *t* taxonomy patterns (288 terms). **Space:** O(n).

- **Advantages:** explainable, repeatable, needs no GPU or training data, cannot invent a skill.
- **Limitations:** finds only skills in the taxonomy; no OCR; it approximates ATS behaviour and is not any employer's real system.
- **Status:** **IMPLEMENTED**

The nine ATS criteria

Criterion	Points	Rule
Skills coverage	20	min(8, skills × 0.55) + min(4, languages × 1.4) + min(4, frameworks × 1.2) + min(4, CS fundamentals × 1.4)
Section structure	15	(required sections found ÷ required) × 9 + min(6, optional sections × 1.5)
Experience evidence	15	min(7, projects × 2.2) + min(5, internships × 2.5) + min(3, jobs × 3)
Contact information	10	name 3 + email 3 + phone 2 + LinkedIn/GitHub 2
Education details	10	education 4 + CGPA 3 + institution 1.5 + graduation year 1.5
Quantified impact	10	min(10, quantified phrases × 1.8)
Formatting hygiene	10	starts at 10, with deductions for few bullets, very long lines, table layout, unusual symbols
Action verbs	5	min(5, distinct verbs × 0.45) − min(2, weak phrases × 0.7)
Length	5	200–700 words scores 5; shorter or longer scores less

Each `min(...)` is a cap. It gives diminishing returns, so listing forty skills cannot outscore a balanced resume. Projects weigh more than jobs because the users are freshers.

Which parts are deterministic and which use AI

Step	Deterministic	Uses a model
Text extraction, sections, fields	Yes	spaCy NER for the name only (a pre-trained statistical model)
Skill extraction	Yes (dictionary)	No
ATS score	Yes	No
Matched / missing keywords	Yes	No, even when the fine-tuned model is enabled
Strengths, weaknesses, advice	Yes (templates)	Optional fine-tuned model, currently off

9.3 Job matching

- **Problem:** which jobs suit this profile, and what is missing for a given job.
- **Input:** the student's skills, CGPA and backlogs; a job description.
- **Processing and algorithm:**

```
match = 0.55 × required-skill coverage
      + 0.20 × preferred-skill coverage
      + 0.15 × semantic similarity (profile text vs JD text)
      + 0.10 × eligibility fit (CGPA, backlogs)
```

- Coverage is computed on canonical skills. Similarity is the cosine of two embedding vectors. Failing a stated CGPA or backlog limit sets `eligible = false` whatever the score.
- **Database:** `JobDescription`, `Company`, `JobRole`, `StudentProfile`.
- **AI:** embeddings only; no generative model.
- **Output:** match score, missing skills, eligibility, a preparation plan.
- **Time complexity:** O(s + d) for *s* skills and embedding dimension *d* = 384. **Space:** O(d).
- **Limitations:** the default embedding is lexical, not semantic.
- **Status:** **IMPLEMENTED**

9.4 Assessment engine

- **Problem:** fair, recoverable timed tests.
- **Processing:** the deadline is stored on the server and the browser timer is only a display. Answers autosave and survive a refresh. Grading happens on the server; correct answers never reach the browser. Integrity signals such as tab switches produce a LOW, MEDIUM or HIGH indicator for a human to review.
- **Algorithm:** server-authoritative timing; rule-based grading; threshold-based integrity level.
- **Database:** `Assessment`, `AssessmentQuestion`, `AssessmentAttempt`, `MonitoringEvent`.
- **AI:** none.
- **Output:** score, topic breakdown, and the integrity log shown openly to the student.
- **Time complexity:** O(q) for *q* questions.
- **Limitations:** a signal never fails a student automatically, by design.
- **Status:** **IMPLEMENTED**

9.5 DSA practice and the online judge

Figure 11. DSA judge: from Submit to verdict

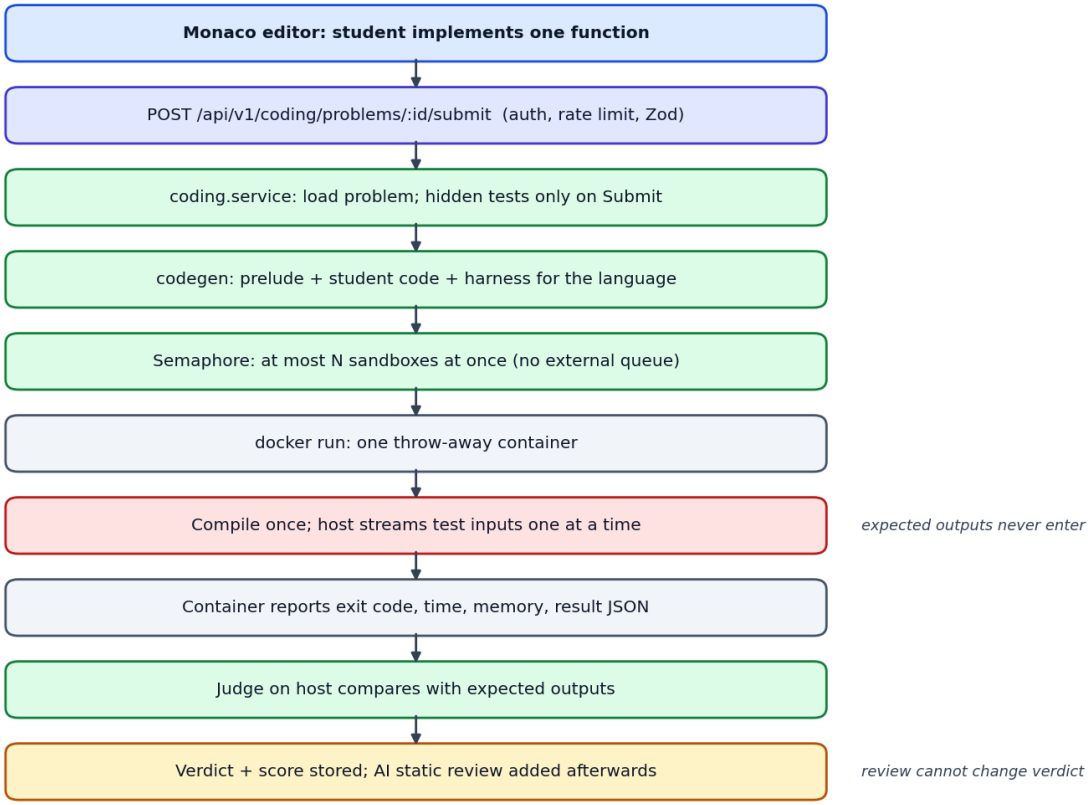


Figure 11. DSA judge: from Submit to verdict

- **Problem:** practise coding questions and get a trustworthy verdict.
- **Input:** a language (Python, C++, Java or JavaScript) and the body of one function.
- **Processing:**
  1. **Problem management.** 28 problems across 16 topic tags and three difficulties, loaded from `database/seeds/dsaProblems.ts` by `npm run seed:dsa`. Each has a typed signature, public tests and hidden tests.
  2. **Code generation.** From the signature the backend generates the starter code and a harness per language. The harness reads arguments from standard input, calls the function and writes the returned value as JSON to a result file. Standard output stays free for the student's own prints.

3. **Execution.** One throw-away container per submission compiles once, then runs each test. The host streams test inputs one at a time.
4. **Judging.** The host compares each returned value with the expected value and classifies the test.
5. **Storage.** Submit stores the verdict, counts, runtime and memory. Run stores nothing.

- **Algorithm:** code generation from a typed signature; structural JSON comparison; verdict classification.
- **Technology:** Node.js `child_process.spawn` of the Docker CLI with a fixed argument list; Monaco editor in the browser.
- **Database:** `CodingProblem`, `CodingSubmission`.
- **AI:** none in the verdict. After the verdict is final, an advisory "Static Code Analysis" is requested.
- **Output:** verdict, passed count, runtime, memory, per-test detail for public tests only.
- **Time complexity:** O(T · L) wall time for *T* tests and time limit *L*; the comparator is O(size of the answer). **Space:** bounded by the memory limit.
- **Status:** judge logic **IMPLEMENTED**; container isolation **PARTIALLY IMPLEMENTED** — written and unit-tested, not yet executed on a machine with Docker.

What exists and what does not

Item in a typical online judge	In this project
Problem list with search, difficulty and tag filters	**IMPLEMENTED**
Monaco editor, language switch, per-language starter code	**IMPLEMENTED**
Languages	Python (CPython 3), C++17 (g++), Java (OpenJDK), JavaScript (Node.js). **C is not supported.**
Run on public tests; Submit on public + hidden tests	**IMPLEMENTED**
Submission history and solved progress	**IMPLEMENTED**
Reference solutions and test validation	**IMPLEMENTED** — stored server-side; the test suite requires every one to be Accepted
Admin screen to create or publish problems	**PLANNED** — problems come from the seed file
Queue and separate judge workers	**PLANNED** — an in-process concurrency limiter is used
Custom test input from the editor	**PLANNED**

Verdicts

Verdict	When	Status
ACCEPTED	Every test ran and matched	**IMPLEMENTED**
WRONG_ANSWER	Returned value differs from expected	**IMPLEMENTED**
COMPILATION_ERROR	Compile or syntax check failed; no test runs	**IMPLEMENTED**
RUNTIME_ERROR	Non-zero exit, or no value returned	**IMPLEMENTED**
TIME_LIMIT_EXCEEDED	Stopped at the limit	**IMPLEMENTED**
MEMORY_LIMIT_EXCEEDED	Killed for memory, or an out-of-memory error	Classification **IMPLEMENTED**; real enforcement needs the container
INTERNAL_ERROR	The sandbox could not run the code	**IMPLEMENTED** — reported as "not executed", never guessed
OUTPUT_LIMIT_EXCEEDED	—	Not a separate verdict. Output is truncated at 64 KB; an oversized report stream is treated as a sandbox failure

The verdict of a submission is the verdict of its **first failing test**. Score = passed ÷ total × 100.

Why user code must not run on the backend

Submitted code is untrusted. On the API host it could read environment secrets, open the database, read other students' files, or exhaust the machine. Running it in the Node process (`eval`, `vm`) is not a security boundary. The project therefore has exactly one way to run student code: inside a container.

Figure 12. Sandbox isolation applied to every container

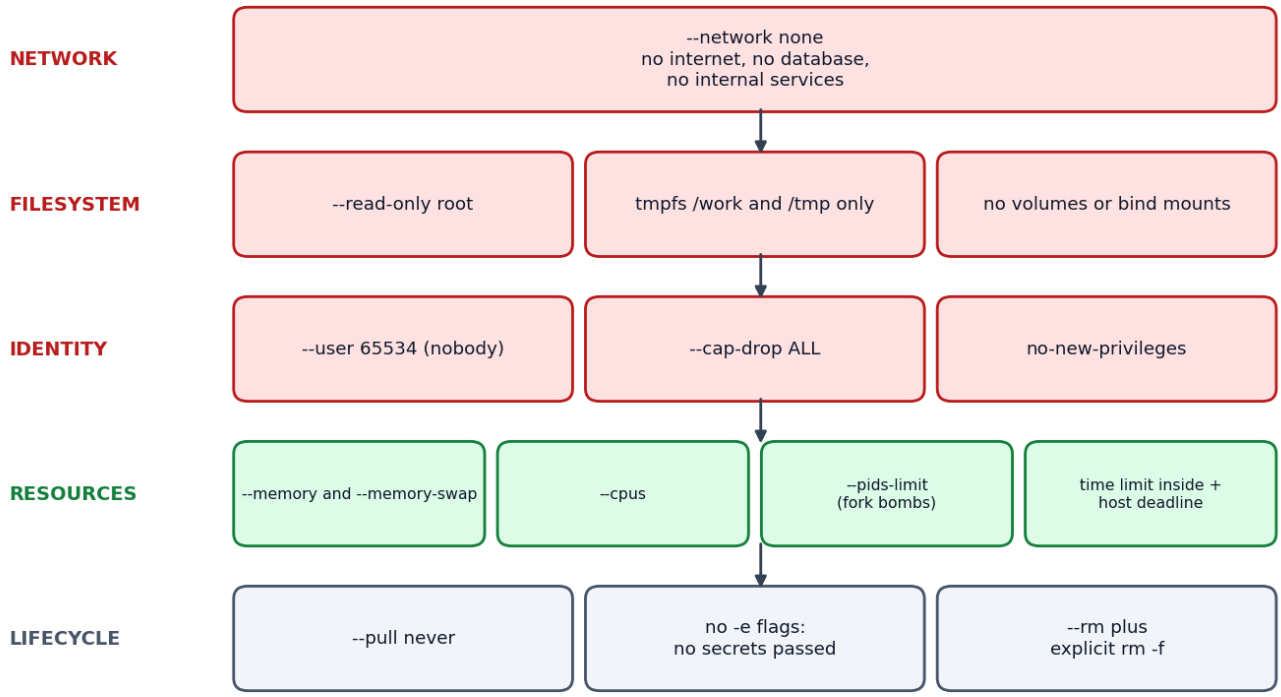


Figure 12. Sandbox isolation applied to every container

Limit	Mechanism
Network	`--network none`
Filesystem	`--read-only`, tmpfs for `/work` and `/tmp`, no volumes
Identity	`--user 65534:65534`, `--cap-drop ALL`, `no-new-privileges`
Memory	`--memory` and `--memory-swap`; plus per language: Java `-Xmx`, Node `--max-old-space-size`, C++ address-space limit
CPU and processes	`--cpus`, `--pids-limit` (stops fork bombs)
Time	`timeout` inside the container and a host-side deadline; default 2 s, scaled ×1 C++, ×2 Java, ×2 JavaScript, ×3 Python
Secrets	no `-e` flags, so no host environment variable enters
Cleanup	`--rm` plus an explicit `docker rm -f`

Hidden-test security

- Expected outputs **never enter the container**; comparison happens on the host.
- Inputs are streamed one test at a time, public tests first, so hidden inputs are not present while public tests run.
- `hiddenTestCases` is excluded from every database query unless the judge asks for it by name.
- In a response, a hidden test appears only as passed/failed, its status and its timing. An API test asserts that hidden inputs never appear in any response or stored record.

If hidden tests were exposed, a student could hard-code the expected answers and "pass" without solving the problem.

9.6 AI mock interview

- **Problem:** practise answering aloud or in text and get specific feedback.
- **Input:** mode (technical, HR, behavioural), difficulty, number of questions, then each answer (typed, or spoken and transcribed into an editable transcript).
- **Processing:** generate a question → the student answers → the evaluator scores five dimensions → the overall score is recomputed in code → difficulty adapts → next question.
- **Algorithm:**
  - *Difficulty controller:* look at the last three answer scores; mean ≥ 78 steps up, ≤ 42 steps down, otherwise hold. Nothing changes until two answers exist.
  - *Score:* weighted sum of relevance, correctness, completeness, communication and structure, with weights that depend on the mode (technical gives correctness 0.35; behavioural gives structure 0.25).
  - *Delivery signals:* word count, filler-word rate, `clarity = 100 − 600 × fillerRate − 6 × hedgeCount`. Computed from the transcript text only.
- **Technology:** LLM provider interface (or the mock provider), Web Speech API for speech-to-text and text-to-speech, optional server Whisper fallback.
- **Database:** `InterviewSession`, `InterviewQuestion`, `InterviewResponse`, `InterviewEvaluation`.
- **AI:** an LLM writes questions and scores the five dimensions. It does not compute the overall score.
- **Output:** per-answer scores, strengths, improvements, a session report.
- **Limitations:** with `AI_PROVIDER=mock` the questions and feedback come from an offline generator, not a language model.
- **Status:** **IMPLEMENTED**

9.7 APIE — Placement Readiness Score

Figure 14. What triggers a readiness update

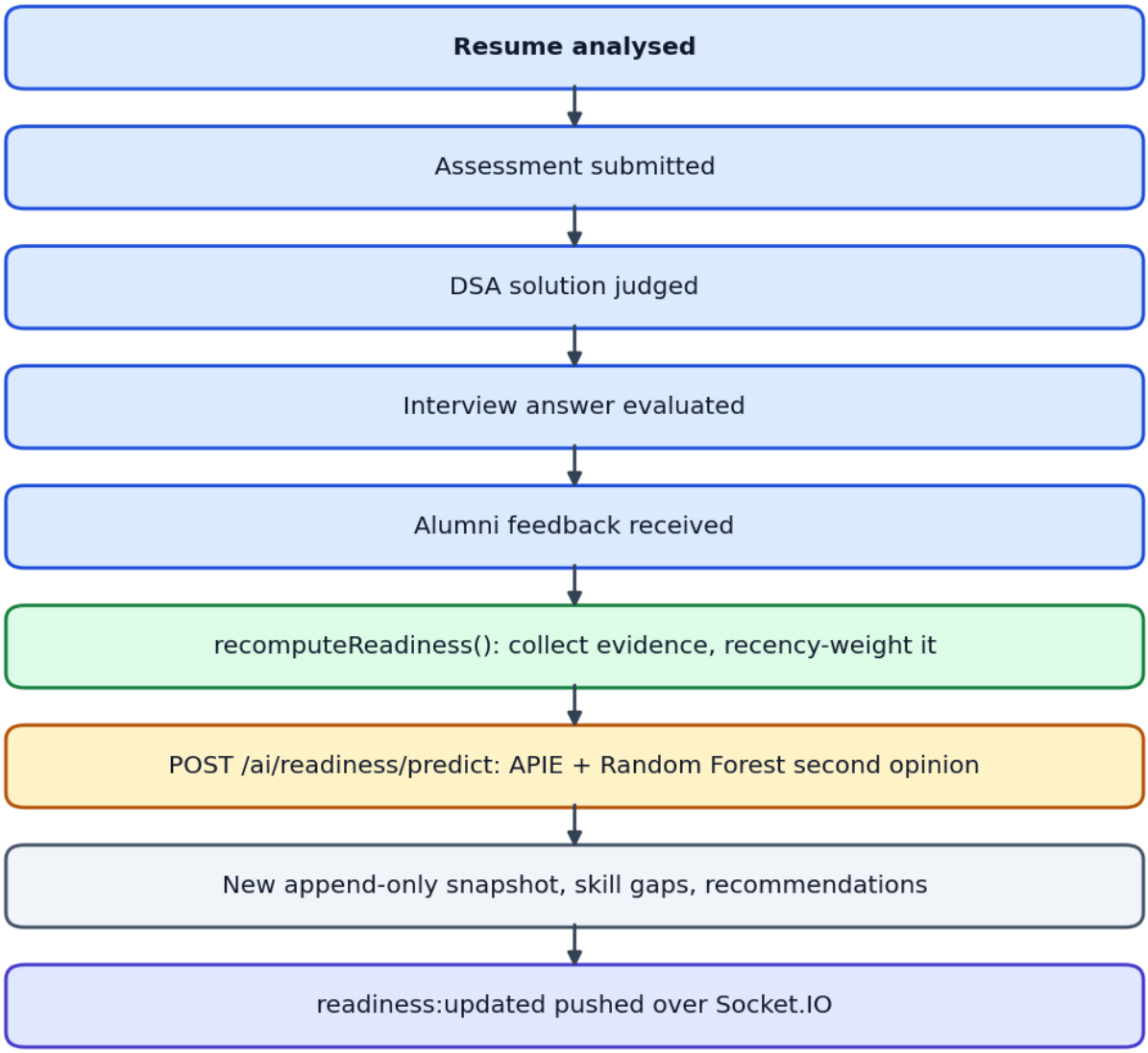


Figure 14. What triggers a readiness update

- **Problem:** turn scattered results into one explainable number and a priority list.
- **Input:** every scored activity for the student.
- **Processing and algorithm:**
  1. **Recency weighting.** Per component, `value = Σ(score_i × 1/(i+1)) / Σ 1/(i+1)`, newest first. The newest result has weight 1, then ½, then ⅓.
  2. **Weighted base.** Coding 0.25, technical interview 0.20, communication 0.15, resume/ATS 0.10, HR interview 0.10, aptitude 0.10, academic 0.10.
  3. **Bounded modifiers.** Projects up to +3, certifications up to +2, internships up to +3, alumni feedback −4 to +4.
  4. **Bands.** Below 40 At Risk; 40–60 Needs Improvement; 60–80 Placement Ready; 80 and above Highly Ready.
- **Database:** `PlacementReadiness` (append-only), `SkillGap`, `Recommendation`.
- **AI:** none in the score. A Random Forest gives a second opinion beside it.
- **Output:** score, band, component breakdown, strengths, weaknesses, skill gaps, recommendations, trend.
- **Time complexity:** O(e) for *e* evidence items.

- **Advantages:** weighted, recency-aware and evidence-aware; missing evidence is shown as "no evidence", never as a silent zero.
- **Limitations:** the weights are a design choice and are **not validated against real placement outcomes**. Every screen labels it a prototype scoring model.
- **Status:** **IMPLEMENTED**

9.8 Career Plan

Figure 6. Career Plan pipeline

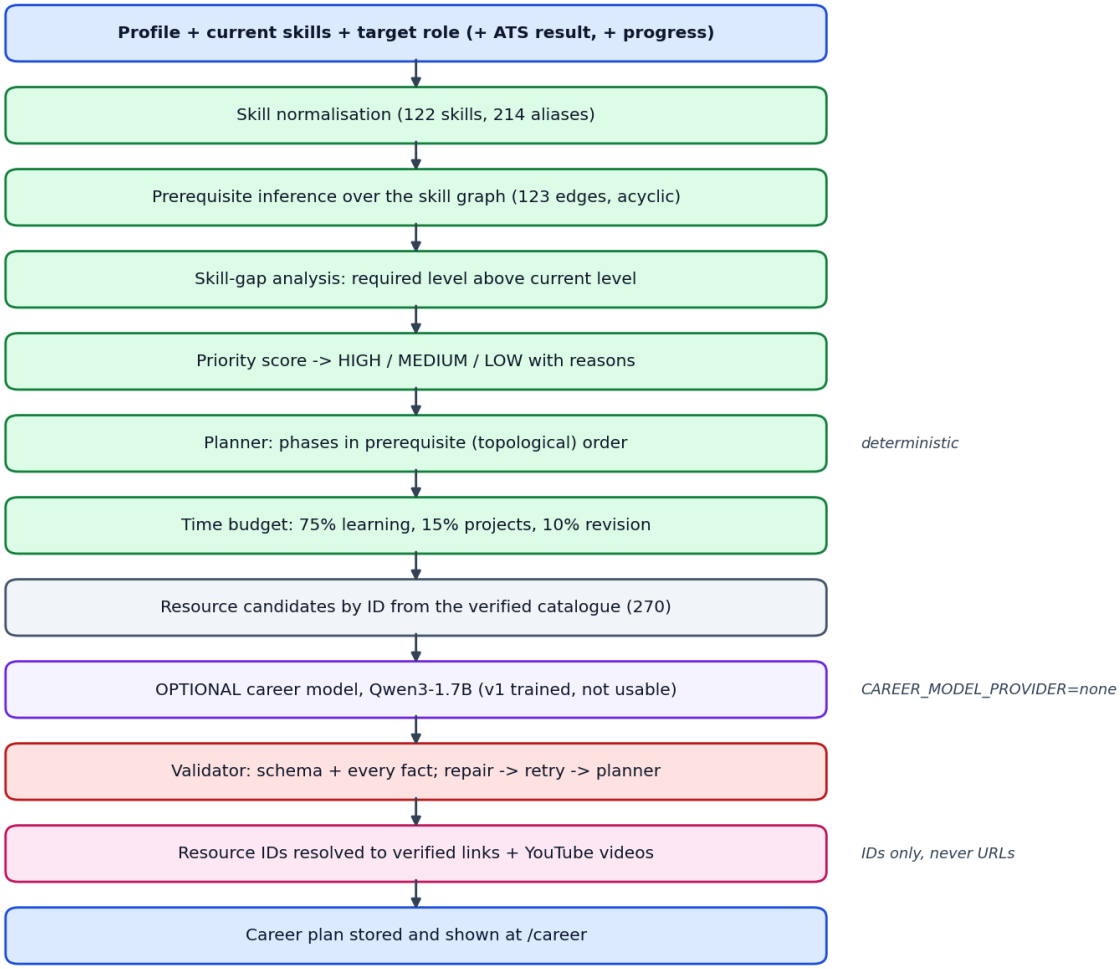


Figure 6. Career Plan pipeline

- **Problem:** "what should I learn, in what order, in the time I have, and from where?"
- **Input:** current skills with levels, target role (one of 20), hours per day, days per week, optional deadline, optional ATS result, optional progress.
- **Processing:**
  1. **Skill normalisation** against 122 skills and 214 aliases.
  2. **Prerequisite inference.** Knowing React at intermediate level is treated as evidence of at least beginner JavaScript.
  3. **Gap analysis.** For each role requirement, gap = required level above current level; missing prerequisites of gap skills are added.
  4. **Priority score.** `importance × 10 + 8 if core + 6 per skill it unblocks (cap 24) + 12 if the ATS analysis listed it as missing + 4 per level to climb`. HIGH at 62 or more, MEDIUM at 40 or more, otherwise LOW.
  5. **Planning.** Phases follow the prerequisite graph (123 edges, verified acyclic, depth 6), so a skill is never scheduled before what it depends on.

6. **Time budget.** 75% learning, 15% projects, 10% revision and interview preparation. If a deadline does not fit, low-priority gaps are deferred first; a skill is never deferred while a kept skill depends on it; if it still does not fit, the plan says the deadline is not achievable.
7. **Resources.** Candidates are chosen by ID from the verified catalogue.

- **Algorithm:** graph-based prerequisite ordering (topological order), additive priority scoring, feasibility-constrained scheduling.
- **Technology:** Python, JSON knowledge files under `ai-service/app/career/data`.
- **Database:** `CareerPlan`.
- **AI:** the rule-based planner produces every plan. The fine-tuned model is optional and off.
- **Output:** readiness summary, skill gaps with reasons, phased plan, weekly and daily schedule, projects, interview topics, resources per skill.
- **Time complexity:** O(V + E) for the graph (122 + 123) plus O(g log g) to sort *g* gaps.
- **Limitations:** the role knowledge (levels, hours, prerequisites) was written by one author and is not validated; 81% of role-to-skill links are corroborated by O*NET, ESCO or job postings, which supports the mapping only.
- **Status:** **IMPLEMENTED** with the planner; model path **PARTIALLY IMPLEMENTED**.

9.9 Learning-resource recommendation and YouTube integration

Figure 9. Resource recommendation and verification

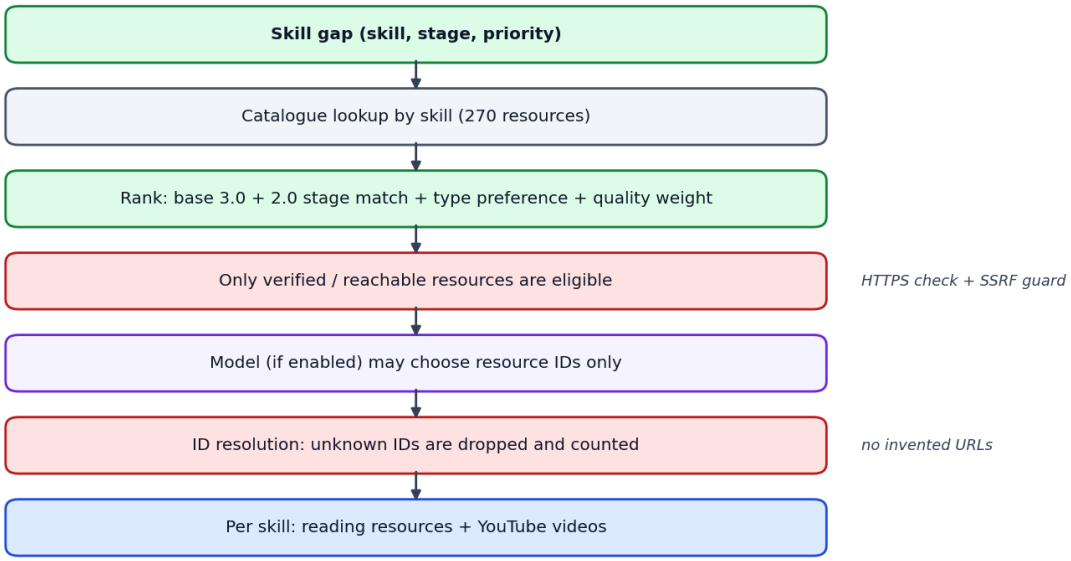


Figure 9. Resource recommendation and verification

- **Problem:** recommend material that exists and matches the skill and level.
- **Input:** a skill, a stage (beginner, intermediate, advanced), an optional preference.
- **Processing:** look up the catalogue by skill; score each resource `3.0 (covers the skill) + 2.0 if the difficulty matches the stage + 1.5 if verified (0.5 if only reachable) + quality weight (0 to 1) + 0.5 if the type matches the learner's preference + 0.3 if the language matches`; ties break on the resource ID so the order is reproducible. Unreachable resources are excluded.
- **The catalogue:** 270 resources — 65 documentation pages, 13 free courses, 8 practice platforms, 4 books and 180 YouTube videos. 239 verified, 29 reachable, 2 unreachable and excluded. 112 of 122 skills have at least one usable resource.
- **Verification:** each non-video URL was fetched over HTTPS. The verifier refuses anything but HTTPS on the provider's own domain, refuses hosts resolving to private or loopback addresses, follows at most three redirects and caps the bytes read. Videos were checked through the YouTube Data API as

public and embeddable.

- **AI:** none. `quality_score` is an editorial weight by resource type, not a user rating.
- **Status:** **IMPLEMENTED**

Figure 10. YouTube integration

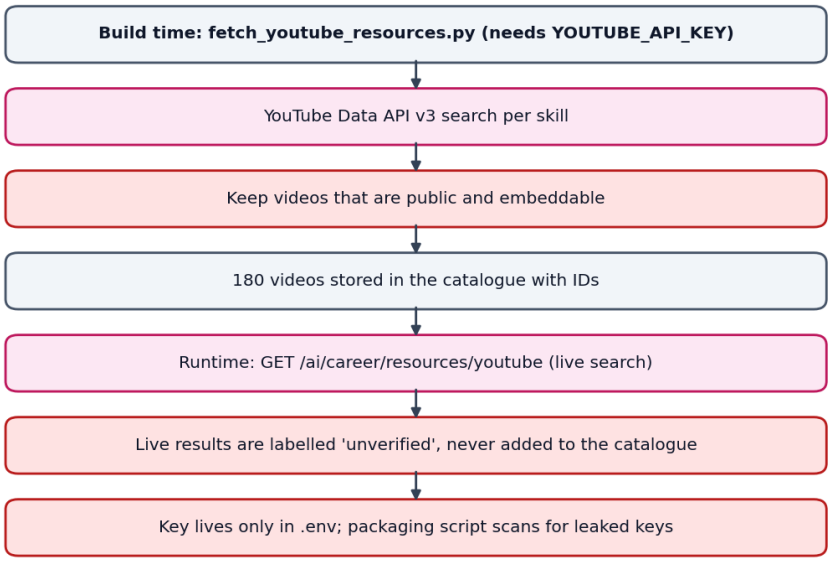


Figure 10. YouTube integration

Why the language model must not memorise links

Layer	Job	Why it is separate
Language model	Reasoning, personalisation, wording	A model's knowledge is frozen at training time
Retrieval layer	Current resources	Videos and courses are removed, renamed or replaced
Verification layer	Does the link work	A model cannot check a URL; it can only produce one that looks right
Ranking layer	Which resource fits	Ranking rules can change without retraining

A model asked to recall a URL will produce a plausible string that may never have existed. In this project the model is given resource **IDs** in the prompt and may only choose among them. An ID that is not in the catalogue is dropped and counted as a hallucination. A new resource therefore works without retraining.

9.10 RAG knowledge base

- **Problem:** answer placement questions from stored documents without inventing facts.
- **Processing:** documents are split into chunks of 900 characters with 150 overlap on paragraph and sentence boundaries, embedded, and stored. A question is embedded, the top 5 chunks are retrieved by cosine similarity, and chunks below the threshold (0.18 for the lexical embedder, 0.25 for semantic ones) are discarded. **If nothing survives, the LLM is not called** and a fixed "no verified information" answer is returned.
- **Database:** `RAGDocument`; vectors in the NumPy store by default.
- **Time complexity:** O(N · d) per query (exact search over *N* chunks of dimension *d*).
- **Limitations:** exact brute-force search suits a small corpus only.
- **Status:** **IMPLEMENTED**; available through the admin console.

9.11 Alumni mentorship

- **Processing:** mentors are ranked by `0.35 company + 0.20 role + 0.20 skills + 0.15 gap coverage + 0.05 rating + 0.05 availability`. A request leads to a scheduled session in a WebRTC room (signalling over Socket.IO). Structured feedback becomes a bounded ±4 modifier in APIE.
- **Database:** `AlumniProfile`, `MentorshipRequest`, `MentorshipSession`, `AlumniFeedback`.
- **Limitations:** WebRTC through restrictive networks needs a TURN server, which is configuration only.
- **Status:** **IMPLEMENTED**

9.12 Analytics, reports and administration

Figure 13. Administrator and placement-officer workflow

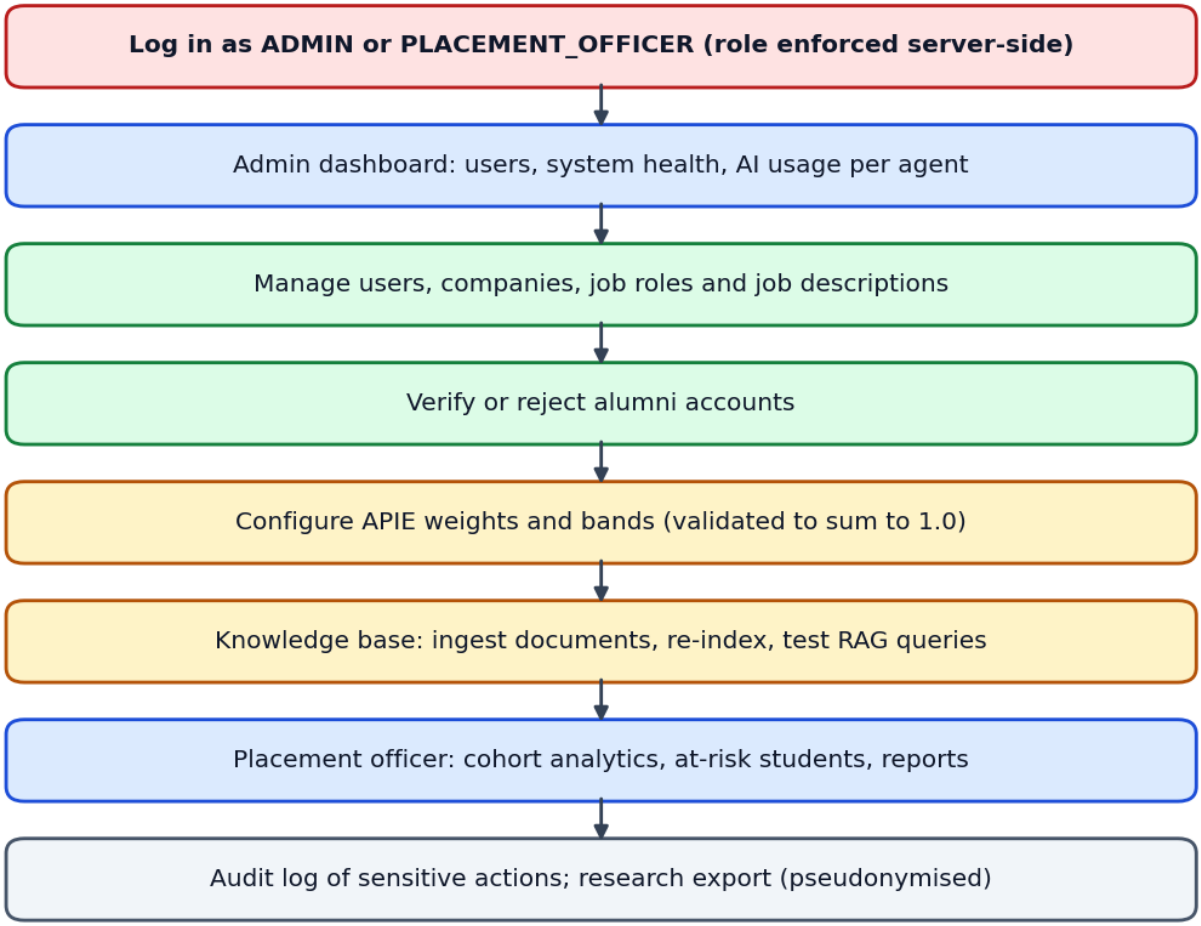


Figure 13. Administrator and placement-officer workflow

- **Placement officer:** cohort readiness, band distribution, common skill gaps, at-risk students, department comparison, a pseudonymised research export.
- **Administrator:** users and roles, companies and jobs, alumni verification, knowledge base, APIE weights (validated to sum to 1.0), AI usage per agent, audit log, flagged assessment attempts.
- **Reports:** ten PDF report types generated with PDFKit.
- **Algorithm:** MongoDB aggregation pipelines.
- **Status:** **IMPLEMENTED**

10. AI architecture

10.1 The design rule

Every AI feature separates seven things. Being able to name them is the strongest answer you can give about the AI design.

Part	In CareerPilot
MODEL	An LLM provider (or the mock), a Random Forest, optional Qwen3 adapters
DATA	Resumes, job descriptions, curated role knowledge, synthetic learner profiles
TRAINING	Offline, in Colab notebooks; never inside the application
INFERENCE	One call through the provider interface, with a timeout
RETRIEVAL	The RAG vector store and the resource catalogue
VERIFICATION	Pydantic schemas, grounding checks, resource-ID resolution, URL checks
DETERMINISTIC LOGIC	ATS score, APIE, job match, judge verdicts, the planner, all final arithmetic

Figure 15. AI architecture: what is deterministic and what is a model

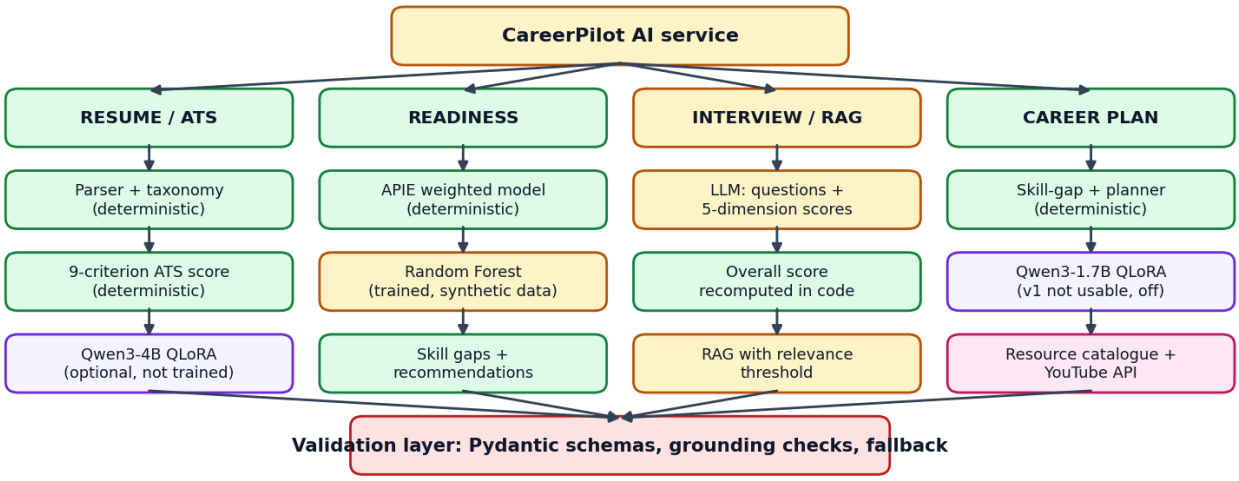


Figure 15. AI architecture: what is deterministic and what is a model

10.2 Agents and orchestration

An agent is a class with one responsibility, a typed input schema, a typed output schema and a `handle()` method. `BaseAgent.run()` wraps every agent with the same four steps: validate input, run, validate output, record latency and provider.

Agent	Responsibility	Uses an LLM
Resume Intelligence	Parse a resume, compute the ATS score, match JD keywords	Only to phrase advice, and only if the fine-tuned model is enabled
Job Matching	Score a student against a job	No
Coding Assessment	Static review of submitted code (advisory)	Yes
Technical / HR / Behavioural Interview	Generate the next question	Yes
Interview Evaluation	Score an answer on five dimensions	Yes; overall recomputed in code
Placement Readiness	Run APIE	No

Agent	Responsibility	Uses an LLM
Alumni Mentorship, Feedback Ingest	Match mentors; convert human feedback to signals	No
Analytics	Cohort statistics	No

There are **11 registered agents** and two fixed workflows: `RESUME_TO_MATCH` and `INTERVIEW_TURN`.

Agents never call each other. Composition happens only in the orchestrator, so the call graph is acyclic and every multi-step flow is a named, auditable operation.

> **If asked "is this really agentic AI?"** It is a multi-agent architecture with orchestrated, deterministic workflows. The agents are specialised and adaptive, but they do not plan autonomously or call tools in a loop. That was chosen for auditability in an assessment system.

10.3 The LLM layer

One interface (`complete_json`) with four providers selected by `AI_PROVIDER`: `openai` (any OpenAI-compatible endpoint), `azure`, `ollama` and `mock`. This is the Strategy pattern.

Three controls apply wherever a model is called:

1. **Structured output.** Prompts demand JSON matching a schema; the response is parsed and validated.
2. **Low temperature** (0.15–0.2 for evaluation) so the same answer gets a consistent score.
3. **Score recomputation.** The model returns dimension scores; the overall is recomputed from fixed weights.

The mock provider is not a language model. It assembles schema-valid responses with rules over the real input so the platform runs offline. Every mock response is tagged `provider: "mock"`.

11. Algorithms used in CareerPilot

Only algorithms present in the source are listed. TF-IDF and a learned neural ranker are **not** used anywhere; do not claim them.

11.1 Summary table

#	Algorithm	Used for	Complexity
1	Dictionary matching with custom word boundaries	Skill extraction	O(n · t)
2	Pattern-based section detection	Resume structure	O(lines)
3	Named-entity recognition (spaCy)	Candidate name	Linear in text
4	Capped weighted multi-criteria scoring	ATS score, APIE, job match, mentor match	O(k) criteria
5	Feature hashing with sub-linear term frequency	Default text embedding	O(n)
6	Cosine similarity	Job match, RAG	O(d)
7	Exact vector search	RAG retrieval	O(N · d)
8	Overlapping boundary-aware chunking	RAG ingestion	O(n)
9	Relevance-threshold gating	Hallucination control in RAG	O(k)
10	Harmonic recency weighting	APIE evidence	O(e)
11	Random Forest (and Gradient Boosting)	Readiness band second opinion	Train O(trees · n log n)
12	Threshold controller	Interview difficulty	O(1)
13	Topological ordering of a prerequisite graph	Career plan phases	O(V + E)
14	Additive priority scoring	Skill-gap priority	O(g)
15	Feasibility-constrained scheduling with deferral	Study plan	O(g log g)
16	Deterministic resource ranking	Resource recommendation	O(r log r)
17	Item-based collaborative filtering	Skill recommender (not live)	O(s · V) per query
18	MinHash + LSH near-duplicate clustering	ATS dataset splitting	Near-linear
19	Code generation from a typed signature	DSA harness and starter code	O(parameters)
20	Structural JSON comparison with tolerance	Judge comparator	O(answer size)
21	Verdict classification	Judge	O(1) per test
22	bcrypt, HMAC-signed JWT, token rotation	Security	Fixed cost
23	QLoRA	Fine-tuning both models	See training chapter

11.2 Skill extraction — dictionary matching

- **Input:** resume or JD text. **Output:** canonical skill names.
- **How it works:** each of 107 skills and 181 aliases is compiled to a regular expression with the boundary `(?<![a-zA-Z0-9+#])term(?![a-zA-Z0-9+#])`. Separators inside a term are flexible, so `node.js`, `node js` and `nodejs` all match. Aliases map to one canonical name.
- **Why selected:** deterministic, explainable, needs no training data, cannot invent a skill.
- **Alternative:** a neural NER model for skills. **Why not:** needs labelled data, can hallucinate, and is harder to explain to a student.
- **Limitation:** finds only skills in the taxonomy.

11.3 Feature hashing embedding

- **Input:** text. **Output:** a 384-dimensional unit vector.
- **Steps:** tokenise and count; weight each token `(1 + ln(count)) × sign`; add it to bucket `MD5(token) mod 384`; for tokens longer than five characters add character trigrams at weight 0.25; L2-normalise.
- **Why selected:** needs no download, no network and no GPU.
- **Alternative:** `sentence-transformers` (all-MiniLM-L6-v2) or OpenAI embeddings, selectable with `EMBEDDING_PROVIDER`. **Why not default:** they need a model download or an API key.
- **Be precise:** this is a *lexical* embedding. It captures shared words, not meaning.

11.4 Cosine similarity and vector search

`cos(a, b) = (a · b) / (‖a‖ ‖b‖)`. All vectors are L2-normalised, so cosine similarity equals the dot product and a search is one matrix–vector multiplication. The NumPy store and FAISS `IndexFlatIP` are both **exact** search. Approximate indexes (HNSW, IVF) are not used because the corpus is small.

11.5 Harmonic recency weighting

```
value = Σ (score_i × 1/(i+1)) / Σ 1/(i+1)        i = 0 for the newest result
```

Improvement shows quickly, but one bad day does not erase a track record. **Alternative:** a plain mean (ignores recency) or latest-only (too noisy).

11.6 Random Forest

- **Task:** classify a student into one of four readiness bands from 14 features (7 component scores, project count, certification count, internship months, alumni feedback score, open skill gaps, assessments taken, interviews taken).
- **Settings:** 300 trees, max depth 12, minimum 4 samples per leaf, balanced class weights, `StandardScaler → classifier`, 80/20 stratified split.
- **How it works:** each tree trains on a bootstrap sample and considers a random subset of features at each split; the forest votes. Averaging decorrelated trees reduces variance.
- **Why selected:** works well on small tabular data, needs little tuning, gives feature importances.
- **Alternative:** Gradient Boosting (available in the code), logistic regression, a neural network. **Why not a neural network:** far too little data.

11.7 Prerequisite graph and topological order

The skill graph has 122 nodes, 123 edges, 24 roots and maximum depth 6, verified acyclic at build time. A topological order guarantees every skill appears after its prerequisites. The planner groups gap skills into phases following that order. **Alternative:** ordering by priority alone, which could schedule React before JavaScript.

11.8 Skill-gap priority score

```
priority = importance × 10            role importance 1–5 (a prerequisite inherits 80% of the skill
  that needs it)
        + 8   if a core requirement
        + 6   per gap skill it unblocks (capped at 24)
        + 12  if the ATS analysis listed it as missing
        + 4   per level to climb
HIGH ≥ 62   MEDIUM ≥ 40   otherwise LOW
```

The weights are a stated design choice, not learned values. Each gap is returned with the reasons that produced its score.

11.9 Item-based collaborative filtering (skill recommender)

```
similarity(i, j) = cooccurrence(i, j) / ( sqrt(count_i × count_j) + shrinkage )      shrinkage = 5
score(candidate) = Σ similarity(candidate, s)  over the skills s the student has
```

Trained on 844 skill "baskets" extracted from real job descriptions and resumes. It beat a popularity baseline and a PPMI + SVD embedding model. **It is not wired into a live feature**: the agent that used it was removed. Its co-occurrence data is reused read-only as job-demand evidence when the career knowledge base is built.

11.10 Judge comparator and verdict classification

Both the expected and the returned value are parsed as JSON and compared structurally, so spacing, trailing newlines or `5` versus `5.0` cannot change a verdict. Floating-point answers use a relative tolerance of 1e-6. A problem may declare `unordered` or `unorderedNested` comparison (for example Two Sum, Subsets). The verdict rule is a fixed decision list (see the DSA module).

12. How the AI models are trained

12.1 Beginner: what is an LLM?

A large language model is a neural network trained to predict the next token (a word piece) given the text so far. It has seen a very large amount of text, so it can continue a prompt in a way that reads sensibly. It does not look facts up; it generates what is statistically likely.

12.2 Intermediate: what is fine-tuning?

Fine-tuning continues training a pre-trained model on a smaller, task-specific dataset so it behaves the way a task needs — here, producing a strict JSON structure for an ATS assessment or a career plan.

12.3 Advanced: the terms you must be able to define

Term	Definition	Simple example
SFT (supervised fine-tuning)	Training on input → target pairs	Resume in, the expected JSON assessment out
PEFT	Parameter-efficient fine-tuning: train a small number of extra parameters and freeze the rest	Adding sticky notes to a textbook instead of rewriting it
LoRA	Low-Rank Adaptation: for a weight matrix W, learn two small matrices A and B and use `W + (α/r)·B·A`	A 4096×4096 matrix has ~16.8M numbers; with rank 16, A and B together have ~131K
QLoRA	LoRA on a base model stored in 4-bit precision	The same textbook, printed smaller, with the same sticky notes
Quantization	Storing weights with fewer bits	16-bit to 4-bit cuts memory by about four times
4-bit NF4	A 4-bit format designed for normally distributed weights	Used by bitsandbytes
Rank (r)	Width of the adapter matrices; higher means more capacity	r = 16 here
Alpha (α)	Scaling of the adapter's contribution	α = 32 here, so the scale is α/r = 2
Batch size	Examples processed per forward pass	1 or 2 on a T4
Gradient accumulation	Summing gradients over several small batches before updating	1 × 16 behaves like a batch of 16
Gradient checkpointing	Recomputing activations during the backward pass to save memory	Slower, but fits in memory
Learning rate	Step size of each update	1e-4 here
Scheduler	How the learning rate changes over time	Cosine decay after a short warm-up
Optimizer	The update rule	`paged_adamw_8bit`
Epoch	One pass over the training set	2 planned
Validation	Measuring loss on data not trained on	Used for early stopping
Checkpoint	A saved copy of the adapter during training	Every 25 or 50 steps to Google Drive
Overfitting	Memorising training data instead of learning the task	Training loss falls while validation loss rises
Completion-only loss	Loss computed only on the answer tokens	The model learns the JSON, not to reproduce the resume

12.4 Full fine-tuning, LoRA and QLoRA compared

Figure 8. LoRA and QLoRA

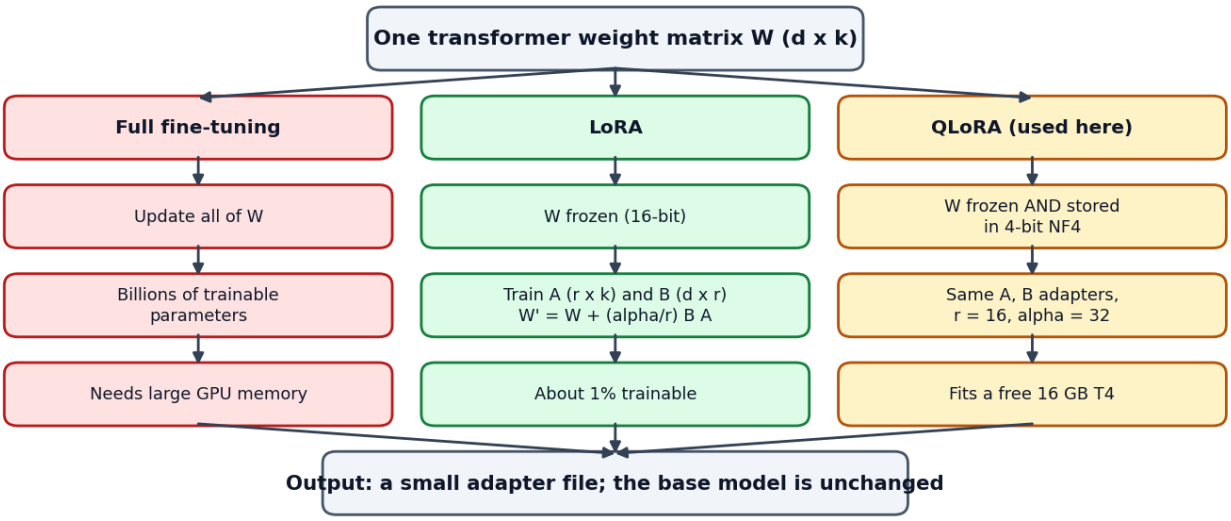


Figure 8. LoRA and QLoRA

	Full fine-tuning	LoRA	QLoRA (used here)
What is trained	Every weight	Small adapter matrices	Small adapter matrices
Base model during training	Updated, 16-bit	Frozen, 16-bit	Frozen, **4-bit**
Trainable parameters	100% (billions)	Around 1%	Around 1% (ATS: about 33 million, ~0.8%)
GPU memory	Very high	Moderate	Lowest; fits a free 16 GB T4
Output	A full model copy (gigabytes)	An adapter file	An adapter file
Risk of forgetting general ability	Higher	Low	Low
Cost	Highest	Low	Lowest, slightly slower per step

Why QLoRA was selected: the only GPU available was a free Colab T4 with 16 GB. Full fine-tuning a 4-billion-parameter model does not fit. QLoRA fits, produces a small adapter that can be versioned and swapped, and leaves the base model untouched so the untuned model can be evaluated for comparison with the adapter disabled.

12.5 The two models

	ATS model	Career model
Version name	`careerpilot-ats-qwen3-4b-v1`	`career-roadmap-qwen3-1.7b-v1`
Base model	Qwen/Qwen3-4B-Instruct-2507	Qwen/Qwen3-1.7B
Approximate size	4 billion parameters	1.7 billion parameters
Licence of base	Apache-2.0	Apache-2.0
Method	QLoRA, r 16, α 32, dropout 0.05, all 7 projection layers	Same
Trainable parameters	About 33 million (~0.8%)	**NOT VERIFIED FROM SOURCE** — not recorded
Quantization	4-bit (Unsloth bnb-4bit; fallback bitsandbytes NF4)	4-bit

	ATS model	Career model
Task	Component scores + evidence as JSON, two modes	A career plan as JSON, 32 task types
Training examples	2,162	7,157 available; **591 used in v1**
Validation / test	470 / 485 (+ 476 for human review)	957 / 1,802
Max sequence length	3,072 on a T4	4,096
Batch × accumulation	1 × 16	2 × 8 (v1); notebook changed to update every 4 for v2
Learning rate, schedule	1e-4, cosine, 10 warm-up steps, weight decay 0.01	1e-4, cosine, weight decay 0.01
Optimizer	`paged_adamw_8bit`	**NOT VERIFIED FROM SOURCE**
Epochs	2 with early stopping (patience 3)	2 planned; **1 in v1**
Checkpoints	Every 25 steps	Every 50 steps, latest 3 kept, auto-resume
Loss	Completion only	Answer tokens only
Hardware	Colab T4 (planned)	Colab T4
Training status	**Not run. No adapter exists in the repository.**	**v1 run: 37 updates, 82 minutes**
Serving status	Off (`ATS_MODEL_PROVIDER=none`)	Off (`CAREER_MODEL_PROVIDER=none`)

Why these base models

- **Open licence.** Both are Apache-2.0, so adapters can be trained and used freely.
- **Size fits the hardware.** A 4B model in 4-bit fits a T4 with sequences of about 3,000 tokens. The career model uses a smaller 1.7B model because its sequences are longer (up to 4,096 tokens) and it only has to organise facts supplied in the prompt.
- **Instruction-tuned.** The ATS base is the non-thinking instruct variant, so it emits JSON without a reasoning block. Qwen3-1.7B is a thinking model; prompts are rendered with thinking disabled and a test confirms the training prompt equals the serving prompt token for token.

No comparison with Llama, Mistral or Gemma was run. Do not claim Qwen is better than them; say it met the licence, size and format requirements.

12.6 ATS fine-tuning pipeline

Figure 7. Fine-tuning pipeline (both models)

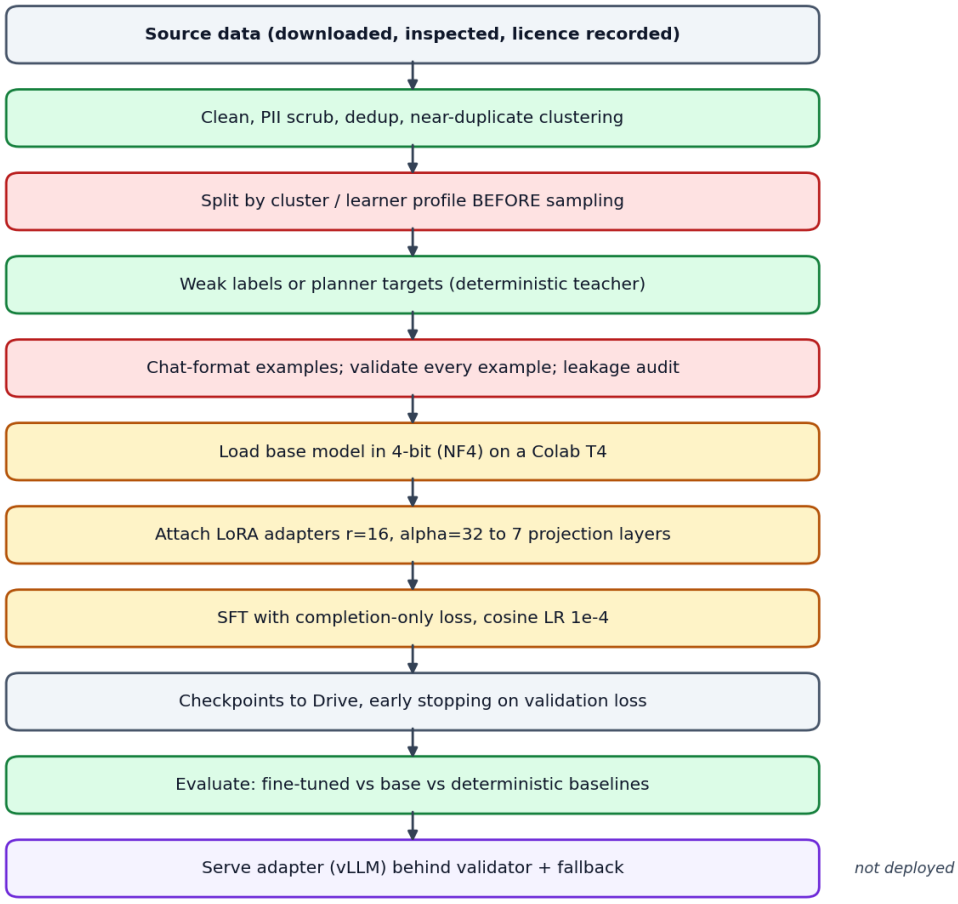


Figure 7. Fine-tuning pipeline (both models)

1. **Sources inspected, not assumed.** Three public datasets were downloaded and examined: 4,817 structured resumes (MIT), 6,374 resume–JD pairs (Apache-2.0) and 93,733 pairs (no licence declared).
2. **A provided score was rejected.** One dataset's numeric `ats_score` turned out to be "class band plus noise": within a fit class it correlated with nothing in the text ($\rho \approx -0.001$). Only the three-class fit label was kept.
3. **Cleaning and privacy.** Unicode normalisation, PII scrub (emails, phones, profile links replaced with placeholders), quality filter, exact de-duplication.
4. **Near-duplicate clustering.** MinHash (64 permutations) with LSH (16 bands), verified by exact Jaccard ≥ 0.8: 6,063 unique resumes became 5,346 clusters; 659 JDs became 382 clusters.
5. **Split before sampling, by cluster.** Resume clusters are assigned to train, validation, test or human review; JD clusters to train or held-out. A pair is usable only when both agree. The leakage audit reports 0 / 0 / 0 and the build refuses to write otherwise.
6. **Weak labels.** Every target is a documented deterministic function built on the existing ATS engine. **No label is produced by an LLM, and none is a human score.**
7. **Truncate before labelling**, so the model is never trained to cite a skill it cannot see.
8. **Training** in the Colab notebook with the settings in the table above.
9. **Evaluation** on three separate axes: teacher fidelity, external validity against fit labels on unseen JDs, and human agreement (not available until the review sheet is filled).
10. **Serving safety.** JSON extraction, strict schema, grounding guard, one corrective retry, then the deterministic engine.

The model predicts component scores and evidence only. The final ATS score is always computed in Python from fixed weights; a score the model emits is discarded.

12.7 Career model fine-tuning pipeline

1. **Knowledge base.** 122 skills, 214 aliases, 123 prerequisite links, 20 roles. O*NET 31.0 and ESCO v1.2.1 corroborate which skills belong to a role; they contain no prerequisites, study hours or projects, so those layers are hand-written and are editorial judgement.
2. **Examples.** A synthetic learner profile is passed through the deterministic planner; the plan becomes the target. 10,000 generated, 9,916 kept (84 exceeded 4,096 tokens).
3. **Split by learner profile.** Train 7,157, validation 957, test 1,802. Two roles — Site Reliability Engineer and Product Analyst — never appear in training, to test generalisation to an unseen role.
4. **Facts in the prompt.** Requirements, gaps with hours and priorities, prerequisite links and candidate resource IDs are supplied. The model explains and organises; it does not need to recall a requirement or a URL.
5. **Training run v1.** 591 examples, 1 epoch, 37 learning updates, 82 minutes on a T4, final validation loss 0.273.
6. **Result.** Not usable — see the evaluation chapter.

> **What this training teaches, honestly.** The targets come from a rule-based planner, so the model learns to reproduce that planner's reasoning in fluent form. It can be measured for validity and faithfulness to the supplied facts. It cannot be shown to give *better* career advice than the planner it learned from.

12.8 Inference and fallback

Situation	What happens
Model provider is `none` (current setting)	The deterministic engine or planner answers
Model returns valid, grounded JSON	The answer is used; scores are still recomputed in code
JSON invalid or fails the schema	One corrective retry showing the model its error
Still invalid, timeout, crash or out of memory	The deterministic result is returned, marked `degraded` with the reason
Model names a skill not in the resume as matched	Moved to missing and counted
Model names a resource ID not in the catalogue	Dropped and counted

13. Evaluation framework

13.1 Choosing the right metric

Accuracy is only meaningful for classification. Three kinds of quality must be kept apart.

Kind	Question	Examples here
MODEL QUALITY	Is the model's output correct?	F1 for the classifier; JSON validity and hallucination rate for an LLM
SYSTEM QUALITY	Does the pipeline behave correctly?	Tests passing, verdict correctness, fallback behaviour
USER EXPERIENCE	Is it useful to a student?	Human ratings — none collected yet

13.2 Formulas

```
Accuracy  = (TP + TN) / (TP + TN + FP + FN)        share of all predictions that are correct
Precision = TP / (TP + FP)                          of what was predicted positive, how much was rig
  ht
Recall    = TP / (TP + FN)                          of what was truly positive, how much was found
F1        = 2 × Precision × Recall / (Precision + Recall)
MAE       = mean |predicted − target|               average size of a score error
Spearman ρ                                          agreement of rankings, from −1 to +1
MRR       = mean (1 / rank of the correct item)
Recall@K  = share of cases where the correct item is in the top K
```

Metric	Use it when	Do not use it when
Accuracy	Classes are balanced	Classes are imbalanced — a model predicting the majority looks good
Precision	False positives are costly (recommending a skill that is not needed)	You need to know what was missed
Recall	False negatives are costly (missing a real skill)	You need to know how much noise was added
F1	You need one number balancing both	Costs of the two errors differ greatly
MAE, Spearman	The output is a score or a ranking	The output is a category
JSON validity, hallucination rate	The output is generated structured text	—

13.3 What should be measured for each module

Module	Metrics	Available now?
ATS rule-based engine	MAE and rank correlation against weak labels; Spearman against fit labels	**MEASURED** — see the metrics report
ATS fine-tuned model	JSON validity, skill precision/recall/F1, hallucination rate, MAE, Spearman vs fit label on unseen JDs	**NOT YET MEASURED**
ATS human agreement	Correlation with placement-officer scores	**NOT YET MEASURED** — the review sheet is empty
Readiness classifier	Accuracy, macro precision/recall/F1, confusion matrix, cross-validation	**MEASURED** on synthetic data
Skill recommender	MRR, Recall@5, Recall@10 with a confidence interval	**MEASURED**
RAG	Relevance score separation for on-topic and off-topic questions	**MEASURED** informally for threshold calibration

Module	Metrics	Available now?
Career model	13 measures including JSON validity, schema compliance, skill-gap F1, feasibility, resource hallucination	**MEASURED** for v1 on 60 examples
Career plan quality	Expert rating on seven criteria	**NOT YET MEASURED** — sheet generated, cells empty
DSA judge	Reference solutions accepted; verdict cases	**MEASURED** with local compilers
DSA latency, throughput, memory	Per-submission time, concurrent capacity	**NOT YET MEASURED**
API latency, page load	Benchmarks	**NOT YET MEASURED**

> **Rule for the viva.** If a number is not in the metrics report that follows, say "that has not been measured yet" and then say how you would measure it. Never estimate a number aloud.

14. Metrics and evaluation report

Every number here is copied from a file in the repository, named in the "Source" column. Nothing is estimated. Where a measurement does not exist the table says **NOT YET MEASURED** and how to obtain it.

14.1 Measured — ATS deterministic baselines

Source: `careerpilot_ats/evaluation/metrics.json`. Test split of 485 records (263 general, 222 job-match). "Teacher fidelity" compares a system's score with the weak label. "Versus fit label" compares with a dataset label that no system sees at test time, on job descriptions never seen in training.

System	Mode	MAE	RMSE	Pearson	Spearman	Spearman vs fit label
Predict the training mean (trivial floor)	General	7.50	10.31	—	—	—
Predict the training mean (trivial floor)	Job match	10.96	13.37	—	—	—
Existing rule-based engine	General	5.18	5.97	0.941	0.873	—
Existing engine (keyword coverage)	Job match	24.86*	27.93*	0.823	0.795	0.079
Rubric heuristic without label anchoring	Job match	4.05	5.13	0.948	0.942	0.227
Base Qwen3-4B	Both	**NOT YET MEASURED**				
Fine-tuned Qwen3-4B	Both	**NOT YET MEASURED**				

* Different scale: keyword coverage is not a 0–100 fit score, so its MAE is not comparable; its rank correlations are.

Meaning. These describe how closely the *rule-based* engine follows the documented rubric (weak labels). They are not accuracy against human judgement. The key finding is that plain keyword coverage barely tracks job fit on unseen jobs (ρ = 0.079), which is the headroom a fine-tuned model would have to beat (ρ = 0.227 for the richer heuristic).

Human agreement: NOT YET MEASURED. `careerpilot_ats/dataset/careerpilot_human_review.csv` holds 476 examples with empty score columns. Evaluation required before claiming this metric.

14.2 Measured — ATS dataset

Source: `careerpilot_ats/README.md`, `logs/dataset_report.json`.

Split	Total	General	Job match (No / Potential / Good fit)
Train	2,162	1,225	937 (353 / 335 / 249)
Validation	470	263	207 (84 / 73 / 50)
Test	485	263	222 (89 / 80 / 53)
Human review	476	250	226 (85 / 83 / 58)

Leakage audit across all split pairs: 0 resume clusters, 0 exact pairs, 0 job-description clusters shared. Longest example: 2,771 tokens.

14.3 Measured — readiness classifier

Source: `ai-service/artifacts/readiness_metrics.json`, trained 7 October 2026. **Dataset provenance: SYNTHETIC.**

Metric	Value	Dataset	Method
Accuracy	0.735	400 synthetic test rows	80/20 stratified split of 2,000 rows
Macro precision	0.740	same	same
Macro recall	0.737	same	same
Macro F1	0.737	same	same
Cross-validation accuracy	0.739 ± 0.020	2,000 rows	3-fold stratified

Class	Precision	Recall	F1	Support
At Risk	0.806	0.823	0.814	96
Needs Improvement	0.674	0.600	0.635	100
Placement Ready	0.661	0.739	0.698	111
Highly Ready	0.820	0.785	0.802	93

Confusion matrix (rows = actual, columns = predicted, in the class order above):

```
At Risk            79  16   1   0
Needs Improvement  18  60  21   1
Placement Ready     1  13  82  15
Highly Ready        0   0  20  73
```

Errors fall almost entirely on neighbouring bands, which is what you expect from an ordered target with noise. Top feature importances: academic 0.191, coding 0.157, technical 0.151, ATS 0.099, communication 0.087.

Meaning. "Prototype evaluation — synthetic dataset." It shows the pipeline works. It is **not** evidence that the model predicts real placements.

14.4 Measured — skill recommender

Source: `ai-service/artifacts/skill_recommender.json`, `careerpilot_skills/skill_recommender_report.json`. Real data: 844 skill baskets (594 train, 115 validation, 135 test), 84 skills. Leave-one-out evaluation, 1,473 trials.

Model	MRR	Recall@5	Recall@10
Popularity baseline	0.195	0.272	0.476
Item similarity (selected)	**0.313**	**0.411**	**0.585**
PPMI + SVD embeddings	0.238	0.348	0.483

Gain over popularity in Recall@5: +0.138, 95% confidence interval +0.101 to +0.176 (paired bootstrap, 2,000 rounds). **Status:** trained and evaluated, but not connected to a live feature.

14.5 Measured — career model, run v1

Source: `careerpilot_career/models/career_roadmap/evaluation/evaluation_report.json` (11:37, 7 October 2026) and `final/model_metadata.json`. Training: 591 examples, 1 epoch, 37 learning updates, 82 minutes on a Colab T4, final validation loss 0.273. Evaluation on held-out test examples **before any repair**.

Metric	Fine-tuned v1 (60 examples)	Untuned base (8 examples)
Valid JSON	51.7%	87.5%
Passes the schema	6.7%	0%

Metric	Fine-tuned v1 (60 examples)	Untuned base (8 examples)
Fully valid plan without repair	**0%**	0%
Role alignment (where checkable)	100%	—
Resource-ID validity (where checkable)	100%	—
Resource hallucination (where checkable)	0%	—

Only four outputs passed the schema, so the remaining ten measures rest on too few cases to mean anything. The base model was run on 8 examples only, so this is **not** a fair before/after comparison.

Verdict: not usable. 21 of 60 outputs never stopped and were cut at the 3,000-token limit; 8 more had broken JSON; training loss was still falling (0.65 to 0.30) when the run ended. The model is switched off and the planner generates every plan.

14.6 Measured — career knowledge and resources

Source: `ai-service/app/career/data/*.json`.

Item	Value
Skills / aliases	122 / 214
Prerequisite edges, roots, maximum depth	123, 24, 6 (verified acyclic)
Roles	20
Role-to-skill links corroborated by O*NET, ESCO or job postings	222 of 275 (80.7%)
Resources	270 (239 verified, 29 reachable, 2 unreachable and excluded)
By type	180 videos, 65 documentation, 13 free courses, 8 practice platforms, 4 books
Skills with a usable resource	112 of 122

14.7 Measured — RAG threshold calibration

Source: comment and settings in `ai-service/app/core/config.py`. Cosine scores with the lexical embedder against the seeded corpus: on-topic questions 0.28–0.31; domain-adjacent but unanswerable 0.11–0.14; unrelated 0.02–0.07. Threshold 0.18. This was an informal calibration, not a labelled retrieval benchmark; precision and recall at k are **NOT YET MEASURED**.

14.8 Measured — DSA judge correctness

Source: `backend/tests/dsa.judge.test.ts` (full matrix run).

Check	Result
Reference solutions judged Accepted	28 problems × 4 languages = 112 programs, every public and hidden test
Wrong Answer, Runtime Error, Compilation Error, Time Limit Exceeded	Each reproduced with a real program
Hidden test data in any API response or stored record	None (asserted by test)
Execution environment for these checks	Local compilers through a test-only transport — **not** Docker
Container isolation (network, filesystem, fork bomb, memory)	**NOT YET MEASURED** — run `npm run judge:verify` on a machine with Docker

14.9 Measured — automated tests

Suite	Result	Notes
Backend (Vitest)	128 passed, 0 failed (7 files)	auth, RBAC, assessment, voice, DSA judge, DSA API, resume/JD
AI service (pytest)	546 passed, 0 failed	agents, ATS model layer, career planner and service, RAG/APIE/ML, resume pipeline, recommender, voice
Frontend (Vitest)	39 passed, 0 failed (3 files)	components, formatting, speech
End-to-end, platform	61 checks passed	against the live AI service
End-to-end, voice interview	19 checks passed	against the live AI service
End-to-end, career plan	25 checks passed	against the live AI service

Passing tests show **system quality** — the code does what it was written to do. They are not a measure of model quality or of usefulness to students.

14.10 Not yet measured — and how to measure it

Metric	Why it is missing	How to measure
Fine-tuned ATS model: JSON validity, skill precision/recall/F1, hallucination rate, MAE, Spearman vs fit label	Training needs a GPU and has not been run	Run the notebook on a T4; cells 14–15 produce the comparison against the base model and the baselines above
ATS human agreement	No reviewer has filled the sheet	Have placement officers score 100 examples; report Pearson and Spearman with the engine
Career plan quality	No expert ratings	`evaluate.py human-sheet` produces a seven-criterion sheet; average the 1–5 ratings
Career model v2	Not run	Run the notebook at `medium` (about 375 updates) and regenerate the evaluation table
APIE validity	No real outcomes	Correlate readiness scores with placement results; report calibration
Classifier on real data	No real data	Retrain and report macro F1 with a confusion matrix
RAG retrieval quality	No labelled questions	Build a question set with relevant chunks; report Precision@5, Recall@5, MRR
Interview scoring agreement	No human scores	Have two raters score 50 answers; report inter-rater and model agreement
API latency and throughput	No load test	k6 or autocannon; p50/p95/p99 at 10, 50, 100 users
Judge latency and capacity	Docker not installed here	Per-language compile and run time over 50 submissions; queue wait under concurrency
Model inference latency, GPU memory	No model is served	Tokens per second and time to a complete JSON answer on the serving GPU
Frontend load time, bundle size	Not profiled	Lighthouse; bundle analyser with and without code splitting

15. Performance and speed

No latency, throughput or load benchmark exists in the repository. The table lists the techniques that are actually in the code and marks every one as unmeasured. **Do not quote a speed-up number.**

Technique	Problem it addresses	Where	Measured ?	Expected benefit
Database indexes (about 186 declarations)	Slow lookups by student, slug, status, date	Mongoose schemas	No — performance benchmark required	Indexed queries instead of collection scans
Pagination	Large lists	List endpoints (`page`, `limit`)	No	Bounded response size
Field selection and `lean()`	Loading whole documents	Services	No	Less data read and serialised
`select: false` on hidden tests and solutions	Loading large arrays that are rarely needed	`CodingProblem`	No	Smaller reads; also a security control
Route-level code splitting (35 lazy-loaded pages)	Large first download	`React.lazy` in `App.tsx`	No	Smaller initial bundle; Monaco loads only on the problem page
Client cache	Repeated identical requests	TanStack Query	No	Fewer round trips
Response compression	Payload size	`compression` middleware	No	Smaller responses
In-memory configuration cache	Reading weights on every request	`config.service`	No	No database read per readiness computation
Docker status cache (20 s)	Checking Docker on every page view	`dockerRunner`	No	One check per 20 seconds
Concurrency cap on sandboxes	Host exhaustion under a burst	Semaphore in `codeExecution.service`	No	Bounded resource use; excess requests wait or are refused
Compile once per submission	Recompiling for each test	Runner script	No	One compile, many test runs
Stop after a time-limit or memory failure	A hanging program occupying a sandbox for every test	Judge	No	Bounded worst-case time per submission
Normalised vectors	Computing norms at query time	Embeddings	No	Similarity is one dot product
4-bit quantization (QLoRA)	Model does not fit GPU memory	Training notebooks	Not as a benchmark	Training fits a 16 GB T4
LoRA adapters	Training and storing full model copies	Training notebooks	Not as a benchmark	Around 1% of parameters trained; small adapter file
Deterministic engines by default	Slow, costly model calls for every score	ATS, APIE, planner, judge	No	Scores need no GPU and no network

Not present in the repository: a queue, a Redis-style cache, response streaming, batching of model calls, approximate vector indexes, precomputation jobs.

15.1 Recommended benchmark method

1. **API latency.** Use a load tool (k6 or autocannon) against five endpoints: login, dashboard, problem list, resume analyse, career plan. Report p50, p95 and p99 at 10, 50 and 100 virtual users.

2. **Index effect.** Run the same query with `explain("executionStats")` with and without the index; report documents examined and time.
3. **Judge.** For each language submit a reference solution 50 times; report compile time, per-test time, total time and container start-up overhead; then raise concurrency and report the queue wait.
4. **Frontend.** Lighthouse on the dashboard and the problem page; report bundle sizes with and without code splitting.
5. **Model.** If an adapter is served, report tokens per second, time to a complete JSON answer and GPU memory.

16. Novelty, innovation and engineering contribution

Be exact about which word applies. A jury will respect "this is integration and careful engineering" far more than an inflated novelty claim.

Term	Meaning
NOVELTY	Something not done before in this form
INNOVATION	A new combination or application of known ideas
ENGINEERING IMPROVEMENT	A known idea implemented carefully to remove a real weakness
FEATURE	A capability users expect; ordinary functionality

16.1 What the project contributes

#	Contribution	Category	Existing problem	CareerPilot approach	Benefit
1	One readiness engine (APIE) fed by every module	INNOVATION	Scores from separate tools cannot be combined	A weighted, recency-aware, evidence-aware score recomputed after every activity, with human alumni feedback as a bounded modifier	One explainable number and a priority list
2	"Facts from code, judgement from the model, arithmetic from code"	INNOVATION	LLM-scored systems are unrepeatable and can hallucinate	Models return dimension scores or wording; final scores, matched skills and links come from deterministic code	Scores survive an LLM outage and can be explained
3	Resource IDs instead of URLs	ENGINEERING IMPROVEMENT	Models invent links	The model may only choose catalogue IDs; unknown IDs are dropped and counted	A hallucinated link cannot reach a student
4	Hidden tests that never enter the sandbox	ENGINEERING IMPROVEMENT	A naive judge ships expected outputs with the code	Only inputs are streamed, one test at a time; comparison is on the host	Submitted code cannot read the answers
5	One typed signature generating four language harnesses	ENGINEERING IMPROVEMENT	Per-language problem definitions drift	Starter code and harness are generated from a signature	Adding a language changes no problem
6	Leakage-controlled dataset construction	ENGINEERING IMPROVEMENT	Near-duplicate resumes leak between train and test	Cluster first, split by cluster, hold out job descriptions, audit and refuse to write on leakage	The test set measures unseen jobs
7	Honest failure reporting	ENGINEERING IMPROVEMENT	Systems fake a result when a dependency is down	"Not executed", "degraded", "no verified information", provenance labels	Trustworthy outputs
8	Voice excluded from evaluation	ENGINEERING IMPROVEMENT	Spoken answers can be judged on accent or delivery	The input mode is stored but never sent to the evaluator	Identical words get identical scores
9	Resume analysis, DSA, interviews, career plan, mentorship in one account	FEATURE (integration)	Students use many tools	One platform	Convenience

#	Contribution	Category	Existing problem	CareerPilot approach	Benefit
10	Admin screens, CRUD, reports	FEATURE	—	Standard	—

What is genuinely novel? Nothing here is a new algorithm. The honest claim is: *the integration of a single explainable readiness model with a strict separation between deterministic scoring and generative language, applied to campus placement.* Rows 1 and 2 are the ones to lead with.

What to avoid claiming: that the ATS model is "AI-powered" in the sense of a trained LLM (it is rule-based today); that the career plan is "AI-generated" by a fine-tuned model (the planner generates it); that any accuracy figure reflects real students.

17. Comparison with conventional platforms

Compared with a *conventional single-purpose platform* in each category. Many commercial products combine several of these; the table describes the typical single-purpose tool, not a specific competitor.

Capability	Typical job portal	Typical resume checker	Typical learning platform	Typical DSA platform	CareerPilot AI
Resume analysis	Limited	Yes	No	No	Yes, with a nine-criterion breakdown
JD comparison	Sometimes	Often	No	No	Yes, predefined or custom
Career guidance	Limited	No	Course suggestions	No	Skill-gap based plan
Skill gap vs a role	No	Sometimes	Sometimes	No	Yes, with priorities and reasons
Roadmap with schedule	No	No	Learning paths	Study plans	Yes, time-feasible
Learning resources	No	No	Own catalogue	Editorials	Verified catalogue + videos
DSA practice with a judge	No	No	Sometimes	Yes, far larger problem sets	Yes, 28 problems
Mock interview	No	No	Sometimes	Sometimes	Yes, text and voice
Combined readiness score	No	No	No	No	Yes (prototype weights)
Alumni mentorship	No	No	No	No	Yes
Institutional analytics	Recruiter-side	No	Sometimes	Sometimes	Yes, placement-side

Where CareerPilot is weaker: a dedicated DSA site has thousands of problems, a hardened production judge and contest features; a dedicated learning platform has far more content. CareerPilot's value is the connection between modules, not depth in any single one.

18. Limitations and future work

18.1 Current limitations

Area	Limitation
ATS LLM	Not trained. No adapter exists; no LLM result may be quoted
Career LLM	v1 trained on 591 examples for 37 updates; 0% of plans were fully valid; switched off
APIE	Weights and band thresholds are design choices, not validated on real placements
ML classifier	Trained and evaluated on synthetic data only
Embeddings	The default is lexical (feature hashing), not semantic
Skill extraction	Finds only the 107 taxonomy skills; no OCR for scanned resumes
LLM provider	The local configuration uses the mock provider, which is not a language model
DSA sandbox	Not yet executed on a machine with Docker; isolation is unverified there
DSA scale	28 hand-written problems; no queue; no admin problem editor; C not supported; no custom input
Sandbox hardening	Docker's default seccomp profile; container isolation is not a VM boundary
Career knowledge	Levels, hours and prerequisites written by one author; not reviewed by placement staff
Resources	10 skills have no usable resource; videos are search results that passed an availability check, not quality-reviewed
Human evaluation	The ATS review sheet and the career plan review sheet are empty
Data	Training resumes are mostly US-style, not Indian campus resumes
Performance	No benchmark has been run
Security	No multi-factor authentication; no dedicated prompt-injection filter
Operations	No CI pipeline, monitoring or cloud deployment in the repository
Roles	Recruiter is reserved but not built

18.2 Future enhancements

All items below are **PLANNED**.

1. Train the ATS model on a GPU and record the comparison against the deterministic baselines.
2. Run the career model at `medium` size (about 375 updates) and compare with v1.
3. Collect human scores from placement officers and report agreement.
4. Collect real placement outcomes, validate APIE weights and retrain the classifier.
5. Install Docker, run `npm run judge:verify`, then add a queue with judge workers and a stricter seccomp profile.
6. Add an admin problem editor, more problems, custom test input and C support.
7. Switch to semantic embeddings and evaluate retrieval quality.
8. Add Indian campus resumes with consent and recalibrate.
9. Add CI, monitoring, load testing and a recruiter module.

19. End-to-end demo script

19.1 Before you start

- Run `npm run dev` and open `http://localhost:5173`. Confirm all three services are up.
- Demo accounts (password `Demo@Pass123`): `student@careerpilot.demo`, `alumni@careerpilot.demo`, `officer@careerpilot.demo`, `admin@careerpilot.demo`. Confirm each logs in before the session.
- Have one resume PDF and one job description (text or file) ready.
- **Docker:** if Docker Desktop is not installed and the judge image is not built, Run and Submit on DSA problems will say "your code was not run". Decide beforehand whether to show that honestly or to install Docker and run `npm run judge:build` and `npm run judge:verify` first.
- **LLM:** the local setting is `AI_PROVIDER=mock`. Interview questions and written feedback will be marked as mock output. Say so if asked.

19.2 Screen-by-screen

#	Screen	What to show	What to say	Technical explanation	Why it matters
1	Login	Sign in as the student	"Access is role-based; this account can only see its own data."	bcrypt password check, 15-minute JWT, refresh token in an httpOnly cookie with rotation	Security is the base of everything else
2	Dashboard	Readiness score, component cards, DSA progress, next action	"One score built from seven components, updated after every activity."	APIE: recency weighting, weighted sum, bounded modifiers, bands	The student sees what to fix first
3	Profile	Skills, CGPA, target companies and roles	"Targets drive the gap analysis and job matching."	`StudentProfile` with arrays of target ids	Personalisation needs a profile
4	Resume & ATS	Upload a PDF	"The resume is parsed and scored with nine transparent criteria."	PyMuPDF extraction, section detection, taxonomy skill matching, capped rule-based scoring	Explainable, repeatable, no GPU
5	Resume & ATS	Choose "My own job description", paste or upload a JD, analyse	"Matched and missing skills come from exact matching, not a guess."	JD keywords canonicalised; coverage = matched ÷ total	Shows what to add for a specific job
6	Job Matching	Pick a company and role	"Eligibility is a hard gate; the score cannot override a CGPA cut-off."	0.55 required + 0.20 preferred + 0.15 similarity + 0.10 eligibility	Mirrors real screening
7	Assessments	Start one, refresh mid-test	"The timer lives on the server; the page recovers after a refresh."	Server-stored deadline, autosave, server-side grading	Fairness and reliability
8	DSA Practice	Filter by tag, open Two Sum, switch language	"The student writes only the function. Starter code for four languages comes from one signature."	Code generation from a typed signature	Extensible by design
9	DSA Practice	Run, then Submit	"Run uses public tests. Submit adds hidden tests whose inputs are never shown."	One container per submission; expected outputs stay on the host; deterministic verdict	Trustworthy verdicts, safe execution
10	DSA Practice	Results tab	"Execution Result is from the judge. Static Code Analysis is advisory AI and cannot change the verdict."	Verdict stored before the review is requested	AI does not decide correctness

#	Screen	What to show	What to say	Technical explanation	Why it matters
11	AI Interview	Answer two questions	"Difficulty adapts to my recent scores, and each answer gets five dimension scores."	Threshold controller; overall recomputed in code with mode weights	Adaptive practice with explainable feedback
12	Readiness	Component breakdown, ML second opinion	"The rule-based score and a Random Forest prediction are shown side by side, labelled as a prototype."	APIE plus a classifier trained on synthetic data	Honest about what is validated
13	Career Plan	Pick a role, set hours, generate	"Gaps are prioritised with reasons, ordered by prerequisites, and fitted to my time."	Skill graph, priority score, feasibility scheduling, verified resources by ID	A plan that can actually be followed
14	Career Plan	Resources tab	"Every link was checked; videos come from the YouTube API."	Catalogue of 270 resources with verification status	No invented links
15	Alumni	Request a mentor	"Mentors are ranked by company, role and skill overlap."	Weighted match formula; WebRTC room; feedback feeds APIE	Human in the loop
16	Officer login	Cohort analytics, at-risk list	"Placement staff see who needs help before results arrive."	MongoDB aggregation pipelines	Institutional value
17	Admin login	Users, AI usage, APIE weights, audit log	"Weights are configurable and every sensitive action is audited."	`SystemConfiguration`, `AuditLog`, weights validated to sum to 1.0	Governance

19.3 If something fails during the demo

Failure	What to say
DSA Run says "not executed"	"The sandbox needs Docker on this machine. The system refuses to guess a verdict; it never runs student code on the server as a fallback."
AI features fail	"The AI service is a separate process. Scores still work because they are deterministic; only wording degrades."
A page is slow	"Analysis runs synchronously in this prototype; a queue is planned."

20. Five-minute presentation

Speak at a steady pace; each block is about 70 to 130 words.

20.1 0:00 – 0:30 Problem

"A student preparing for placement uses one tool for the resume, another for coding, another for aptitude and another for interviews. None of them share information. So the student cannot answer two simple questions: how ready am I overall, and what should I fix first? The placement cell has the same blind spot across the whole batch."

20.2 0:30 – 1:00 Existing limitations

"Single-purpose tools give isolated scores. Many resume checkers return a number without explaining it. And AI-only tools can produce different answers for the same input, or invent facts. For something that affects a student's confidence, a score has to be explainable and repeatable."

20.3 1:00 – 1:30 Proposed solution

"CareerPilot AI puts the whole preparation journey on one account: resume analysis, job matching, assessments, DSA practice, mock interviews, a career plan and alumni mentorship. One engine, which we call APIE, combines all the evidence into a single Placement Readiness Score and tells the student what to work on next."

20.4 1:30 – 2:30 Architecture

"There are three services. A React and TypeScript frontend. A Node.js and Express gateway that handles authentication, role-based access, MongoDB and real-time events. And a Python FastAPI service that holds the AI: eleven agents, the resume parser, the scoring engines, retrieval and machine learning. The browser never talks to the AI service; only the gateway does, with an internal key. Student code for DSA problems runs in a separate isolated Docker container, never on the server."

20.5 2:30 – 3:30 AI and ML

"Our design rule is: facts from code, judgement from the model, arithmetic from code. The ATS score is nine rule-based criteria. The readiness score is a weighted model. A language model writes interview questions and scores answers on five dimensions, but the final score is recomputed in code. We also built QLoRA fine-tuning pipelines for two Qwen models. I want to be accurate here: the ATS model has not been trained yet, and the first career model we trained was measured and found not usable, so both are switched off. The product runs on the deterministic engines."

20.6 3:30 – 4:15 Key functionality

"Three things to notice in the demo. First, resume analysis with a comparison against any job description, showing exactly which skills are missing. Second, the DSA environment: four languages, hidden test cases that never enter the sandbox, and a deterministic verdict. Third, the career plan: skill gaps prioritised with reasons, ordered by prerequisites, fitted to the time the student has, with verified resources."

20.7 4:15 – 4:45 Innovation

"We do not claim a new algorithm. Our contribution is a single explainable readiness model fed by every module, and a strict separation between deterministic scoring and generative language. A language model here cannot set a score, cannot invent a link, and cannot decide whether code is correct."

20.8 4:45 – 5:00 Results and future scope

"The platform is tested across all three services, and every one of our 28 reference solutions is accepted in all four languages. Next we will train and evaluate the models on a GPU, validate the readiness weights with real placement outcomes, and collect human ratings. Thank you."

21. Ten-minute technical presentation

Time	Section	Speaking points
0:00 – 0:45	Problem	Preparation is spread over disconnected tools. No overall readiness view for the student or the placement cell. Unexplained scores reduce trust.
0:45 – 1:30	Existing system	Single-purpose resume checkers, coding sites and learning platforms. Each is good at one thing; none combine the evidence. AI-only scoring is not repeatable.
1:30 – 2:15	Proposed system	One platform, four roles (student, alumni, placement officer, administrator). One readiness engine fed by every scored activity.
2:15 – 3:30	Architecture	React + TypeScript SPA. Node.js + Express gateway: JWT, RBAC, Zod validation, MongoDB with 33 models, Socket.IO, 135 endpoints. Python FastAPI AI service: 11 agents and an orchestrator. One trust boundary: only the gateway calls the AI service. Docker Compose for deployment.
3:30 – 4:30	Modules	Resume and ATS; job matching; assessments with a server-side timer; DSA judge; AI interview with voice; APIE readiness; career plan; mentorship; analytics and PDF reports.
4:30 – 5:30	Algorithms	Dictionary skill extraction with custom boundaries. Capped multi-criteria ATS scoring. Feature-hashing embeddings and cosine similarity. RAG with a relevance threshold. Harmonic recency weighting in APIE. Random Forest. A prerequisite graph in topological order for the plan. Structural JSON comparison in the judge.
5:30 – 6:30	AI models	LLM behind a provider interface; mock provider for offline use. Random Forest trained on synthetic data, accuracy 0.735, labelled a prototype. Two QLoRA pipelines: Qwen3-4B for ATS (not trained), Qwen3-1.7B for career plans (v1 not usable).
6:30 – 7:15	Training	QLoRA: frozen 4-bit base model plus rank-16 adapters, about 1% of parameters. ATS data: three public datasets inspected, PII scrubbed, near-duplicates clustered, split by cluster with a leakage audit, weak labels from the existing engine. Career data: synthetic profiles through the planner, two roles held out.
7:15 – 8:00	Evaluation	What is measured: deterministic ATS baselines, the classifier, the skill recommender, career model v1, and the judge with 112 reference programs. What is not: any fine-tuned ATS result, human agreement, performance benchmarks. State this plainly.
8:00 – 8:30	Innovation	One explainable readiness model; "facts from code, judgement from the model, arithmetic from code"; resource IDs instead of URLs; hidden tests that never enter the sandbox.
8:30 – 9:00	Security	bcrypt, JWT with refresh rotation and family revocation, RBAC, Zod and sanitisation, rate limits, upload magic-byte checks, audit log. Student code only in an isolated container with no network, read-only filesystem, non-root user and resource limits.
9:00 – 9:30	Performance	Indexes, pagination, lazy-loaded routes, client caching, concurrency cap on sandboxes. No benchmark yet; method defined.
9:30 – 10:00	Demo pointer and future work	Train and evaluate the models; validate weights with real outcomes; human ratings; verify the sandbox with Docker; queue and admin problem editor.

21.1 Lines worth memorising

- "Deterministic algorithms compute every score; language models are used only for language."
- "The model reasons; the application knows."
- "A language model here cannot set a score, cannot invent a link, and cannot decide whether code is correct."
- "The accuracy figure is on synthetic data. It shows the pipeline works, not that it predicts real placements."
- "When the sandbox is unavailable the system says 'not executed'. It never guesses a verdict."

22. Viva questions and answers

104 questions in 24 groups. Each has a **short** answer (one breath), a **technical** answer, a **deep** answer for a follow-up, and the **common mistake** to avoid.

22.1 A. Basic project questions

Q1. What is CareerPilot AI?

- **Short:** A placement-preparation platform that combines resume analysis, coding practice, mock interviews and a career plan into one readiness score.
- **Technical:** A React frontend, a Node.js/Express gateway with MongoDB, and a Python FastAPI AI service with 11 agents.
- **Deep:** Every scored activity triggers APIE, a weighted, recency-aware engine, to recompute a Placement Readiness Score with components, reasons and next actions.
- **Common mistake:** Calling it "an AI that predicts placement". It measures preparation; it does not predict outcomes.

Q2. What problem does it solve?

- **Short:** Preparation tools are disconnected, so students cannot see overall readiness or priorities.
- **Technical:** It integrates the evidence from each stage into one explainable score and one plan.
- **Deep:** The placement cell gets the same data as cohort analytics and an at-risk list.
- **Common mistake:** Listing features instead of stating the problem.

Q3. Who are the users?

- **Short:** Students, alumni, placement officers and administrators.
- **Technical:** Four active roles enforced by `authorize(role)` middleware; a recruiter role is reserved but not built.
- **Deep:** Each role has its own dashboard and route set; ownership checks stop a student reading another student's data.
- **Common mistake:** Mentioning faculty or recruiters as working roles.

Q4. What is your one-line technical summary?

- **Short:** Deterministic algorithms compute every score; language models are used only for language.
- **Technical:** Model outputs are schema-validated and final scores are recomputed from fixed weights.
- **Deep:** This makes scores repeatable, explainable and available even when the LLM is down.
- **Common mistake:** Saying "everything is AI-powered".

Q5. Which parts actually use a language model today?

- **Short:** Interview questions, answer evaluation, and the advisory code review.
- **Technical:** Through a provider interface; the local setting is the mock provider.
- **Deep:** The ATS score, readiness score, job match, career plan and judge verdicts use no LLM at all.
- **Common mistake:** Claiming the ATS score or career plan comes from a fine-tuned LLM.

22.2 B. Architecture

Q6. Why three services?

- **Short:** Node is good at web I/O; Python has the ML libraries.
- **Technical:** The gateway handles auth, sockets and the database; FastAPI hosts NLP, scoring and ML.

- **Deep:** A shared TypeScript package keeps frontend and backend types identical; the gateway is the single AI caller so auth and audit live in one place.
- **Common mistake:** Calling it microservices. It is three services with one database.

Q7. How do the services communicate?

- **Short:** HTTP and WebSocket from the browser; HTTP with an internal key between gateway and AI service.
- **Technical:** `aiClient` adds the key, retries connection errors and maps failures to typed errors.
- **Deep:** The AI service is not exposed publicly, so the key and model access never reach the browser.
- **Common mistake:** Saying the frontend calls the AI service.

Q8. Is it monolithic or microservices?

- **Short:** Service-oriented with three services.
- **Technical:** No service discovery, message bus or per-service database.
- **Deep:** The split follows technology and trust boundaries, not team or scaling boundaries.
- **Common mistake:** Over-claiming microservices.

Q9. What happens if the AI service is down?

- **Short:** Scores still work in principle because they are deterministic; AI-dependent calls return a clear error.
- **Technical:** The gateway returns `AI_SERVICE_UNAVAILABLE`; the resume is saved and can be re-analysed.
- **Deep:** The test suite points the AI client at a closed port to prove graceful degradation.
- **Common mistake:** Saying "nothing breaks". Analysis that needs the Python engine cannot run without it.

Q10. Where is the single source of truth for types?

- **Short:** The `shared` package.
- **Technical:** Enums, DTOs and constants imported by both TypeScript apps.
- **Deep:** A change to a DTO breaks the build on both sides, so drift is caught at compile time.
- **Common mistake:** Not knowing the package exists.

22.3 C. Frontend

Q11. Why React with TypeScript?

- **Short:** Component reuse and type safety.
- **Technical:** Vite for builds, Tailwind and Radix primitives for UI.
- **Deep:** Types come from the shared package, so API shapes are checked at compile time.
- **Common mistake:** Saying "because it is popular".

Q12. How is state managed?

- **Short:** TanStack Query for server data, Zustand for auth.
- **Technical:** Queries are cached by key and invalidated after mutations.
- **Deep:** Separating server state from client state avoids a large global store.
- **Common mistake:** Saying Redux.

Q13. How is the bundle kept small?

- **Short:** Route-level code splitting.
- **Technical:** 35 pages are loaded with `React.lazy`; Monaco loads only on the problem page.
- **Deep:** No size benchmark was run; the benefit is expected, not measured.

- **Common mistake:** Quoting a load-time number.

Q14. Why Monaco?

- **Short:** It is the editor behind VS Code.
- **Technical:** Syntax highlighting for all four languages; read-only while a run is pending.
- **Deep:** Drafts are saved per problem and language in local storage.
- **Common mistake:** Claiming IntelliSense for user code.

Q15. How does voice input work?

- **Short:** The browser's Web Speech API, on the device.
- **Technical:** Speech becomes an editable transcript; `speechSynthesis` reads questions aloud.
- **Deep:** A server Whisper fallback exists behind a provider setting; the input mode is not sent to the evaluator.
- **Common mistake:** Saying audio is analysed for tone or emotion. It is not.

22.4 D. Backend

Q16. Why Express?

- **Short:** A mature, simple middleware model.
- **Technical:** Each route is a chain: authenticate, authorize, validate, controller.
- **Deep:** Controllers stay thin; services hold the rules; this keeps logic testable.
- **Common mistake:** Putting business logic in controllers when describing it.

Q17. How are requests validated?

- **Short:** Zod schemas.
- **Technical:** `validate({ body, query, params })` rejects bad input with field-level details.
- **Deep:** A sanitiser also strips MongoDB operator keys to prevent NoSQL injection.
- **Common mistake:** Saying validation is only on the frontend.

Q18. How are errors handled?

- **Short:** One central error handler.
- **Technical:** `ApiError` carries a status and a machine-readable code.
- **Deep:** Stack traces are logged with pino, not returned to the client.
- **Common mistake:** Saying errors are just `console.log`.

Q19. How do you log?

- **Short:** Structured JSON logs with pino.
- **Technical:** Each request has an id; AI calls are recorded in `AIInteraction`.
- **Deep:** Sensitive actions are written to an audit log that administrators can read.
- **Common mistake:** Claiming a monitoring stack. There is none.

Q20. How do real-time updates work?

- **Short:** Socket.IO.
- **Technical:** The server emits notifications and `readiness:updated`.
- **Deep:** The same channel carries WebRTC signalling for the alumni interview room.
- **Common mistake:** Saying polling.

22.5 E. Database

Q21. Why MongoDB?

- **Short:** The data is nested and changed shape during development.
- **Technical:** Analyses, snapshots and plans are documents; 33 Mongoose models.
- **Deep:** The cost is weaker referential integrity, handled in the service layer.
- **Common mistake:** Saying "NoSQL is faster".

Q22. How are relationships stored without foreign keys?

- **Short:** `ObjectId` references.
- **Technical:** Mongoose `populate` resolves them.
- **Deep:** Many-to-many relations use a join document, for example `CodingSubmission` between student and problem.
- **Common mistake:** Saying MongoDB has foreign keys.

Q23. Why are readiness records append-only?

- **Short:** To keep history.
- **Technical:** Each computation is a new snapshot with the weights used.
- **Deep:** Old scores stay interpretable after an administrator changes the weights.
- **Common mistake:** Saying the score is overwritten.

Q24. How are hidden test cases protected in the database?

- **Short:** They are excluded from queries by default.
- **Technical:** `select: false` on `hiddenTestCases` and `referenceSolutions`.
- **Deep:** Only the Submit path asks for them by name, and the DTO builder never reads them.
- **Common mistake:** Saying they are "filtered out in the frontend".

Q25. Which indexes matter?

- **Short:** Lookups by student, slug, status and date.
- **Technical:** About 186 index declarations, including unique `email` and `slug`.
- **Deep:** Their effect has not been benchmarked.
- **Common mistake:** Quoting a query time.

22.6 F. APIs

Q26. How many endpoints are there?

- **Short:** 135 under `/api/v1`.
- **Technical:** In 15 route files mounted on 19 paths.
- **Deep:** The AI service has a separate internal API.
- **Common mistake:** Guessing a number.

Q27. What does a response look like?

- **Short:** A common envelope.
- **Technical:** `{ success, data, meta }` or `{ success: false, error: { code, message, details } }`.
- **Deep:** Error codes are shared constants, so the frontend can branch on them.
- **Common mistake:** Saying raw objects are returned.

Q28. Which endpoint analyses a resume?

- **Short:** `POST /api/v1/resumes/:id/analyze`.
- **Technical:** Optional `jobDescriptionId` or `jobDescriptionText`, never both.
- **Deep:** A custom JD is used for that analysis only and is not stored.
- **Common mistake:** Inventing `/api/resume/analyze`.

Q29. Run versus Submit?

- **Short:** Run uses public tests and stores nothing; Submit adds hidden tests and stores the result.
- **Technical:** `POST /coding/problems/:id/run` and `/submit`.
- **Deep:** Only Submit updates readiness and requests the advisory review.
- **Common mistake:** Saying Run also uses hidden tests.

Q30. Is the API versioned?

- **Short:** Yes, `/api/v1`.
- **Technical:** The prefix is configuration.
- **Deep:** A breaking change would be released as `/api/v2` beside it.
- **Common mistake:** Saying there is no versioning.

22.7 G. AI and ML

Q31. What ML model is trained and in use?

- **Short:** A Random Forest readiness classifier.
- **Technical:** 300 trees, 14 features, four classes.
- **Deep:** Trained on 2,000 synthetic rows; accuracy 0.735, macro F1 0.737, three-fold CV 0.739 ± 0.020.
- **Common mistake:** Presenting 73.5% as accuracy on real students.

Q32. Why synthetic data?

- **Short:** There are no real placement outcomes yet.
- **Technical:** Students are drawn from four ability archetypes with Gaussian noise and 6% label flips.
- **Deep:** Noise is added on purpose; without it the labels would be a clean function of the features and accuracy would be about 99% and meaningless.
- **Common mistake:** Hiding that it is synthetic.

Q33. Why Random Forest?

- **Short:** It suits small tabular data.
- **Technical:** Little tuning, handles mixed scales, gives feature importances.
- **Deep:** Gradient Boosting is available as an alternative; a neural network would overfit this little data.
- **Common mistake:** Saying "it has the best accuracy" without a comparison.

Q34. What are the top features?

- **Short:** Academic, coding and technical scores.
- **Technical:** Importances 0.191, 0.157, 0.151.
- **Deep:** These are global importances, not a per-student explanation such as SHAP.
- **Common mistake:** Calling them causal.

Q35. What is the difference between APIE and the classifier?

- **Short:** APIE is a rule-based score; the classifier is a learned second opinion.
- **Technical:** Both read the same components.
- **Deep:** They are shown side by side; APIE is authoritative because it is explainable.
- **Common mistake:** Saying the classifier produces the readiness score.

22.8 H. LLM

Q36. Which LLM does the project use?

- **Short:** Any provider behind one interface; the local setting is the mock.
- **Technical:** `AI_PROVIDER` selects OpenAI-compatible, Azure, Ollama or mock.
- **Deep:** The mock is rule-based and tagged `provider: "mock"`; it is not a language model.
- **Common mistake:** Naming a specific model as "the one we use" without checking the setting.

Q37. How do you control LLM output?

- **Short:** JSON schemas, low temperature and recomputed scores.
- **Technical:** Pydantic validation on every agent output.
- **Deep:** The model returns dimension scores; the overall comes from fixed weights in code.
- **Common mistake:** Saying "we trust the model".

Q38. What is a transformer?

- **Short:** A neural network built on attention.
- **Technical:** Each token attends to others to build a context-aware representation.
- **Deep:** Stacked attention and feed-forward layers; LoRA adapters are attached to their projection matrices.
- **Common mistake:** Confusing it with an RNN.

Q39. What is tokenization?

- **Short:** Splitting text into pieces the model reads.
- **Technical:** Sequence limits are in tokens: 3,072 for ATS, 4,096 for the career model.
- **Deep:** Over-long career examples were dropped rather than truncated; ATS text is truncated before labelling.
- **Common mistake:** Saying tokens are words.

Q40. What is hallucination and how do you handle it?

- **Short:** A fluent statement with no basis.
- **Technical:** Schemas, a grounding guard, resource IDs and a RAG threshold.
- **Deep:** Facts come from deterministic extraction; a "matched" skill not in the resume is moved to missing and counted.
- **Common mistake:** Claiming hallucination is eliminated.

22.9 I. Fine-tuning

Q41. Which models did you fine-tune?

- **Short:** Two pipelines: Qwen3-4B for ATS and Qwen3-1.7B for career plans.
- **Technical:** Both QLoRA, rank 16, alpha 32.
- **Deep:** The ATS model has not been trained; the career model's first run was measured as not usable. Neither is serving.
- **Common mistake:** Saying "we fine-tuned and deployed Qwen".

Q42. What is supervised fine-tuning?

- **Short:** Training on input and target pairs.
- **Technical:** Chat-format examples with completion-only loss.
- **Deep:** Loss is computed on the answer tokens only, so the model learns the JSON and not to reproduce the resume.
- **Common mistake:** Confusing it with pre-training.

Q43. Where do the ATS labels come from?

- **Short:** Weak labels computed by formulas on the existing engine.

- **Technical:** No label is produced by an LLM or a human.
- **Deep:** Three holistic components are blended with a dataset fit label at λ = 0.35; the rest are pure evidence.
- **Common mistake:** Calling them ground truth.

Q44. How did you prevent data leakage?

- **Short:** Split by near-duplicate cluster before sampling.
- **Technical:** MinHash with LSH, verified by Jaccard ≥ 0.8; job descriptions are held out too.
- **Deep:** An audit over all split pairs reports 0/0/0 and the build refuses to write otherwise.
- **Common mistake:** Saying "we used a random split".

Q45. Why was a dataset's provided ATS score discarded?

- **Short:** It carried no information beyond the class label.
- **Technical:** Within a class its correlation with text features was about −0.001.
- **Deep:** The three classes sat in disjoint score bands with the same spread: class band plus noise.
- **Common mistake:** Using a provided label without inspecting it.

22.10 J. LoRA

Q46. What is LoRA?

- **Short:** Training small adapter matrices instead of the whole model.
- **Technical:** `W' = W + (α/r)·B·A` with W frozen.
- **Deep:** A and B are low-rank, so the number of trained parameters is small.
- **Common mistake:** Saying LoRA compresses the model.

Q47. What do rank and alpha mean?

- **Short:** Rank is adapter capacity; alpha scales its effect.
- **Technical:** r = 16, α = 32, so the scale is 2.
- **Deep:** Higher rank can fit more but risks overfitting on small data.
- **Common mistake:** Confusing rank with the number of layers.

Q48. Which layers have adapters?

- **Short:** All seven projection layers.
- **Technical:** The attention and feed-forward projections.
- **Deep:** For the ATS model that is about 33 million trainable parameters, about 0.8%.
- **Common mistake:** Quoting a parameter count for the career model; it was not recorded.

Q49. What is the output of LoRA training?

- **Short:** A small adapter file.
- **Technical:** `adapter_model.safetensors` and its config.
- **Deep:** The base model is unchanged, so the untuned model can be evaluated by disabling the adapter.
- **Common mistake:** Saying a new full model is produced.

22.11 K. QLoRA

Q50. What is QLoRA?

- **Short:** LoRA on a base model loaded in 4-bit.
- **Technical:** 4-bit NF4 quantization with bitsandbytes or Unsloth.
- **Deep:** The frozen weights are quantized; adapters train in higher precision.

- **Common mistake:** Saying the adapters are 4-bit.

Q51. Why QLoRA and not full fine-tuning?

- **Short:** Only a free 16 GB T4 was available.
- **Technical:** Full fine-tuning of a 4B model does not fit.
- **Deep:** QLoRA also gives a swappable, versioned adapter and a clean base-versus-tuned comparison.
- **Common mistake:** Saying QLoRA is more accurate.

Q52. What does quantization cost?

- **Short:** A small loss of precision.
- **Technical:** Each training step is somewhat slower.
- **Deep:** No quantized-versus-full comparison was run in this project.
- **Common mistake:** Claiming zero quality loss.

Q53. What is gradient accumulation and why use it?

- **Short:** Simulating a larger batch on a small GPU.
- **Technical:** 1 × 16 for ATS behaves like batch 16.
- **Deep:** For the career model, updating every 16 examples gave only 37 updates in v1, which is why the notebook now updates every 4.
- **Common mistake:** Saying it speeds training up.

22.12 L. NLP

Q54. Which NLP techniques are used?

- **Short:** Regex, dictionary matching and spaCy NER.
- **Technical:** Section detection by heading patterns; skills from a taxonomy.
- **Deep:** The only statistical NLP model in the resume path is spaCy's NER, used for the name.
- **Common mistake:** Claiming TF-IDF or BERT.

Q55. Why not `\b` word boundaries?

- **Short:** They break on `C++` and `C#`.
- **Technical:** A custom look-around excludes letters, digits, `+` and `#`.
- **Deep:** It also stops "Java" matching inside "JavaScript"; this is tested.
- **Common mistake:** Saying substring search is used.

Q56. How are skill variations handled?

- **Short:** Aliases map to a canonical name.
- **Technical:** 181 aliases for 107 skills, with flexible separators.
- **Deep:** "ReactJS", "React.js" and "react js" all become "React".
- **Common mistake:** Saying fuzzy matching.

Q57. What if the resume is a scanned image?

- **Short:** It is rejected with a clear message.
- **Technical:** Fewer than 120 extracted characters raises a parse error.
- **Deep:** OCR is not implemented; it is future work.
- **Common mistake:** Claiming OCR support.

22.13 M. ATS

Q58. How is the ATS score calculated?

- **Short:** Nine rule-based criteria summing to 100.
- **Technical:** Skills 20, structure 15, experience 15, contact 10, education 10, impact 10, formatting 10, verbs 5, length 5.
- **Deep:** Each uses capped formulas, giving diminishing returns.
- **Common mistake:** Saying an LLM scores the resume.

Q59. How do you know the ATS score is accurate?

- **Short:** It is a transparent rubric, not a validated predictor.
- **Technical:** Against weak labels the engine has MAE 5.18 and Spearman 0.873 in general mode.
- **Deep:** No human agreement exists yet; the review sheet of 476 examples is empty.
- **Common mistake:** Saying "it matches real ATS systems".

Q60. What was learned from the baseline evaluation?

- **Short:** Keyword coverage barely tracks job fit on unseen jobs.
- **Technical:** Spearman 0.079 against fit labels; a richer heuristic reaches 0.227.
- **Deep:** That gap is the headroom a fine-tuned model would have to beat.
- **Common mistake:** Claiming the fine-tuned model beat it. It was not measured.

Q61. If the model were enabled, what would it decide?

- **Short:** Component scores and wording only.
- **Technical:** The final score is aggregated in Python.
- **Deep:** Matched and missing keywords stay deterministic even then.
- **Common mistake:** Saying the model outputs the ATS score.

22.14 N. Career plan

Q62. How is the career plan generated?

- **Short:** By a deterministic planner.
- **Technical:** Normalise skills, find gaps, score priority, order by prerequisites, fit to time.
- **Deep:** A fine-tuned model can optionally reword and organise it; that path is off.
- **Common mistake:** Saying a fine-tuned LLM writes the plan.

Q63. How is priority decided?

- **Short:** An additive score with stated weights.
- **Technical:** Importance × 10, +8 core, +6 per skill unblocked, +12 if ATS flagged it, +4 per level.
- **Deep:** HIGH at 62, MEDIUM at 40; each gap returns its reasons.
- **Common mistake:** Saying the weights were learned.

Q64. How do you ensure a correct learning order?

- **Short:** A prerequisite graph.
- **Technical:** 122 skills, 123 edges, verified acyclic, topological order.
- **Deep:** A skill is never deferred while a kept skill depends on it.
- **Common mistake:** Saying order is by priority only.

Q65. What if the deadline is too short?

- **Short:** Low-priority gaps are deferred, and the plan says so.
- **Technical:** 75/15/10 time budget; defer LOW then MEDIUM secondary gaps.
- **Deep:** If it still does not fit, the plan states the deadline is not achievable at this pace.
- **Common mistake:** Saying the plan always fits.

Q66. Where does role knowledge come from?

- **Short:** A curated knowledge base, corroborated by O*NET, ESCO and job postings.
- **Technical:** 81% of 275 role-to-skill links are corroborated.
- **Deep:** Levels, hours and prerequisites are editorial and unvalidated; O*NET and ESCO do not contain them.
- **Common mistake:** Saying the roadmap "comes from O*NET".

22.15 O. Recommendation systems

Q67. How are resources recommended?

- **Short:** A deterministic ranking over a verified catalogue.
- **Technical:** Points for covering the skill, stage match, verification, quality weight, preference and language.
- **Deep:** Ties break on the resource ID so the order is reproducible.
- **Common mistake:** Calling it collaborative filtering.

Q68. Is there a learned recommender?

- **Short:** Yes, for skills, but it is not live.
- **Technical:** Item-based collaborative filtering with shrinkage.
- **Deep:** Recall@5 0.411 against 0.272 for popularity, with a 95% interval of +0.101 to +0.176.
- **Common mistake:** Presenting it as a running feature.

Q69. Why did the simpler recommender win?

- **Short:** The dataset is small.
- **Technical:** 844 baskets; PPMI + SVD embeddings scored lower.
- **Deep:** A popularity penalty was also tested and rejected because it lowered validation scores.
- **Common mistake:** Assuming the more complex model is better.

Q70. What is `quality_score`?

- **Short:** An editorial weight by resource type.
- **Technical:** Between 0 and 1.
- **Deep:** It is not a user rating; there is no rating data.
- **Common mistake:** Calling it a rating.

22.16 P. YouTube API

Q71. Why use the YouTube Data API?

- **Short:** To get current videos that actually exist.
- **Technical:** Search per skill, keep public and embeddable videos.
- **Deep:** 180 videos are in the catalogue; a live search endpoint returns results labelled unverified.
- **Common mistake:** Saying the model generates YouTube links.

Q72. What if YouTube returns an irrelevant video?

- **Short:** It ranks lower or is not used.
- **Technical:** 151 of 180 name the skill in the title; the other 29 rank lower.
- **Deep:** Videos are availability-checked, not quality-reviewed; that is a stated limitation.
- **Common mistake:** Claiming relevance is guaranteed.

Q73. Where is the API key?

- **Short:** Only in the local `.env` file.
- **Technical:** Never in code, the example file or the Colab package.
- **Deep:** The packaging script scans the archive for key-shaped strings and fails if it finds one.
- **Common mistake:** Showing the key on screen.

22.17 Q. DSA judge

Q74. How is submitted code executed?

- **Short:** In a throw-away Docker container.
- **Technical:** Compile once, then run each test with inputs streamed by the host.
- **Deep:** The harness writes the return value to a result file; the host compares it with the expected value.
- **Common mistake:** Saying code runs on the backend.

Q75. How is the verdict decided?

- **Short:** By a fixed decision list.
- **Technical:** Time limit, memory, runtime error, wrong answer, accepted.
- **Deep:** The submission's verdict is its first failing test; no AI is involved.
- **Common mistake:** Saying AI checks the code.

Q76. How do you compare outputs?

- **Short:** Structurally, as JSON.
- **Technical:** Float tolerance 1e-6; optional order-insensitive modes.
- **Deep:** Formatting cannot change a verdict because values, not text, are compared.
- **Common mistake:** Saying string comparison.

Q77. How are four languages supported from one problem?

- **Short:** Code generation from a typed signature.
- **Technical:** Starter code and harness are generated per language.
- **Deep:** Arguments use a token format that every language can parse without a JSON library.
- **Common mistake:** Saying each problem is written four times.

Q78. How was the judge tested?

- **Short:** 28 reference solutions in four languages, all accepted.
- **Technical:** 112 programs on every public and hidden test.
- **Deep:** Through a test-only local transport with local compilers; container isolation was not exercised.
- **Common mistake:** Saying the Docker sandbox was tested.

22.18 R. Docker

Q79. Why Docker?

- **Short:** Isolation with resource limits.
- **Technical:** Namespaces and cgroups separate the process and cap memory, CPU and processes.
- **Deep:** It is not a VM boundary; a dedicated host and stricter profiles are recommended for hostile traffic.
- **Common mistake:** Saying Docker is fully secure.

Q80. How do you stop an infinite loop?

- **Short:** A time limit.
- **Technical:** `timeout` inside the container and a host-side deadline.

- **Deep:** After a time-limit failure the remaining tests are not run, bounding total time.
- **Common mistake:** Saying the server detects loops.

Q81. How do you stop a fork bomb?

- **Short:** A process limit.
- **Technical:** `--pids-limit`.
- **Deep:** The verification script includes a fork-bomb probe.
- **Common mistake:** Saying memory limits handle it.

Q82. How are memory limits enforced?

- **Short:** A container ceiling plus per-language limits.
- **Technical:** `-Xmx` for Java, `--max-old-space-size` for Node, an address-space limit for C++.
- **Deep:** C++ and Java containers get a larger ceiling because their compilers need it.
- **Common mistake:** Saying one flag does everything.

Q83. Why is the judge disabled in Docker Compose?

- **Short:** A containerised backend would need the host's Docker socket.
- **Technical:** Mounting that socket is root on the host.
- **Deep:** So the backend runs on the host when the judge is needed.
- **Common mistake:** Suggesting the socket be mounted.

22.19 S. Security

Q84. How are passwords stored?

- **Short:** As bcrypt hashes.
- **Technical:** 12 rounds with a per-password salt.
- **Deep:** The cost factor makes brute force slow.
- **Common mistake:** Saying encrypted. Hashing is one-way.

Q85. How does authentication work?

- **Short:** A short-lived access token and a rotating refresh token.
- **Technical:** 15 minutes and 7 days; refresh in a signed httpOnly cookie.
- **Deep:** Reuse of an old refresh token revokes the whole family.
- **Common mistake:** Saying tokens are stored in the database as sessions.

Q86. How do you prevent injection?

- **Short:** Validation and sanitisation.
- **Technical:** Zod types; operator keys stripped.
- **Deep:** Mongoose casts values to schema types as a further layer.
- **Common mistake:** Talking about SQL injection for MongoDB.

Q87. How are uploads secured?

- **Short:** Type, content and size checks.
- **Technical:** Extension, MIME type, magic bytes, 5 MB.
- **Deep:** Files are stored under generated keys and served only as attachments.
- **Common mistake:** Saying only the extension is checked.

Q88. How are resumes protected?

- **Short:** Owner-only access with audit.
- **Technical:** Ownership checks; staff access is logged.

- **Deep:** Training data was PII-scrubbed; privacy endpoints support export and deletion.
- **Common mistake:** Claiming encryption at rest. Not in the repository.

22.20 T. Performance

Q89. What performance optimisations exist?

- **Short:** Indexes, pagination, lazy loading, caching, a concurrency cap.
- **Technical:** See the performance table.
- **Deep:** None has been benchmarked.
- **Common mistake:** Quoting a speed-up.

Q90. How would you benchmark?

- **Short:** Load tests and query plans.
- **Technical:** p50/p95/p99 latency at fixed users; `explain("executionStats")`.
- **Deep:** For the judge: compile time, per-test time and queue wait under concurrency.
- **Common mistake:** Saying "it feels fast".

Q91. What is the bottleneck?

- **Short:** Synchronous analysis and the judge.
- **Technical:** Both run inside the request.
- **Deep:** A queue with workers is the planned fix.
- **Common mistake:** Saying there is none.

22.21 U. Algorithms

Q92. What is cosine similarity?

- **Short:** The angle between two vectors.
- **Technical:** Dot product over the product of norms.
- **Deep:** With normalised vectors it equals the dot product.
- **Common mistake:** Confusing it with Euclidean distance.

Q93. Do you use TF-IDF?

- **Short:** No.
- **Technical:** Feature hashing with sub-linear term frequency.
- **Deep:** There is no inverse document frequency term; it is a lexical embedding.
- **Common mistake:** Saying yes.

Q94. What is RAG?

- **Short:** Retrieve relevant text, then generate from it.
- **Technical:** Chunk, embed, top-5 search, relevance filter, answer with sources.
- **Deep:** Below the threshold the LLM is not called at all.
- **Common mistake:** Saying RAG removes hallucination.

Q95. How was the RAG threshold chosen?

- **Short:** By measurement.
- **Technical:** On-topic 0.28–0.31, adjacent 0.11–0.14, unrelated 0.02–0.07; 0.18 sits in the gap.
- **Deep:** It depends on the embedder, so semantic embedders use 0.25.
- **Common mistake:** Saying it was guessed.

Q96. Why harmonic recency weighting?

- **Short:** Recent results matter more, without ignoring history.
- **Technical:** Weights 1, ½, ⅓.
- **Deep:** A mean ignores improvement; latest-only is too noisy.
- **Common mistake:** Saying exponential decay.

22.22 V. Innovation

Q97. What is novel?

- **Short:** The integration, not a new algorithm.
- **Technical:** One explainable readiness model fed by every module.
- **Deep:** Plus a strict split between deterministic scoring and generative language.
- **Common mistake:** Calling CRUD or a dashboard an innovation.

Q98. What is your measurable improvement?

- **Short:** None measured against another product.
- **Technical:** Internal measurements exist: baselines, classifier, recommender, judge correctness.
- **Deep:** A user study or outcome comparison is future work.
- **Common mistake:** Inventing a percentage.

22.23 W. Limitations

Q99. What is the biggest limitation?

- **Short:** Nothing is validated against real placement outcomes.
- **Technical:** Weights are design choices; ML data is synthetic.
- **Deep:** Both fine-tuned models are off; no human evaluation exists.
- **Common mistake:** Saying there are no limitations.

Q100. Why is the career model not usable?

- **Short:** Too little training.
- **Technical:** 591 examples, 37 updates; 51.7% valid JSON, 6.7% passed the schema, 0% fully valid.
- **Deep:** 21 of 60 outputs never stopped; loss was still falling when the run ended.
- **Common mistake:** Hiding the result.

Q101. Is the sandbox verified?

- **Short:** The logic yes, the container isolation not yet.
- **Technical:** Docker was not installed on the development machine.
- **Deep:** `npm run judge:verify` runs hostile programs to check each control.
- **Common mistake:** Saying "fully tested".

22.24 X. Future scope

Q102. What is next?

- **Short:** Train and evaluate the models, and validate the weights.
- **Technical:** GPU runs, human review sheets, real outcome data.
- **Deep:** Then a judge queue, an admin problem editor and semantic embeddings.
- **Common mistake:** Listing unrelated features.

Q103. How would you validate APIE?

- **Short:** Against real outcomes.

- **Technical:** Collect placement results, fit weights, compare with the current ones.
- **Deep:** Report calibration and discrimination, and re-validate each year.
- **Common mistake:** Saying "ask students if they like it".

Q104. How would it scale?

- **Short:** Stateless services behind a load balancer.
- **Technical:** A queue and judge workers; a shared cache; a managed database.
- **Deep:** Socket.IO would need a shared adapter; none of this is built.
- **Common mistake:** Saying it already scales.

23. Questions the jury may ask to challenge the project

Each answer is written to be said aloud. They are honest first and strong second; a jury trusts the second only after hearing the first.

1. Why did you use an LLM instead of traditional ML? We used both, each where it fits. Scores are rule-based or classical ML because they must be repeatable and explainable. An LLM is used only where the task is language: writing an interview question, judging a free-text answer, phrasing advice. Traditional ML cannot generate a follow-up question; an LLM should not be trusted to add up a score.

2. Why QLoRA instead of full fine-tuning? Hardware and risk. The only GPU was a free 16 GB Colab T4, and full fine-tuning of a 4-billion-parameter model does not fit. QLoRA freezes the base model in 4-bit and trains about 1% of the parameters. It also gives a small adapter we can version, and lets us evaluate the untuned base model simply by disabling the adapter.

3. Why this base model? Three requirements: an open licence, a size that fits a T4 at our sequence length, and reliable instruction-following for JSON. Qwen3 met them and is Apache-2.0. We did not benchmark it against Llama, Mistral or Gemma, so we do not claim it is better — only that it met the requirements.

4. How do you prevent hallucination? By not asking the model for facts. Skills come from dictionary extraction, scores from formulas, links from a catalogue. Where a model is used, its output must pass a strict schema; a skill it claims is matched but is not in the resume is moved to missing; a resource ID not in the catalogue is dropped; and RAG refuses to answer below a relevance threshold without calling the model. We reduce and detect hallucination; we do not claim to eliminate it.

5. How do you calculate the ATS score? Nine criteria with fixed maximum points summing to 100: skills 20, section structure 15, experience 15, contact 10, education 10, quantified impact 10, formatting 10, action verbs 5, length 5. Each is a capped formula, so piling up keywords gives diminishing returns. The breakdown is returned with the score.

6. How do you know the score is accurate? We do not claim it predicts a real employer's ATS. It is a transparent rubric. What we measured: against our weak labels the engine has a mean absolute error of 5.18 and rank correlation of 0.873 in general mode. What we have not measured: agreement with human reviewers. A 476-example review sheet exists and is empty.

7. How do you evaluate a generated career roadmap? Not with accuracy, because it is generation, not classification. The evaluator measures 13 things: JSON validity, schema compliance, skill-gap F1 against the deterministic analysis, priority agreement, prerequisite correctness, ordering, role alignment, personalisation, schedule feasibility, resource-ID validity, resource hallucination, contradictions and duplicates. Whether the advice is *good* needs expert rating; that sheet is prepared and not yet filled.

8. Why not use RAG for everything? RAG fits questions answered by documents. A readiness score needs arithmetic, a study plan needs constraint handling, and a code verdict needs execution. Retrieval does none of those. We use RAG where it belongs — the knowledge base — and deterministic code elsewhere.

9. Why do you need the YouTube API? Because a model's knowledge is frozen at training time and videos change. The API returns videos that exist now and tells us whether they are public and embeddable. The model's job is to reason about which skill to study; the API's job is to find current material.

10. What happens if YouTube returns irrelevant videos? Catalogue videos whose title does not name the skill rank lower: 29 of 180 are in that group. Live search results are labelled "unverified" and are never added to the catalogue. Honestly, videos are availability-checked, not quality-reviewed; a human review step is future work.

11. What happens if the model recommends a broken course? It cannot recommend a course directly. It may only choose resource IDs that were supplied in the prompt. Each ID resolves to a catalogue entry that was fetched and verified; an unknown ID is dropped and counted as a hallucination. A link can still go stale after verification, which is why each entry stores when it was last verified.

12. How do you verify resources? Each non-video URL was requested over HTTPS: 88 of 90 verified, 2 excluded. Videos were checked through the YouTube Data API. The verifier itself is hardened against server-side request forgery: HTTPS only, the provider's own domain only, no private or loopback addresses, at most three redirects, and a cap on bytes read.

13. How do you prevent prompt injection? We do not have a dedicated injection filter, and I will not claim one. The defence is structural: the model has no tools, cannot set a final score because scores are recomputed from fixed weights, and anything outside the schema is rejected. So an instruction hidden in a resume can at most distort wording, not a score. A classifier for injected instructions is future work.

14. How do you protect user resumes? Uploads are checked by extension, MIME type and magic bytes; stored under generated keys; and returned only to the owner or, with an audit entry, to staff. Downloads are sent as attachments with `nosniff`. Training data was scrubbed of emails, phones and profile links. Encryption at rest is not implemented in the repository and would be the storage provider's responsibility.

15. How is user code executed securely? In its own container with no network, a read-only filesystem, a non-root user, all capabilities dropped, and limits on memory, CPU, processes and time. No environment variables are passed in, so no secret is present. Expected outputs never enter the container. I should add: this was verified with unit tests and local compilers, but not yet run under Docker on our machine.

16. Why Docker? It is the standard isolation primitive, with namespaces to separate the process and cgroups to cap resources, and containers start quickly. It is not as strong as a virtual machine. For hostile public traffic we would add a dedicated host, a custom seccomp profile or gVisor.

17. How do you prevent infinite loops? A time limit enforced twice: `timeout` inside the container, and a deadline on the host that removes the container if the inner timer fails. After a time-limit failure the remaining tests are skipped, so one submission cannot occupy a sandbox for the whole test set.

18. How do you handle memory limits? The container has a hard memory ceiling with swap disabled. Because compilers need more memory than the problem allows, the per-problem limit is also enforced on the program: `-Xmx` for Java, `--max-old-space-size` for Node, an address-space limit for C++. Exceeding it gives Memory Limit Exceeded.

19. What happens if the model goes down? Scores are unaffected in principle because none is produced by a model. Features that need the AI service return a clear error rather than a fabricated result. For the fine-tuned paths, a timeout or crash falls back to the deterministic engine and marks the response `degraded` with the reason.

20. What is your fallback architecture? Every model has a deterministic twin: the ATS engine behind the ATS model, the planner behind the career model, the mock provider behind the LLM. The order is: valid model answer; else one corrective retry; else the deterministic result, labelled as such. A degraded answer is never presented as a model answer.

21. What is actually novel? No new algorithm. The contribution is integration with discipline: one explainable readiness model fed by every module, and a hard line between deterministic scoring and generative language. Smaller engineering contributions support it — resource IDs instead of URLs, hidden tests that never enter the sandbox, leakage-audited datasets.

22. What is your measurable improvement? I have no measured improvement over another product or over a baseline cohort, and I will not invent one. What is measured is internal: the baselines for the ATS engine, the classifier and recommender metrics, the career model's results, and judge correctness on 112 reference programs. A user study is the next step.

23. What is the biggest limitation? Validation. The readiness weights are a design choice, the classifier is trained on synthetic data, the fine-tuned models are not in service, and no human has rated the outputs. The system is engineered carefully, but its usefulness to real students is not yet demonstrated.

24. Why should someone use this instead of existing platforms? Not for depth in one area — a dedicated coding site has far more problems. The reason is connection: a resume gap changes the career plan, a coding result moves the readiness score, an alumni's feedback adjusts it, and the placement cell sees all of it. That linkage is what a set of separate tools cannot give.

25. What happens when the job market changes? Nothing needs retraining for facts. Role requirements, prerequisites and resources are data files, and the model receives them in the prompt, so a new role or resource works immediately. Two roles were held out of training precisely to test that. What does need periodic human review is the curated knowledge itself.

26. You say "AI platform", but most of it is rules. Is that not misleading? It would be if we hid it. We state it openly: rules compute the scores because scores must be defensible, and AI is used for language. We consider that the right design for a system that judges students, not a weakness.

27. Your accuracy is only 73.5%. Is that not poor? It is on purpose not higher. The data is synthetic with deliberate noise; without the noise the labels would be a clean function of the features and accuracy would be about 99%, which would mean nothing. The figure shows the pipeline works. It says nothing about real placements.

28. Why did you remove Group Discussion and the earlier roadmap? They were removed from the product to focus the scope. The readiness weights were rebalanced — the former 10% went to coding and communication — and stored records were migrated. The career plan replaced the earlier roadmap with a deterministic, testable planner.

29. Did you write all of this yourself? Answer truthfully about your own contribution and the tools you used. Whatever the answer, be able to explain any module from this document: that is what the jury is really checking.

30. If you had one more month, what would you do first? Collect ground truth. Have placement officers score 100 resumes and rate 30 career plans, gather last year's placement outcomes, and then train and evaluate the models against those. Everything else — more problems, a queue, better embeddings — matters less than knowing whether the scores are right.

24. Concepts explained like a professor

Each concept: what it is, how CareerPilot uses it, a simple example, and the technical point to add if pressed.

24.1 Web and system concepts

Concept	Definition	In CareerPilot	Simple example	Technical point
Frontend	The part that runs in the browser	React + TypeScript single-page app	The Resume page you see	Renders from JSON returned by the API
Backend	The server-side program	Node.js + Express gateway	Checks your login and saves a submission	Stateless request handling with middleware chains
REST API	Resources addressed by URLs and HTTP verbs	135 endpoints under `/api/v1`	`GET /coding/problems` lists problems	JSON envelope with typed error codes
Database	Organised persistent storage	MongoDB with 33 Mongoose models	Your resume analysis is one document	Documents with `ObjectId` references and indexes
Authentication	Proving who you are	Email and password, then a token	Logging in	bcrypt comparison, then a signed JWT
Authorization	Deciding what you may do	Role middleware and ownership checks	A student cannot open the admin page	`authorize(role)` plus per-record owner check
JWT	A signed token carrying claims	15-minute access token	A wristband that expires	HMAC signature; verified without a database lookup
MongoDB	A document database	The only database	A plan stored as one nested document	No joins or foreign keys; references are populated
Services (not microservices)	Separate programs with separate jobs	Three: frontend, gateway, AI service	Python does ML, Node does web	One database, no message bus
Docker	Packaging and isolating a process	The judge sandbox; Compose for deployment	A sealed box for one run	Namespaces isolate, cgroups limit
Container	A running isolated process from an image	One per code submission	The box is thrown away after the run	`--rm`, no network, read-only root
API gateway	A single entry point in front of services	The Node backend plays this role	The reception desk	It alone holds the internal AI key
Queue	A waiting line for background jobs	**Not used**; an in-process limiter instead	—	A queue with workers is planned
Caching	Keeping a result to avoid recomputing	Client query cache; in-memory config cache	Not refetching the problem list	No Redis; server caches are per process

24.2 AI and NLP concepts

Concept	Definition	In CareerPilot	Simple example	Technical point
NLP	Processing human language	Resume parsing and skill extraction	Finding "Python" in a resume	Regex, dictionary matching, spaCy NER
Embedding	A vector representing text	384-dimensional hashed vectors	Two similar texts get nearby vectors	Feature hashing, sub-linear TF, L2-normalised

Concept	Definition	In CareerPilot	Simple example	Technical point
Vector database	A store searched by vector similarity	NumPy store by default; FAISS or Chroma optional	Finding the closest chunks	Exact inner-product search
Semantic search	Search by meaning, not exact words	Available by switching the embedder; default is lexical	"car" finding "automobile"	Needs a learned embedding model
RAG	Retrieve text, then generate from it	Knowledge-base answers	Answering from the placement handbook	Top-5 retrieval, relevance threshold, sources returned
LLM	A model that predicts the next token	Interview questions, answer evaluation	Writing a follow-up question	Called through a provider interface
Fine-tuning	Further training on a task	Two QLoRA pipelines	Teaching a strict JSON format	Adapters only; base frozen
SFT	Fine-tuning on input–target pairs	Both pipelines	Resume → expected JSON	Completion-only loss
PEFT	Training few parameters	LoRA adapters	Sticky notes on a textbook	About 1% trainable
LoRA	Low-rank adapters added to weight matrices	r 16, α 32 on 7 projections	Two thin matrices instead of one big one	`W + (α/r)·B·A`
QLoRA	LoRA with a 4-bit base model	Fits a T4	Smaller print, same notes	NF4 quantization via bitsandbytes
Quantization	Fewer bits per weight	4-bit loading	Rounding to fewer decimal places	Trades a little precision for memory
Transformer	The architecture behind LLMs	Qwen3 models	Words looking at each other	Attention plus feed-forward layers
Attention	Weighting which tokens matter to each token	Inside the models	"it" attending to its noun	Query–key–value projections; LoRA attaches here
Tokenization	Splitting text into model units	Length limits of 3,072 and 4,096 tokens	"unhappiness" → pieces	Limits are in tokens, not words
Inference	Running a trained model	One call with a timeout	Asking a question	Validation and fallback follow
Hallucination	Fluent output without basis	Guarded by schemas, grounding, IDs	A made-up URL	Reduced and counted, not eliminated
Prompt injection	Instructions hidden in input data	Structural limits only	"Ignore the rubric" inside a resume	No dedicated filter; scores are code-computed

24.3 Recommendation, similarity and metrics

Concept	Definition	In CareerPilot	Simple example	Technical point
Recommendation system	Suggesting relevant items	Resource ranking; skill recommender (not live)	"Learn Docker next"	Deterministic scoring; item-based CF
Ranking	Ordering items by a score	Resources, mentors, gaps	Best resource first	Additive scores with reproducible tie-breaks
Cosine similarity	Angle between vectors	Job match, RAG	Same direction = similar	Equals the dot product for unit vectors
TF-IDF	Term frequency × inverse document frequency	**Not used**	—	The project uses hashed TF without IDF
Accuracy	Share of correct predictions	Classifier: 0.735 (synthetic)	73 of 100 right	Misleading with imbalanced classes

Concept	Definition	In CareerPilot	Simple example	Technical point
Precision	Correct among predicted positives	Per-class in the classifier	Of flagged, how many were right	TP / (TP + FP)
Recall	Found among actual positives	Per-class; Recall@K for the recommender	Of real ones, how many found	TP / (TP + FN)
F1	Harmonic mean of precision and recall	Macro F1 0.737	One balanced number	2PR / (P + R)
Overfitting	Memorising training data	Guarded by validation loss and early stopping	Good on practice, bad on the exam	Training loss falls, validation loss rises
Underfitting	Not learning enough	Career model v1	Stopped studying too early	37 updates; loss still falling
Train / validation / test split	Separate data for learning, tuning and final check	Split by cluster or learner profile	Practice, mock, final exam	Prevents leakage; test used once

25. Technical glossary

Term	Meaning
Adapter	The small set of trained LoRA weights saved after fine-tuning
Agent	A class with one responsibility and typed input and output
Alias	An alternative spelling mapped to a canonical skill name
APIE	Adaptive Placement Intelligence Engine; computes the readiness score
Append-only	Records are added, never changed, to keep history
ATS	Applicant Tracking System; software that screens resumes
Audit log	A record of sensitive actions and who performed them
Band	A readiness category: At Risk, Needs Improvement, Placement Ready, Highly Ready
Base model	The pre-trained model before fine-tuning
bcrypt	A slow, salted password-hashing function
bitsandbytes	The library that loads models in 4-bit or 8-bit
Bootstrap sample	A random sample with replacement, used by Random Forest
Canonical skill	The single official name for a skill and its aliases
Capability (Linux)	A fine-grained root privilege; all are dropped in the sandbox
cgroup	A Linux feature that limits a process's CPU, memory and process count
Checkpoint	A saved state during training
Chunking	Splitting a document into overlapping pieces for retrieval
Code splitting	Loading page code only when the page is opened
Collaborative filtering	Recommending items that co-occur with what a user has
Comparator	The judge component that compares returned and expected values
Completion-only loss	Training loss computed only on the answer tokens
Confusion matrix	A table of actual versus predicted classes
Container	An isolated process started from an image
CORS	Browser rule controlling which origins may call an API
Cosine schedule	A learning rate that decays along a cosine curve
Cross-validation	Training and testing on several different splits
DTO	Data Transfer Object; the shape of data sent over the API
Degraded	A response produced by a fallback, labelled as such
Deterministic	Same input always gives the same output
Early stopping	Ending training when validation loss stops improving
Embedding	A numeric vector representing text
Epoch	One pass over the training data
ESCO	The European skills and occupations classification
Evidence coverage	The share of readiness components that have data
Express	The Node.js web framework used by the gateway
FAISS	A library for fast vector search
FastAPI	The Python web framework used by the AI service
Feature hashing	Mapping tokens to vector positions with a hash function

Term	Meaning
Feature importance	How much a feature contributes to a tree model's splits
Fork bomb	A program that creates processes endlessly
Gradient accumulation	Summing gradients over small batches before an update
Gradient checkpointing	Saving memory by recomputing activations in the backward pass
Grounding guard	A check that a model's claims are supported by the input
Harness	Generated code that calls the student's function and records the result
Helmet	Express middleware that sets security headers
Hidden test	A test whose input and expected output are never shown
httpOnly cookie	A cookie JavaScript cannot read
Index	A database structure that speeds up lookups
Jaccard similarity	Overlap of two sets divided by their union
JD	Job description
JWT	JSON Web Token; a signed, self-contained token
Lazy loading	Deferring work until it is needed
Leakage	Test data influencing training, inflating results
Lexical	Based on the words themselves, not their meaning
LoRA	Low-Rank Adaptation
LSH	Locality-Sensitive Hashing; groups similar items into buckets
MAE	Mean Absolute Error
Magic bytes	The first bytes of a file that identify its real type
Middleware	A function that runs before a route handler
MinHash	A compact signature for estimating set similarity
Mock provider	A rule-based stand-in for an LLM, used offline
Monaco	The code editor component from VS Code
Mongoose	The MongoDB object-modelling library for Node.js
MRR	Mean Reciprocal Rank
Namespace (Linux)	Isolation of what a process can see
NER	Named-Entity Recognition
NF4	A 4-bit quantization format for normally distributed weights
Nonce	A random value used once; marks genuine sandbox report lines
O*NET	The US occupational information database
ObjectId	MongoDB's document identifier type
Orchestrator	The component that runs agents in fixed workflows
Pagination	Returning a list one page at a time
PEFT	Parameter-Efficient Fine-Tuning
PII	Personally identifiable information
Planner	The deterministic module that builds a career plan
Prerequisite graph	Skills as nodes, "must learn first" as edges
Provenance	Whether data is DEMO, SYNTHETIC or REAL
Pydantic	The Python library for typed data validation
QLoRA	LoRA applied to a 4-bit quantized base model

Term	Meaning
RAG	Retrieval-Augmented Generation
Random Forest	An ensemble of decision trees that vote
Rank (LoRA)	The inner dimension of the adapter matrices
Rate limiting	Restricting how often a client may call an endpoint
RBAC	Role-Based Access Control
Recall@K	How often the right item is in the top K
Refresh token	A longer-lived token used to obtain a new access token
Relevance threshold	The minimum similarity for a retrieved chunk to be used
Result file	The file the harness writes the return value to
Sandbox	An isolated environment for running untrusted code
Schema	A formal description of the shape data must have
seccomp	A Linux filter restricting system calls
Semaphore	A counter that limits how many tasks run at once
SFT	Supervised Fine-Tuning
Shrinkage	A constant that damps similarities built on few observations
Signature (function)	A function's name, parameter types and return type
Socket.IO	The library used for real-time events
Spearman correlation	Agreement between two rankings
SSRF	Server-Side Request Forgery; making a server fetch a hostile URL
Starter code	The empty function a student begins with
Stratified split	A split that preserves class proportions
Taxonomy	A controlled list of skills and their aliases
Temperature	A setting controlling randomness in generation
tmpfs	An in-memory filesystem that disappears with the container
Token	The unit of text a language model reads
Topological order	An ordering in which every prerequisite comes first
Unsloth	A library that speeds up QLoRA training
Verdict	The judge's final result for a submission
vLLM	A server for running LLMs and LoRA adapters efficiently
Weak label	A training label produced by rules rather than humans
WebRTC	Browser technology for real-time audio and video
Zod	The TypeScript validation library used for requests

26. How to defend CareerPilot AI

26.1 What is strongest

- **The scoring is defensible.** Every score has a formula you can write on the board.
- **The separation of duties.** Facts from code, judgement from the model, arithmetic from code.
- **Failure is reported, not hidden.** "Not executed", "degraded", "no verified information".
- **Testing.** Automated suites in all three services; 112 reference programs judged correctly.
- **The data work.** Rejecting a provided score after inspection, leakage audits, held-out jobs and roles.

26.2 What is technically impressive

- Four language harnesses generated from one typed signature.
- A sandbox protocol in which expected outputs never enter the container and inputs are streamed one test at a time.
- Refresh-token rotation with family revocation.
- A resource verifier hardened against server-side request forgery.
- A career planner that respects prerequisites and tells you when a deadline is impossible.

26.3 What to emphasise

1. One readiness score from seven components, recomputed after every activity.
2. A language model here cannot set a score, invent a link, or decide whether code is correct.
3. The honest status of each model.

26.4 What to avoid claiming

Do not say	Say instead
"Our fine-tuned Qwen model scores resumes"	"The ATS score is rule-based. A fine-tuning pipeline is built; training has not been run."
"The AI generates the career roadmap"	"A deterministic planner generates it. A model was trained to reword it and was not good enough, so it is off."
"73.5% accurate at predicting placement"	"0.735 accuracy on a synthetic dataset; a pipeline check, not a real-world result."
"The sandbox is fully tested and secure"	"The judge logic is tested. Container isolation still has to be verified with Docker on this machine."
"It uses TF-IDF / BERT / semantic search"	"The default embedding is feature hashing, which is lexical. Semantic embedders can be switched on."
"It is faster than other platforms"	"No benchmark has been run."
"This is a novel algorithm"	"The contribution is integration and a strict design rule."

26.5 Limitations to admit without being asked twice

Weights not validated · classifier on synthetic data · both fine-tuned models off · mock LLM provider locally · sandbox not run under Docker · 28 problems · no human evaluation · no benchmarks.

26.6 Handling question types

The jury asks	How to answer
"Why…?"	State the requirement, the option chosen, one alternative, and why not. Example: "We needed repeatable scores, so rules; an LLM would vary between runs."
"How…?"	Walk the pipeline in order: input → steps → output. Name the file or formula if you can.
"What if…?"	Name the failure, then the fallback. Example: "If the model times out, the deterministic engine answers and the response is marked degraded."
"What is the novelty?"	"Not a new algorithm. One explainable readiness model fed by every module, with deterministic scoring kept separate from generative language."
Something you do not know	"I have not measured that" or "I would need to check the code for that detail". Then say how you would find out. Never guess a number.
A hostile framing	Agree with the true part first ("yes, most scoring is rule-based"), then give the reason it is a strength.

27. Final cheat sheet

27.1 The essentials

Item	Answer
One line	A placement-preparation platform that turns all of a student's preparation evidence into one explainable readiness score and a personal plan
Problem	Disconnected tools; no overall readiness view; unexplained scores
Solution	One account, one scoring engine (APIE), deterministic scores, AI for language only
Architecture	React SPA → Node.js/Express gateway (MongoDB, Socket.IO) → Python FastAPI AI service; Docker sandbox for code
Size	33 models, 135 endpoints, 38 pages, 11 agents
Roles	Student, Alumni, Placement Officer, Administrator
Frontend	React 18, TypeScript, Vite, Tailwind, TanStack Query, Monaco
Backend	Node.js, Express, Mongoose, Zod, JWT, Socket.IO, PDFKit
AI service	FastAPI, Pydantic, spaCy, PyMuPDF, scikit-learn, NumPy
Database	MongoDB 7; references by ObjectId; append-only readiness snapshots
Security	bcrypt, JWT 15 min + refresh 7 days with rotation, RBAC, Zod, rate limits, magic-byte upload checks, audit log, sandbox

27.2 Formulas to know by heart

```
ATS       = 9 capped criteria: skills 20, structure 15, experience 15, contact 10,
            education 10, impact 10, formatting 10, verbs 5, length 5
Job match = 0.55 required + 0.20 preferred + 0.15 similarity + 0.10 eligibility
APIE      = coding 0.25 + technical 0.20 + communication 0.15 + resume 0.10
            + HR 0.10 + aptitude 0.10 + academic 0.10, then bounded modifiers
Recency   = Σ(score_i / (i+1)) / Σ(1 / (i+1))
Bands     = <40 At Risk | 40–60 Needs Improvement | 60–80 Placement Ready | ≥80 Highly Ready
Mentor    = 0.35 company + 0.20 role + 0.20 skills + 0.15 gap + 0.05 rating + 0.05 availability
Priority  = importance×10 + 8 core + 6 per unblocked (cap 24) + 12 if ATS-missing + 4 per level
LoRA      = W + (α/r)·B·A     with r = 16, α = 32
```

27.3 Models

Model	Status	Number to quote
Rule-based ATS engine	In use	MAE 5.18, Spearman 0.873 against weak labels
APIE	In use	Weights not validated
Random Forest	In use as a second opinion	Accuracy 0.735, macro F1 0.737 — synthetic data
Skill recommender	Trained, not live	Recall@5 0.411 vs 0.272 baseline
Qwen3-4B ATS (QLoRA)	Not trained	None
Qwen3-1.7B career (QLoRA)	v1 trained, not usable, off	51.7% valid JSON, 0% fully valid
LLM for interviews	Provider interface; mock locally	None

27.4 DSA judge

Four languages (Python, C++, Java, JavaScript) · 28 problems, 16 tags · one container per submission · no network, read-only, non-root, memory/CPU/process/time limits · expected outputs stay on the host ·

seven verdicts · first failing test decides · 112 reference programs accepted with local compilers · Docker isolation still to be verified.

27.5 Training

QLoRA = frozen 4-bit base + rank-16 adapters, about 1% of parameters, fits a free T4. ATS data: 2,162 train / 470 validation / 485 test, weak labels, leakage audit 0/0/0. Career data: 7,157 / 957 / 1,802, two roles held out; v1 used 591 examples and 37 updates.

27.6 Innovation, in one sentence

One explainable readiness model fed by every module, with deterministic scoring kept strictly separate from generative language.

27.7 Future scope, in one sentence

Train and evaluate the models, validate the weights with real outcomes, collect human ratings, and verify and harden the sandbox.

28. "What is CareerPilot AI?" at four lengths

28.1 30-second answer

"CareerPilot AI is a placement-preparation platform. A student analyses their resume, practises coding and interviews, and gets a personal career plan, all on one account. Every activity feeds one engine that produces an explainable Placement Readiness Score and tells the student what to fix first. Scores come from deterministic algorithms; AI is used only for language."

28.2 1-minute answer

"Students preparing for placement use many disconnected tools and cannot see their overall readiness. CareerPilot AI brings resume analysis, job matching, assessments, DSA practice, mock interviews, a career plan and alumni mentorship into one platform. It has a React frontend, a Node.js gateway with MongoDB, and a Python AI service. A rule-based engine scores the resume on nine criteria, a weighted engine called APIE combines seven components into a readiness score, and a deterministic planner builds a study plan from the student's skill gaps. A language model writes interview questions and evaluates answers, but every final score is recomputed in code, so results are repeatable and explainable."

28.3 3-minute answer

Start with the 1-minute answer, then add:

"Three design decisions define the project. First, a single trust boundary: the browser talks only to the gateway, which alone calls the AI service. Second, a strict rule for AI: facts from code, judgement from the model, arithmetic from code. A model cannot set a score, cannot invent a link because it may only choose verified resource IDs, and cannot decide whether code is correct because a deterministic judge does that. Third, honest failure: if the code sandbox is unavailable the system says the code was not run; if retrieval finds nothing relevant it says so instead of answering.

For the DSA environment, a student writes one function in Python, C++, Java or JavaScript. The backend generates the surrounding harness from a typed signature and runs it in a throw-away container with no network and strict limits. Expected outputs never enter the container.

We also built QLoRA fine-tuning pipelines for two Qwen models. To be accurate: the ATS model has not been trained yet, and the first career model was measured and found not usable, so both are off and the deterministic engines serve users."

28.4 5-minute answer

Use the five-minute presentation script in the presentation chapter.

29. Top 20 things to remember tomorrow

1. **One line:** deterministic algorithms compute every score; language models are used only for language.
2. **Three services:** React frontend, Node.js/Express gateway, Python FastAPI AI service. The browser never calls the AI service.
3. **Numbers:** 33 models, 135 endpoints, 38 pages, 11 agents, 4 active roles.
4. **ATS = nine rule-based criteria summing to 100.** No LLM.
5. **APIE weights:** coding 0.25, technical 0.20, communication 0.15, resume, HR, aptitude and academic 0.10 each. **Not validated.**
6. **Job match:** 0.55, 0.20, 0.15, 0.10 — and eligibility is a hard gate.
7. **Random Forest: accuracy 0.735 on synthetic data.** Always say "synthetic".
8. **The ATS fine-tuned model is NOT trained.** Pipeline and integration are built.
9. **The career model v1 is NOT usable** (0% fully valid plans) and is switched off.
10. **The career plan comes from a deterministic planner:** gaps → priority → prerequisite order → time budget → verified resources.
11. **QLoRA = frozen 4-bit base + rank-16, alpha-32 adapters, about 1% trained**, chosen because only a free T4 was available.
12. **Resources by ID, never by URL.** 270 in the catalogue; videos via the YouTube Data API.
13. **DSA: four languages, 28 problems, seven verdicts.** C is not supported.
14. **Student code runs only in a container:** no network, read-only, non-root, limits. Never on the server.
15. **Hidden tests and expected outputs never enter the sandbox** and never reach the browser.
16. **The Docker sandbox has not been run on this machine yet.** The judge logic was verified with local compilers.
17. **The local LLM setting is the mock provider**, which is not a language model.
18. **No TF-IDF, no BERT, no queue, no Redis, no faculty role.** Do not mention them as present.
19. **No benchmark, no human evaluation, no real-outcome validation.** Say "not yet measured" and how you would measure it.
20. **If you do not know, say so.** "I have not measured that" is a stronger answer than a guessed number.

30. Factual validation report

This report records how each claim in the document was checked against the repository on 8 October 2026. "Verified" means the value was read from source code, a data file or a test run during this audit.

30.1 Verified

Claim	Evidence
Three services: React/Vite frontend, Node/Express backend, Python/FastAPI AI service	`frontend/`, `backend/`, `ai-service/`, root `package.json` workspaces
33 Mongoose models	`backend/src/models/` (33 model files)
135 endpoints in 15 route files, mounted on 19 paths	Counted from `backend/src/routes/v1/*.ts` and `routes/index.ts`
38 page components; 35 lazy-loaded	`frontend/src/pages/`, `React.lazy` calls in `App.tsx`
Roles: STUDENT, ALUMNI, PLACEMENT_OFFICER, ADMIN; RECRUITER reserved	`shared/src/enums.ts`
11 registered agents; workflows `RESUME_TO_MATCH`, `INTERVIEW_TURN`	Printed from the orchestrator registry
Dependencies and versions in the stack table	`backend/package.json`, `frontend/package.json`, `ai-service/requirements.txt`
ATS: nine criteria and their formulas	`ai-service/app/services/ats.py`, `docs/ALGORITHMS.md`
Skill taxonomy: 107 skills, 181 aliases, custom boundaries	`ai-service/app/nlp/skills_taxonomy.py`
APIE weights 0.25 / 0.20 / 0.15 / 0.10 × 4	`shared/src/constants.ts`; live `/readiness` response shows seven components
Job-match and mentor-match formulas	Agent source files
RAG: chunk 900/150, top-k 5, thresholds 0.18 / 0.25	`ai-service/app/core/config.py`
Random Forest metrics (0.735, 0.740, 0.737, 0.737, CV 0.739 ± 0.020) and confusion matrix	`ai-service/artifacts/readiness_metrics.json`
Skill recommender metrics and confidence interval	`ai-service/artifacts/skill_recommender.json`
ATS dataset sizes, leakage audit, baseline table	`careerpilot_ats/README.md`, `evaluation/metrics.json`
ATS training settings (r 16, α 32, LR 1e-4, 1 × 16, 3,072 tokens, 2 epochs)	`careerpilot_ats/README.md`
No ATS adapter exists	`careerpilot_ats/model/` is empty
Career adapter config: LoRA r 16, α 32, dropout 0.05, base `unsloth/Qwen3-1.7B-unsloth-bnb-4bit`	`careerpilot_career/models/career_roadmap/final/adapter_config.json`
Career v1 run: 591 examples, 1 epoch, LR 1e-4, max length 4,096, fp16	`final/model_metadata.json`
Career v1 results: 51.7% valid JSON, 6.7% schema, 0% fully valid; base on 8 examples	`evaluation/evaluation_report.json`
Career knowledge: 122 skills, 123 edges, depth 6, 24 roots, 20 roles, 80.7% corroborated	`ai-service/app/career/data/*.json`
Resource catalogue: 270 total, 239 verified, 29 reachable, 2 unreachable; 112 of 122 skills covered	`resources.json` summary
Resource ranking formula	`ai-service/app/career/resources.py`
Skill-gap priority formula and thresholds 62 / 40	`ai-service/app/career/skill_gap.py`
Planner time budget 75 / 15 / 10 and deferral rules	`ai-service/app/career/planner.py`

Claim	Evidence
DSA: 28 problems, 16 tags, four languages, defaults 2 s / 128 MB	`database/seeds/dsaProblems.ts`, `shared/src/dsa.ts`; live API returned 28
Docker isolation flags	`backend/src/services/execution/dockerRunner.ts`; asserted by a unit test
Time factors ×1 / ×2 / ×2 / ×3; per-language memory strategy	`execution/languages.ts`
Hidden tests `select: false`; never in responses	`models/CodingProblem.ts`; `tests/dsa.api.test.ts`
112 reference programs accepted	Full-matrix run of `tests/dsa.judge.test.ts`
Concurrency cap, no queue	`codeExecution.service.ts` (Semaphore)
Security middleware: Helmet, CORS, compression, sanitiser, rate limiters	`backend/src/app.ts`, `middleware/`
Refresh token in a signed cookie; rotation	`controllers/auth.controller.ts`, `services/auth.service.ts`
Upload checks including magic bytes	`middleware/upload.ts`, `services/storage.service.ts`
Custom job description for resume analysis	`resume.service.ts`, `ResumePage.tsx`; live test returned 30% coverage on a sample JD
Compose services: mongodb, ai-service, backend, frontend; judge disabled there	`docker-compose.yml`
Local provider settings: `AI_PROVIDER=mock`, `EMBEDDING_PROVIDER=hash`, `VECTOR_DB_PROVIDER=numpy`	`.env` (names and non-secret values only)
Test results: backend 128, AI service 546, frontend 39 — all passing	Suites run during this audit
End-to-end runs: 61, 19 and 25 checks passing	Run on 7 October 2026 against the live AI service

30.2 Partially verified

Claim	What is verified	What is not
DSA sandbox security	Argument list, protocol, verdict logic, local execution	Behaviour under real Docker — Docker is not installed on the development machine
Memory Limit Exceeded	Classification logic (unit tests)	Real enforcement, which needs the container
ATS fine-tuning pipeline	Dataset build, baselines, integration, validation layer, tests	Training and any LLM result
Career fine-tuning	One short run and its evaluation	A run long enough to be usable
Voice interview	API path and identical scoring for spoken and typed input (end-to-end run)	Browser speech recognition itself cannot be exercised from Node
Career Plan page	Builds, type-checks, API tested end to end	Not reviewed screen by screen in a browser during this audit
ESCO usage	Used for corroboration	Its reuse licence is not verified
Trainable parameter count	About 33 million for the ATS model (documented)	Not recorded for the career model

30.3 Not verified from source

Item	Note
Any production or cloud deployment	Only Docker Compose and local development exist
CI pipeline, monitoring, alerting	Not present
Encryption at rest	Not implemented in the repository

Item	Note
Performance figures of any kind	No benchmark exists
Comparison of Qwen with Llama, Mistral or Gemma	Never run
Which LLM would answer in the demo	Depends on `AI_PROVIDER` at the time; the local value is `mock`
Human agreement with any score	Review sheets are empty

30.4 Planned (not built)

Item	Where it is referenced
Recruiter module	Role constant only
Trained ATS model and its evaluation	Notebook ready; needs a GPU
Career model v2	Notebook updated; not run
Judge queue and workers; stricter seccomp profile	`docs/DSA_JUDGE.md` limitations
Admin problem editor; custom test input; C language	Not present
vLLM serving of adapters	Documented commands only
Validation of APIE weights with real outcomes	Limitations

30.5 Corrections made while validating

- The existing viva guide listed 15 agents and older classifier metrics; the current values are 11 agents and accuracy 0.735. `docs/ALGORITHMS.md` was corrected.
- An older career README states that 17 skills have no verified resource; the current catalogue summary shows 10 (112 of 122 covered). This document uses the data file.
- Older documents mention Group Discussion and a Career Roadmap agent; both were removed from the product and are not described as features here.

30.6 Things this document deliberately does not contain

- No API key, password, token or `.env` secret. Secret names appear with their values [REDACTED].
- No invented accuracy, precision, recall, F1, latency or speed-up figure.
- No claim that a fine-tuned model is deployed.